AI / LLM / Agents / Hardware Notes

Foundations · Architecture · Systems · Frontiers

Jeriel Chan

March 19, 2026

Contents

I Foundations **9**

1 Math & Probability Refresher **10**
1.1 Linear algebra essentials . . . 10
1.2 Probability & information theory . . . 10
1.3 Calculus & gradients . . . 10
1.4 Statistics for ML . . . 10

2 Classical ML & Optimization **10**
2.1 Supervised learning fundamentals . . . 10
2.2 Gradient descent variants . . . 10
2.3 Convex vs non-convex optimization . . . 10
2.4 SVMs, trees, ensembles . . . 10

3 Neural Networks from Scratch **10**
3.1 Perceptrons & MLPs . . . 10
3.2 Activation functions . . . 11
3.3 Initialization strategies . . . 11
3.4 Universal approximation theorem . . . 12
3.5 Convolutional Neural Networks . . . 12
3.6 RNNs, LSTMs & GRUs . . . 13

4 Backpropagation & Autodiff **15**
4.1 Computational graphs . . . 15
4.2 Forward & reverse mode autodiff . . . 15
4.3 Vanishing & exploding gradients . . . 15
4.4 PyTorch autograd internals . . . 15

5 Regularization, Loss Functions & Training Tricks **15**
5.1 L1, L2, dropout, batch norm . . . 15
5.2 Loss function zoo . . . 15
5.3 Learning rate schedules . . . 15
5.4 Mixed precision training . . . 15

II LLM Architecture **15**

6 Transformer Architecture Deep Dive **16**

6.1 Encoder-decoder vs decoder-only . . . 16
6.2 Multi-head self-attention . . . 17
6.3 Feed-forward sublayer . . . 18
6.4 Layer norm & residual stream . . . 19

7 Attention Mechanisms **22**
7.1 Scaled dot-product attention . . . 22
7.2 Multi-head attention (MHA) . . . 22
7.3 Multi-query & grouped-query attention (MQA, GQA) . . . 22
7.4 Sparse & linear attention variants . . . 23
7.5 FlashAttention . . . 23

8 Positional Encodings **25**
8.1 Sinusoidal encodings (original) . . . 25
8.2 Learned absolute positions . . . 25
8.3 Relative position encodings . . . 25
8.4 RoPE (rotary position embedding) . . . 25
8.5 ALiBi . . . 25

9 Tokenization & Vocabulary **25**
9.1 BPE, WordPiece, Unigram . . . 25
9.2 SentencePiece & tiktoken . . . 26
9.3 Vocabulary size trade-offs . . . 27
9.4 Multilingual tokenization . . . 27

10 Scaling Laws & Emergent Capabilities **29**
10.1 Chinchilla scaling laws . . . 29
10.2 Compute-optimal training . . . 29
10.3 Emergent abilities & phase transitions . . . 29
10.4 Scaling inference compute . . . 29

11 Mixture of Experts (MoE) **29**
11.1 Sparse MoE fundamentals . . . 29
11.2 Routing mechanisms . . . 29
11.3 Load balancing losses . . . 29
11.4 MoE in practice (Mixtral, GPT-4 rumours) . . . 29

12 State Space Models: Mamba & RWKV **29**
12.1 S4 & structured state spaces . . . 29
12.2 Mamba & selective SSMs . . . 29
12.3 RWKV: RNN-transformer hybrid . . . 30
12.4 SSM vs attention trade-offs . . . 30

III Hardware & Systems **31**

13 GPU Architecture **33**
13.1 SM, warp, thread hierarchy . . . 33
13.2 Tensor cores & TF32/BF16 . . . 33
13.3 Memory: HBM, L2, SRAM . . . 33
13.4 CUDA programming model . . . 33
13.5 NVLink & GPU interconnects . . . 33

14 TPUs, NPUs & Custom ASICs **33**
14.1 Google TPU architecture . . . 33

14.2 Systolic arrays . . . 33
14.3 Apple Neural Engine . . . 33
14.4 Groq LPU & Cerebras WSE . . . 33
14.5 Custom ASIC design trade-offs . . . 33

15 Memory Hierarchy & Bandwidth Bottlenecks **33**
15.1 Roofline model . . . 33
15.2 Arithmetic intensity . . . 33
15.3 Memory-bound vs compute-bound ops . . . 33
15.4 HBM bandwidth & capacity limits . . . 33

16 CUDA Kernels for ML **33**
16.1 Matrix multiplication (GEMM) . . . 33
16.2 Softmax & layer norm kernels . . . 33
16.3 Fused attention kernel . . . 33
16.4 Shared memory tiling . . . 33
16.5 Profiling with Nsight . . . 33

17 Triton & Custom Op Authoring **33**
17.1 Triton language overview . . . 33
17.2 Writing a fused kernel in Triton . . . 33
17.3 Triton vs CUDA trade-offs . . . 33
17.4 Integration with PyTorch . . . 33

18 Profiling & Optimization Workflows **33**
18.1 PyTorch Profiler & trace viewer . . . 33
18.2 Nsight Systems & Compute . . . 33
18.3 Bottleneck identification workflow . . . 33
18.4 Operator fusion strategies . . . 33

IV Training at Scale **33**

19 Pretraining Pipelines & Data Curation **34**
19.1 Web-scale data collection . . . 34
19.2 Deduplication & quality filtering . . . 34
19.3 Data mixing strategies . . . 34
19.4 Pretraining objectives (CLM, MLM, etc.) . . . 34

20 Distributed Training **35**
20.1 Data parallelism (DDP) . . . 35
20.2 Tensor parallelism . . . 35
20.3 Pipeline parallelism . . . 35
20.4 FSDP & ZeRO optimizer . . . 36
20.5 3D parallelism & Sequence/Context parallelism . . . 36

V Post-Training & Alignment **36**

21 Fine-tuning: SFT, LoRA, QLoRA **37**
21.1 Supervised fine-tuning (SFT) . . . 37
21.2 Parameter-efficient fine-tuning (PEFT) . . . 37
21.3 LoRA & AdaLoRA . . . 38
21.4 QLoRA & quantized fine-tuning . . . 38

22 RLHF, DPO & Reward Modelling **40**
22.1 Human preference collection . . . 40
22.2 Reward model training . . . 40
22.3 PPO for language models . . . 40
22.4 Direct preference optimization (DPO) . . . 41
22.5 GRPO & alternatives . . . 41

23 Constitutional AI & RLHF Safety **43**
23.1 Constitutional AI (Anthropic) . . . 43
23.2 RLHF safety properties . . . 43
23.3 Red-teaming methodologies . . . 43
23.4 Refusal & harmlessness trade-offs . . . 43

VI Inference & Deployment **43**

24 Inference & Decoding Strategies **44**
24.1 Greedy, beam search, sampling . . . 44
24.2 Temperature, top-k, top-p . . . 44
24.3 KV cache mechanics . . . 45
24.4 Speculative decoding . . . 45
24.5 Continuous batching . . . 46
24.6 Real-time action chunking for robot deployment . . . 46

25 Test-time Compute & Reasoning Models **48**
25.1 Scaling inference compute . . . 48
25.2 OpenAI o1 / o3 architecture (known) . . . 48
25.3 Process reward models (PRMs) . . . 48
25.4 Best-of-N & self-consistency . . . 49
25.5 Thinking tokens & scratchpads . . . 49

26 Quantization & Model Compression **50**
26.1 Post-training quantization (PTQ) . . . 50
26.2 Quantization-aware training (QAT) . . . 50
26.3 GPTQ, AWQ, GGUF formats . . . 50
26.4 Pruning & distillation . . . 51

27 Inference Optimization on Hardware **52**
27.1 FlashAttention 1, 2, 3 . . . 52
27.2 Paged KV cache (vLLM) . . . 52
27.3 Continuous batching engines . . . 52
27.4 TensorRT & compilation . . . 52

28 Running LLMs On-device **52**
28.1 llama.cpp & GGUF . . . 52
28.2 Ollama & LM Studio . . . 52
28.3 MLX (Apple Silicon) . . . 52
28.4 Quantization for edge . . . 52
28.5 Privacy & latency trade-offs . . . 52

29 Embedded AI & Microcontroller Inference **52**
29.1 TensorFlow Lite & TFLite Micro . . . 52
29.2 ONNX Runtime on ARM . . . 52
29.3 Keyword spotting pipelines . . . 52
29.4 Power & memory constraints . . . 52

30 Edge Deployment **52**
30.1 Model pruning for edge . . . 52
30.2 Post-training quantization (INT4, INT8) . . . 52
30.3 ONNX export & optimization . . . 52
30.4 Neural Architecture Search (NAS) . . . 52

VII Prompting & Retrieval **52**

31 Prompt Engineering & In-context Learning **53**
31.1 Zero-shot & few-shot prompting . . . 53
31.2 Chain-of-thought (CoT) . . . 53
31.3 Structured output prompting . . . 54
31.4 System prompts & jailbreaks . . . 54

32 RAG & Long-context Retrieval **55**
32.1 Naive RAG pipeline . . . 55
32.2 Embedding models & vector DBs . . . 55
32.3 Chunking strategies . . . 55
32.4 Retrieval methods: dense, sparse, hybrid . . . 56
32.5 Reranking & advanced RAG . . . 56
32.6 Long-context vs retrieval trade-offs . . . 57

VIII Agent Systems **57**

33 AI Agent Fundamentals & Architectures **58**
33.1 Agent loop: perceive, plan, act . . . 58
33.2 ReAct framework . . . 58
33.3 Reflexion & self-critique . . . 58
33.4 Agent scaffolding patterns . . . 58

34 Tool Use, Function Calling & MCP **58**
34.1 Function calling APIs . . . 58
34.2 Model Context Protocol (MCP) . . . 58
34.3 Tool schema design . . . 59
34.4 Error handling in tool loops . . . 59

35 Agent Memory: Episodic, Semantic, Working **60**
35.1 In-context (working) memory . . . 60
35.2 External episodic memory . . . 60
35.3 Semantic memory via vector DBs . . . 60
35.4 Context engineering . . . 61
35.5 Memory consolidation strategies . . . 62

36 Planning: MCTS, CoT, o1-style Reasoning **63**
36.1 Classical planning (STRIPS, PDDL) . . . 63
36.2 Monte Carlo Tree Search (MCTS) . . . 63
36.3 Chain-of-thought & scratchpads . . . 63
36.4 ReAct & agent planning loops . . . 63
36.5 Plan-and-Execute . . . 64
36.6 o1 / o3 test-time compute scaling . . . 64

37 Multi-agent Systems & Orchestration **64**
37.1 Orchestrator-subagent patterns . . . 64

37.2 AutoGen, CrewAI, LangGraph & MetaGPT . . . 65
37.3 Inter-agent communication . . . 66
37.4 Routers, state machines & workflow engines . . . 66
37.5 Swarm intelligence analogies . . . 67

IX Agent Platforms & Tooling 67

38 OpenClaw & Agent OS Integration 68
38.1 OpenClaw architecture overview . . . 68
38.2 OS-level agent primitives . . . 68
38.3 Filesystem & process access . . . 69
38.4 Security sandboxing . . . 69
38.5 Desktop automation patterns . . . 70

39 AI Coding Agents & IDEs 71
39.1 Landscape overview . . . 71
39.2 Cursor: architecture & edit model . . . 71
39.3 GitHub Copilot & Codex . . . 71
39.4 Windsurf (Codeium) . . . 71
39.5 Antigravity . . . 71
39.6 Devin & SWE-agent . . . 71
39.7 Aider & CLI coding agents . . . 71
39.8 Claude Code remote control . . . 72
39.9 How AI IDEs use context (repo map, RAG) . . . 72
39.10 Diff application & conflict resolution . . . 72
39.11 Evaluation: SWE-bench, Aider Polyglot & coding benchmarks . . . 73

40 Agentic Platforms: Manus, Computer Use & Beyond 75
40.1 Manus AI overview (Butterfly Effect) . . . 75
40.2 NVIDIA NemoClaw / OpenShell . . . 75
40.3 SaaS → AaaS: the agentic transformation . . . 76
40.4 Anthropic Computer Use . . . 76
40.5 OpenAI Operator & Agents SDK . . . 77
40.6 Browser-use & web automation . . . 77
40.7 Desktop agent architectures . . . 77
40.8 Security & sandboxing for autonomous agents . . . 77
40.9 Meta's agent research (LLaMA, CICERO) . . . 78

X Multimodal & World Models 78

41 Vision-Language Models 79
41.1 CLIP & contrastive pretraining . . . 79
41.2 LLaVA, Flamingo, PaLI . . . 79
41.3 Vision tokens & patch embeddings . . . 79
41.4 Grounding & spatial reasoning . . . 79

42 Diffusion Models & Generative Vision 79
42.1 GANs & VAEs (generative predecessors) . . . 79
42.2 DDPM & DDIM . . . 80
42.3 Score matching & SDE perspective . . . 81
42.4 Stable Diffusion architecture . . . 82
42.5 ControlNet, LoRA for images . . . 83
42.6 Flow matching . . . 84

43 Audio, Speech & Multimodal Fusion **86**
43.1 Whisper & speech recognition . . . 86
43.2 Audio tokenization . . . 86
43.3 TTS & voice cloning . . . 86
43.4 Multimodal fusion strategies . . . 86

44 World Model Theory **86**
44.1 LeCun's JEPA architecture . . . 86
44.2 Dreamer (RSSM) . . . 88
44.3 World models in RL . . . 89
44.4 Predictive coding theories . . . 90

45 Video & Predictive Models **93**
45.1 Sora & video diffusion . . . 93
45.2 Genie & interactive world models . . . 94
45.3 Temporal consistency challenges . . . 95
45.4 Next-frame prediction as pretraining . . . 97

46 Embodied AI & Simulation **99**
46.1 SLAM & robot perception . . . 99
46.2 Sim-to-real transfer . . . 100
46.3 RT-2, PaLM-E, embodied LLMs . . . 100
46.4 Project GR00T & foundation models for robotics . . . 101
46.5 Simulation environments (Isaac, MuJoCo) . . . 102
46.6 Manipulation & locomotion tasks . . . 103
46.7 Humanoid loco-manipulation & VLA models . . . 104
46.8 HumanDex: IMU-based whole-body teleoperation . . . 106

XI Safety & Interpretability **107**

47 AI Safety Landscape & Threat Models **108**
47.1 Alignment problem overview . . . 108
47.2 Reward hacking & Goodhart's law . . . 108
47.3 Deceptive alignment . . . 108
47.4 Containment & control . . . 108
47.5 Training-time safety: data exclusion & perplexity verification . . . 108

48 Interpretability & Mechanistic Analysis **110**
48.1 Probing classifiers . . . 110
48.2 Superposition hypothesis . . . 110
48.3 Sparse autoencoders (SAE) . . . 110
48.4 Circuit-level analysis . . . 111
48.5 Activation patching . . . 111

XII Evaluation **112**

49 Evaluation Methodology & Benchmark Design **113**
49.1 Evaluation taxonomy . . . 113
49.2 Contamination & data leakage . . . 113
49.3 Long-context evaluation: needle-in-a-haystack . . . 114
49.4 Human vs automated evaluation . . . 115
49.5 Statistical significance in evals . . . 116

50 Standard Benchmarks **117**
50.1 MMLU, HellaSwag, ARC . . . 117
50.2 HumanEval, SWE-bench . . . 117
50.3 MT-Bench & Chatbot Arena . . . 117
50.4 MATH, GSM8K . . . 117
50.5 GPQA & frontier benchmarks . . . 117

51 Evals for Agents & Agentic Tasks **117**
51.1 WebArena, OSWorld . . . 117
51.2 AgentOps & observability . . . 117
51.3 SWE-bench Verified . . . 118
51.4 Tool-use evals . . . 118
51.5 Long-horizon task evaluation . . . 118
51.6 Safety evals for agents . . . 119
51.7 Agent failure modes . . . 119

XIII Frontiers **119**

52 Neurosymbolic AI & Program Synthesis **120**
52.1 Neural-symbolic integration . . . 120
52.2 Program induction . . . 121
52.3 AlphaCode & code generation . . . 123
52.4 Formal verification + LLMs . . . 124

53 Research Reading Log & Paper Notes **126**
53.1 How to use this chapter . . . 126
53.2 Paper template . . . 126
53.3 Reading list . . . 126

XIV Reference **128**

54 University AI Modules **129**
54.1 CS2109S — AI and Machine Learning . . . 129
54.2 CS3263 — Foundations of AI . . . 129
54.3 CS4246 — AI Planning and Decision Making . . . 129
54.4 MA4275 — Mathematics for Reinforcement Learning . . . 129

55 General-purpose LLM + Agent Map **130**
55.1 How to use this chapter . . . 130
55.2 Where each concept belongs in this repo . . . 130
55.3 Distillation vs Quantization (high-value distinction) . . . 130
55.4 Distillation: practical trade-offs . . . 130
55.5 Quantization: practical intuition + numeric example . . . 131
55.6 Minimal glossary (highest-yield) . . . 131
55.7 Agent types taxonomy . . . 131
55.8 Agent architecture components . . . 132
55.9 One mental model . . . 132

Part I

Foundations

Math & Probability Refresher

Linear algebra essentials

Probability & information theory

Calculus & gradients

Statistics for ML

Classical ML & Optimization

Supervised learning fundamentals

Gradient descent variants

Convex vs non-convex optimization

SVMs, trees, ensembles

Neural Networks from Scratch

Perceptrons & MLPs

The **perceptron** is the simplest neural network unit—a linear classifier. Given input $\mathbf{x} \in \mathbb{R}^d$, it computes:

$$\text{output} = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

where $\mathbf{w}$ is a weight vector and b is a bias term. The perceptron can only separate linearly separable data; it famously fails on the **XOR problem**, where no linear decision boundary exists.

Intuition

The XOR problem demonstrates that a single perceptron cannot learn a function whose decision boundary is non-linear. This limitation motivates the use of multiple layers.

A **Multilayer Perceptron (MLP)** stacks multiple layers of neurons to overcome this limitation. The forward pass for layer l is:

$$\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right)$$

where σ is a non-linear activation function, $\mathbf{W}^{(l)}$ is the weight matrix, and $\mathbf{b}^{(l)}$ is the bias vector. The entire forward pass is a composition of affine transformations and non-linearities:

$$\text{output} = \sigma_L(\mathbf{W}^{(L)}\sigma_{L-1}(\cdots\sigma_1(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})\cdots) + \mathbf{b}^{(L)})$$

Note

Depth vs. Width: Depth (number of layers) allows hierarchical feature composition and enables more complex function representations with fewer parameters. Width (neurons per layer) increases representational capacity but is less efficient—exponential function classes may require polynomial width but only logarithmic depth.

Activation functions

Non-linear activation functions are essential to MLPs; without them, stacking layers produces only a linear transformation. Here are the most common choices:

Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$, range $(0, 1)$. Historically popular but suffers from **vanishing gradients** for large $|x|$, since $\sigma'(x) \approx 0$ at extremes. Also not zero-centered, causing inefficient gradient updates.

Tanh: $\tanh(x)$, range $(-1, 1)$. Zero-centered and slightly better than sigmoid, but still prone to vanishing gradients.

ReLU: $\mathrm{ReLU}(x) = \max(0, x)$. No vanishing gradients for $x > 0$ and computationally cheap (just a comparison). However, suffers from the **dying ReLU problem**—neurons can become stuck outputting zero if the pre-activation is consistently negative, then gradient flow is blocked.

Leaky ReLU / PReLU: $f(x) = \max(\alpha x, x)$ where α is a small constant (e.g., 0.01). Addresses the dying ReLU problem by allowing a small negative slope.

GELU: $x \cdot \Phi(x)$, where Φ is the cumulative distribution function of the standard normal distribution. A smooth, differentiable approximation to $\mathrm{ReLU} \cdot x$. Used in transformer-based models (BERT, GPT).

SiLU / Swish: $x \cdot \sigma(x)$. A self-gated, smooth activation that has gained popularity in modern architectures, including large language models.

SwiGLU: A gated variant combining Swish with a learnable linear gate. Used in modern large language models like LLaMA as the activation function in feed-forward layers.

Vanishing Gradients: Sigmoid and tanh have derivative bounds that shrink in magnitude as layers deepen, causing gradients to exponentially decay during backpropagation. This made training deep networks difficult before the advent of ReLU and careful initialization.

Initialization strategies

Initializing weights carefully is crucial for training neural networks. Poor initialization can cause vanishing or exploding gradients.

Intuition

The motivation is to keep the activation distribution roughly constant across layers. If activations grow exponentially, gradients explode; if they shrink, gradients vanish.

Xavier/Glorot Initialization (suitable for sigmoid/tanh):

$$\mathbf{W}^{(l)} \sim \mathrm{Uniform}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

where n_{in} and n_{out} are the input and output dimensions of layer l. This maintains roughly constant variance of activations across layers.

He/Kaiming Initialization (designed for ReLU):

$$\mathbf{W}^{(l)} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

The factor of 2 accounts for the fact that ReLU zeros out half the inputs on average, so we need higher variance to compensate.

Orthogonal Initialization: Weights are initialized as random orthogonal matrices. Useful for recurrent networks and other settings where controlling gradient flow through matrix multiplications is critical.

- Glorot & Bengio (2010): Understanding the difficulty of training deep feedforward neural networks
- He et al. (2015): Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification

Universal approximation theorem

The **Universal Approximation Theorem** (Hornik 1989, Cybenko 1989) states:

Note

A single hidden-layer MLP with a non-linear activation function and sufficiently many neurons can approximate any continuous function on a compact set to arbitrary precision.

This is a powerful theoretical result, but it comes with important caveats:

- It does *not* specify how many neurons are needed (often exponential in the input dimension).
- It does *not* guarantee that the function is learnable via gradient descent—only that a solution exists.
- It does *not* imply that shallow networks are always sufficient; depth can exponentially reduce the required size.

Intuition

Practical Implication: While shallow networks are theoretically expressive, *depth is preferred over width* in practice. Deep networks can represent certain function classes (e.g., polynomials of degree k) with exponentially fewer parameters than shallow networks. This is why modern deep learning uses many layers.

Convolutional Neural Networks

Images and other spatial data have inherent structure: local regions are highly correlated, and patterns repeat across space (translation invariance). Fully connected layers are parameter-inefficient for such data.

The Convolution Operation: A learnable filter (or kernel) slides over the input image, computing dot products. If the input is $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ and the kernel is $\mathbf{K} \in \mathbb{R}^{k_h \times k_w \times C}$, the output feature map at position (i, j) is:

$$\text{output}[i, j] = \sum_{h=0}^{k_h-1} \sum_{w=0}^{k_w-1} \sum_{c=0}^{C-1} \mathbf{K}[h, w, c] \cdot \mathbf{X}[i + h, j + w, c]$$

Key Properties:

Local Receptive Fields: Each neuron in a convolutional layer sees only a small patch of the input (determined by kernel size), reducing parameters and enabling hierarchical feature learning.

Weight Sharing: The same kernel is applied at every position, further reducing parameters and introducing translation equivariance—if the input shifts, the feature map shifts in the same way.

Translation Equivariance: Convolutions preserve the spatial structure of translation.

Pooling: After convolution, pooling layers downsample the spatial resolution:

- **Max pooling**: Outputs the maximum activation in each local window.
- **Average pooling**: Outputs the mean activation in each window.

Pooling reduces parameters and computation, and adds a degree of translation invariance.

Typical CNN Architecture:

$$[\text{Conv} \to \text{BatchNorm} \to \text{ReLU} \to \text{Pool}]^* \to \text{Flatten} \to \text{FC layers}$$

Convolutional blocks extract spatial features; later layers flatten and perform classification.

Receptive Field Growth: Deeper layers see larger regions of the original input. A single neuron in layer l integrates information from an increasingly large receptive field as l increases, enabling hierarchical composition of features.

AlexNet (2012): The first deep CNN to significantly outperform prior methods on ImageNet classification. AlexNet sparked the deep learning revolution, demonstrating that large, trained-on-GPU convolutional networks could achieve superhuman performance on complex visual tasks.

ResNets (Residual Networks): Skip connections allow training much deeper networks (50–152 layers). A residual block computes:

$$\mathbf{x}_{l+1} = \mathbf{x}_l + F(\mathbf{x}_l, \mathbf{W}_l)$$

instead of the standard $\mathbf{x}_{l+1} = F(\mathbf{x}_l, \mathbf{W}_l)$. The residual connection allows gradients to flow directly through addition, sidestepping vanishing gradient problems in very deep networks.

- LeCun et al. (1998): Gradient-based learning applied to document recognition
- Krizhevsky et al. (2012): ImageNet Classification with Deep Convolutional Neural Networks (AlexNet)
- He et al. (2016): Deep Residual Learning for Image Recognition (ResNets)

RNNs, LSTMs & GRUs

Recurrent Neural Networks (RNNs) process sequences of arbitrary length by maintaining a hidden state that is updated at each time step:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

The same weights $\mathbf{W}_h$ and $\mathbf{W}_x$ are applied at every time step, allowing the network to share parameters across time and handle variable-length sequences.

The Vanishing Gradient Problem: When backpropagating through time (BPTT), gradients are multiplied by $\mathbf{W}_h$ at each step. If $\|\mathbf{W}_h\| < 1$, gradients decay exponentially as we backprop further into the past, making it difficult to learn long-range dependencies. Conversely, if $\|\mathbf{W}_h\| > 1$, gradients explode.

Long Short-Term Memory (LSTM) (Hochreiter & Schmidhuert, 1997) addresses vanishing gradients via a **cell state** $\mathbf{c}_t$ and three gating mechanisms:

Forget Gate: $\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$ — controls how much of the previous cell state to retain.

Input Gate & Candidate: $\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$ and $\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$ — determine what new information to add.

Output Gate: $\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$ — controls which parts of the cell state to output.

The cell state and hidden state are updated as:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

where $\odot$ is element-wise multiplication. Crucially, the cell state is updated via addition (not overwriting), allowing gradients to flow directly through the cell without multiplicative interactions.

GRU (Gated Recurrent Unit): A simplified variant of the LSTM with only two gates (reset and update). GRUs are computationally more efficient and often achieve similar performance to LSTMs.

Bidirectional RNNs: Run a forward RNN and a backward RNN over the sequence, then concatenate their outputs. Bidirectional processing allows each position to attend to context from both past and future, and is standard in encoder-only architectures (BERT-era models).

Note on Modern Usage: While LSTMs and GRUs revolutionized sequence modeling, they are largely *superseded by Transformer-based architectures* for NLP tasks. However, RNNs remain useful for time-series analysis, streaming applications, and scenarios where inference latency is critical.

- Hochreiter & Schmidhuber (1997): Long Short-Term Memory
- Chung et al. (2014): Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling
- Graves (2013): Generating Sequences With Recurrent Neural Networks

Backpropagation & Autodiff

Computational graphs

Forward & reverse mode autodiff

Vanishing & exploding gradients

PyTorch autograd internals

Regularization, Loss Functions & Training Tricks

L1, L2, dropout, batch norm

Loss function zoo

Learning rate schedules

Mixed precision training

Part II

LLM Architecture

Transformer Architecture Deep Dive

Encoder-decoder vs decoder-only

The transformer architecture comes in three main flavors, each with different computational and representational properties. Understanding these distinctions is crucial for knowing why modern language models have converged on the decoder-only design.

Note

The Original Transformer (Vaswani et al., 2017)
The encoder-decoder architecture was designed for sequence-to-sequence tasks like machine translation:

- **Encoder:** A stack of self-attention layers processing the entire input bidirectionally. Each token can attend to all other tokens freely.
- **Decoder:** A stack of self-attention layers with *causal masking* (tokens can only attend to earlier positions) plus *cross-attention* layers that attend to the encoder output.
- The cross-attention mechanism acts as a "bridge," allowing the decoder to query information from the complete encoded representation of the source sequence.

This design is symmetric and interpretable: the encoder builds a rich bidirectional representation, and the decoder autoregressively generates output while attending to it.

Intuition

Why the architectural split? In seq2seq tasks, the encoder's job is to *understand* the full input (machine translation: read the entire source sentence before generating). The decoder's job is to *generate* the output one token at a time, grounded in what the encoder learned. Separating these concerns kept each component focused.

Note

Encoder-only Models (BERT, RoBERTa, etc.)
Encoder-only architectures drop the decoder entirely:

- Fully bidirectional self-attention with no causal masking.
- Trained with **Masked Language Modeling (MLM)**: randomly mask tokens and predict them from context.
- Excels at **understanding tasks**: classification, named entity recognition, semantic similarity.
- **Cannot generate** text (no autoregressive mechanism).
- Parameters used efficiently for fine-tuning on downstream tasks.

BERT-style models dominated NLP from 2018–2022 for this reason: they are stateless (no generation), lightweight, and highly effective for supervised tasks.

Note

Decoder-only Models (GPT, Llama, Mistral, etc.)
A single stack of causal self-attention layers:

- Fully autoregressive: generates one token at a time, attending only to previous tokens and itself.

- Trained with **Causal Language Modeling (CLM)**: predict the next token given preceding tokens.
- **Unified architecture:** the same forward pass handles both understanding (via prompt context) and generation.
- Scales exceptionally well with compute and data.
- The current dominant paradigm for language models.

Decoder-only models can perform understanding tasks too—through *in-context learning* and instruction tuning—without any architectural change.

Why Decoder-only Won

Three factors drove the shift from encoder-decoder and encoder-only to decoder-only:

1. **Parameter efficiency:** Decoder-only models achieve comparable or superior performance to encoder-decoders with fewer parameters (no redundant encoder + decoder). For the same parameter budget, you get a better model.
2. **Scaling properties:** Decoder-only models scale more predictably with additional compute. Experiments showed that decoder-only scaling laws are cleaner and performance improvements are more reliable as you add data and parameters.
3. **Versatility:** A single decoder-only model can be instruction-tuned to perform classification, generation, reasoning, and other tasks—without architectural retraining. Encoder-decoder required task-specific tuning.
4. **Simplicity:** One stack of layers, one objective, one inference pattern. Fewer moving parts = fewer bugs, easier to optimize, easier to deploy at scale.

The win became clear around 2022–2023: GPT-3, InstructGPT, and subsequent models demonstrated that you could build a general-purpose AI system from a single decoder-only stack.

Property	Encoder-only	Decoder-only	Encoder-decoder
Self-attention masking	Bidirectional	Causal	Bidirectional + Causal
Primary objective	MLM	CLM	CLM + cross-attn
Can generate text?	No	Yes	Yes
Can understand context?	Yes	Yes	Yes
Parameter count (est.)	Base	Base	Base + Base
Training stability	High	High	Moderate
Dominant use (2025)	Semantic tasks	General LLMs	Machine translation
Instruction tuning?	Limited	Excellent	Good

Multi-head self-attention

The self-attention mechanism is the core routing component of the transformer—it determines which tokens communicate with which. However, the detailed mechanics of attention (the softmax-weighted sum, the Q/K/V projections, the scaled dot-product) are covered in depth in **Chapter 7 (Attention Mechanisms)**.

Note

For this chapter, the key intuition is:

- **Self-attention acts as a dynamic router:** each token computes relevance scores (via query-key dot products) to all other tokens and aggregates their values accordingly.
- **Multi-head attention** runs several independent attention mechanisms in parallel, each with different learned projections (Q/K/V). This allows the model to attend to different *subspaces* of the representation simultaneously—e.g., one head might focus on syntax, an-

other on semantics.
- The outputs of all heads are concatenated and projected back to the model dimension d_{model}.
- Multi-head attention has $O(n^2)$ complexity in sequence length n, which is the main computational bottleneck in transformers.

Intuition

Think of multi-head attention as a form of **learned, dynamic routing**: instead of predefined rules for how information flows, the model learns to compute routes (attention weights) on the fly, and different heads specialize in different types of routing patterns.

Feed-forward sublayer

Each transformer layer alternates between self-attention and a feed-forward network (FFN). The FFN processes each token independently and is responsible for much of the model's parameter count and capacity.

Note

Architecture
The vanilla feed-forward sublayer is a two-layer MLP:

$$\text{FFN}(x) = W_2 \cdot \text{activation}(W_1 \cdot x + b_1) + b_2$$

where x is the token representation (d_{model} dimensions), W_1 projects to a higher-dimensional space (expansion), and W_2 projects back down.
- **Expansion ratio:** Traditionally $4\times$ (GPT-2, BERT use $d_{\text{model}} \to 4 \cdot d_{\text{model}} \to d_{\text{model}}$).
- **Activation function:** Originally ReLU; modern LLMs predominantly use **SwiGLU** (a gated variant of GELU).

Note

SwiGLU and Modern Activations
SwiGLU (Shazeer, 2020) is a gated linear unit that outperforms simpler activations:

$$\text{SwiGLU}(x, W, V, b, c) = \text{Swish}(xW + b) \odot (xV + c)$$

where $\odot$ is element-wise multiplication (gating), and $\text{Swish}(x) = x \cdot \sigma(x)$ (sigmoid-weighted x).
Key differences from vanilla FFN:
- **Three weight matrices:** W (up-projection), V (gate-projection), and a down-projection. This is $1.5\times$ more parameters than vanilla two-layer FFN.
- To maintain the same parameter budget, the hidden dimension is typically scaled by $\frac{2}{3}$, so expansion becomes $d_{\text{model}} \to \frac{8}{3} \cdot d_{\text{model}} \to d_{\text{model}}$.
- **Gating mechanism:** The gate $(xV + c)$ learns to selectively suppress or amplify different learned features, improving expressivity.
- Used in PaLM, LLaMA, Mistral, and most modern large language models.

Empirical result: SwiGLU-based models train faster and reach higher quality than ReLU-based models at the same parameter count. This is one reason modern models use gated activations.

Mechanistic interpretability work (Rank-One Model Editing, ROME; Meng et al., 2022) provides strong evidence that **factual associations are localized in the MLP layers**, particularly in the feed-forward networks of middle-to-upper layers.
When researchers edited parameters in the FFN layers, they could surgically change stored facts (e.g., "Eiffel Tower is in Paris" to "Eiffel Tower is in Rome") without affecting other capabilities.
Self-attention layers handle routing and composition; MLPs handle storage.
This suggests the FFN's role is not just nonlinearity, but a form of distributed memory.

Note

Parameter Count
For a single transformer layer with FFN dimension $d_{\text{ff}} = 4 \cdot d_{\text{model}}$ (vanilla):

$$\text{Self-attention params} \approx 12 \cdot d_{\text{model}}^2 \quad \text{(3 projections for Q/K/V, output proj)} \tag{1}$$

$$\text{FFN params} = 2 \cdot d_{\text{model}} \cdot d_{\text{ff}} = 8 \cdot d_{\text{model}}^2 \tag{2}$$

The FFN is the parameter bottleneck: roughly **2/3** of parameters in a transformer layer reside in the feed-forward network, not in attention. This is why efficient inference (KV-cache optimization, sparsity, quantization) must also optimize the FFN.

Layer norm & residual stream

The glue binding transformer layers together is residual connections and layer normalization. These components enable very deep networks and are central to the modern understanding of how transformers compute.

Note

Residual Connections
Each sublayer (self-attention, feed-forward) adds its output to its input:

$$x_{l+1} = x_l + \text{Sublayer}(x_l)$$

Gradient flow: In deep networks, gradients can vanish or explode as they backpropagate. Residual connections provide a direct path for gradients to flow: $\frac{\partial \text{Loss}}{\partial x_l} = \frac{\partial \text{Loss}}{\partial x_{l+1}} \left(1 + \frac{\partial \text{Sublayer}}{\partial x_l}\right)$.
The "+1" term ensures gradients do not decay to zero even if the sublayer's Jacobian is small.
This is why residuals enable 100+ layer networks.

Intuition

The Residual Stream View
A powerful conceptual framework (developed in Anthropic's mechanistic interpretability / circuits program) views transformer computation as iterative updates to a shared **residual stream**:

- The residual stream is the sequence of hidden states flowing through the network: $x_0 \to x_1 \to \cdots \to x_L$.
- Each attention head and each MLP layer *reads* from the stream, computes some function, and *adds* its contribution back to the stream.
- No information is destroyed; each layer is additive. The final output is the sum of all accumulated contributions.
- This view makes it easier to understand compositionality: features can be combined additively throughout the network.

This perspective has proven invaluable for interpretability: it predicts which layers specialize in which tasks, which heads attend to which types of tokens, etc.

Note

Layer Normalization

Layer normalization (LayerNorm) normalizes activations within each token's representation across the feature dimension:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

where μ and σ^2 are computed per token (not per batch), and γ, β are learned affine parameters.

- **No batch statistics:** Unlike batch normalization, LayerNorm does not depend on batch size or composition. This is crucial for LLMs trained with variable-length sequences and deployed with different batch sizes.
- **Prevents internal covariate shift:** Keeps the distribution of activations stable across layers, reducing training instability.

Note

Pre-norm vs Post-norm

Post-norm (original transformer, Vaswani 2017):

$$x_{l+1} = \text{LayerNorm}(x_l + \text{Sublayer}(x_l))$$

Applies LayerNorm *after* the residual addition. This normalizes the next layer's input.

Pre-norm (modern practice):

$$x_{l+1} = x_l + \text{Sublayer}(\text{LayerNorm}(x_l))$$

Applies LayerNorm *before* the sublayer. Normalizes the input to each sublayer.

Pre-norm is superior for deep networks:

- More stable training: gradients flow more reliably.
- Allows deeper networks (100+ layers) without divergence.
- Post-norm networks degrade in stability as depth increases.

Modern LLMs (GPT-3, LLaMA, Mistral) use pre-norm exclusively.

Note

RMSNorm

A recent simplification: **Root Mean Square Layer Norm** (Zhang & Sennrich, 2019) removes mean centering:

$$\text{RMSNorm}(x) = \gamma \odot \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}}$$

- Simpler to compute (one pass instead of two for mean and variance).
- **Empirically equivalent:** achieves the same training stability and final performance as LayerNorm.
- **Faster inference:** fewer arithmetic operations per token.
- Used in LLaMA, Mistral, and other modern efficient models.

Intuition

RMSNorm is a good example of how modern LLMs optimize for practical efficiency: the simplification loses some theoretical justification (mean centering) but gains speed without sacrificing quality.

Modern transformers can be 100+ layers deep because of two complementary techniques:

1. **Residual connections:** enable gradients to flow directly, preventing vanishing gradients.
2. **Pre-norm LayerNorm (or RMSNorm):** stabilizes activation distributions, allowing stable training of very deep networks.

Together, these simple techniques unlock the ability to scale to enormous models. Without them, gradients would explode or vanish, and training would fail.

Attention Mechanisms

Attention allows a model to weight the contribution of different input positions when computing a representation. The transformer's self-attention replaced recurrence with a fully parallelisable mechanism that can model arbitrary long-range dependencies.

Scaled dot-product attention

Given queries $Q \in \mathbb{R}^{n \times d_k}$, keys $K \in \mathbb{R}^{m \times d_k}$, and values $V \in \mathbb{R}^{m \times d_v}$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Why scale by $\sqrt{d_k}$? For large d_k, the dot products $q^\top k$ grow in magnitude, pushing the softmax into regions of very small gradient (saturation). Dividing by $\sqrt{d_k}$ keeps the inputs to softmax in a regime with usable gradients.

Complexity. Naively $O(n^2 d_k)$ time and $O(n^2)$ memory in sequence length n due to the $n \times n$ attention weight matrix. This quadratic scaling is the primary bottleneck for long-context models.

Causal masking. In autoregressive decoders, a lower-triangular mask is applied to the attention weights before softmax, preventing each position from attending to future positions.

Multi-head attention (MHA)

Multi-head attention (Vaswani et al. 2017) runs h independent attention operations ("heads") in parallel, each with its own projection matrices:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Different heads learn to attend to different aspects of the input (e.g. syntactic dependencies, co-reference, positional relations). The concatenated output is projected back to the model dimension by W^O.

Parameter count. For model dimension d_{model} and h heads with $d_k = d_v = d_{\text{model}}/h$: each head has $3 \times d_{\text{model}} \times d_k$ projection parameters, plus d_{model}^2 for W^O. Total: $4d_{\text{model}}^2$.

Multi-query & grouped-query attention (MQA, GQA)

Standard MHA maintains h separate K and V heads, making the KV cache $\propto h \times d_k$ per token — a dominant memory cost at inference.

Multi-Query Attention (MQA) (Shazeer 2019): all query heads share a single K head and a single V head. KV cache shrinks by a factor of h. Slight accuracy degradation, but significant memory and throughput improvement.

Grouped-Query Attention (GQA) (Ainslie et al. 2023): a middle ground. Queries are divided into G groups; each group shares one K/V head. $G = 1 \Rightarrow$ MQA; $G = h \Rightarrow$ MHA. GQA with $G = 8$ is the standard in Llama 2/3, Gemma, and Mistral — it retains most MHA quality while substantially reducing KV cache size.

Compresses the KV cache more aggressively by projecting each token's key and value into a shared low-rank latent vector $c_{KV} \in \mathbb{R}^{d_c}$ ($d_c \ll h \cdot d_k$) via a down-projection. At inference, only c_{KV} (plus a small rope-compressed key) is cached. This reduces KV cache to the latent

dimension rather than the full KV dimension. **FlashMLA** provides an efficient GPU kernel for MLA computation.

Note

MQA $\subset$ GQA $\subset$ MHA in terms of parameter richness. MLA is an orthogonal approach: instead of sharing heads, it compresses the stored representation via learned low-rank projections.

Sparse & linear attention variants

Motivation. Full attention is $O(n^2)$. For $n = 128$K tokens, the attention matrix alone requires $\sim$64 GB in FP32 — infeasible on a single GPU.

Sparse attention restricts each token to attend to only a fixed subset of positions, reducing complexity:

- **Local / sliding window:** each token attends to its w nearest neighbours. Captures local dependencies; used in LongFormer and BigBird.
- **Global tokens:** a few tokens (e.g. `[CLS]`) attend globally; all others attend locally. Provides long-range signal cheaply.
- **Random:** each token also attends to a few randomly selected positions.

Hardware-aligned sparse attention designed for direct GPU execution. Combines (1) compressed coarse-grained attention over blocks, (2) selective fine-grained attention on important tokens, and (3) sliding-window local attention. Achieves near-full-attention quality with a fraction of the FLOPs, and its structure is optimised for tensor-core execution.

Linear attention replaces the softmax with a kernel function $k(\mathbf{q}, \mathbf{k}) \approx \phi(\mathbf{q})^\top \phi(\mathbf{k})$ so that the associative property of matrix multiplication can be exploited to compute the attention in $O(n)$ time via cumulative sums. The cost is that softmax's peaked, normalised distributions are hard to replicate exactly, leading to accuracy losses on tasks requiring sharp attention.

Kimi-Linear is a linear attention architecture from Moonshot AI (Kimi) that enables linear-time sequence modelling for long contexts in production settings.

FlashAttention

IO-aware exact attention algorithm. Standard attention materialises the full $n \times n$ attention matrix in GPU high-bandwidth memory (HBM), causing $O(n^2)$ HBM reads/writes. FlashAttention instead tiles the computation into SRAM-sized blocks, fusing the softmax and the weighted sum into a single kernel pass. **The mathematical output is identical to standard attention** — no approximation.

Performance. FlashAttention reduces HBM accesses from $O(n^2)$ to $O(n^2/M)$ where M is SRAM capacity. On A100 GPUs, this translates to 2–4$\times$ wall-clock speedup and 10–20$\times$ memory saving for typical sequence lengths.

FlashAttention-2 (Dao 2023): improves parallelism by splitting the outer loop over query blocks (enabling more thread-block parallelism) and reducing non-matmul FLOPs by $\sim$2$\times$.

FlashAttention-3 (Shah et al. 2024): H100-specific optimisations including warp specialisation, asynchronous TMA (Tensor Memory Accelerator) loads, and FP8 support. Achieves > 75% FP16 FLOP utilisation on H100.

Intuition

FlashAttention's key insight: the bottleneck in attention is not arithmetic (GPU tensor cores are fast) but *memory bandwidth* (moving the n^2 matrix between HBM and compute units). By keeping intermediate results in fast on-chip SRAM, FlashAttention turns an HBM-bound operation into a compute-bound one.

Positional Encodings

Sinusoidal encodings (original)

Learned absolute positions

Relative position encodings

RoPE (rotary position embedding)

ALiBi

Tokenization & Vocabulary

Tokenization is the process of breaking raw text into discrete units (tokens) that a neural network can process. The choice of tokenizer and vocabulary size directly affects model efficiency, inference speed, and multilingual capability. Modern LLMs use subword tokenization methods that balance character-level flexibility with word-level efficiency.

BPE, WordPiece, Unigram

Byte-Pair Encoding (BPE)

Byte-Pair Encoding is a data compression algorithm adapted for NLP by Sennrich et al. (2016). The algorithm works as follows:

1. Start with a character-level vocabulary (all unique characters in the corpus).
2. Count the frequency of all adjacent token pairs.
3. Merge the most frequent pair into a new token.
4. Repeat until the vocabulary reaches a target size.

The result is a set of subword units that emerge naturally from the data. For example, across words like `low`, `lower`, `newest`, the algorithm might produce common subwords: `low`, `er`, `new`, `est`.

Intuition

BPE is *greedy and deterministic*: the merge sequence is fixed by frequency counts, and there is no ambiguity in how text is tokenized. This makes it fast and reproducible, though it doesn't optimize for any downstream task.

GPT-2 popularized *byte-level BPE*, where the initial vocabulary is all 256 bytes rather than characters. This ensures no "unknown token" problem even for unusual Unicode sequences, since any byte sequence can be represented.

WordPiece

WordPiece is used in BERT and similar models. It is similar to BPE but differs in the merge criterion:

- BPE merges the *most frequent* adjacent pair.
- WordPiece merges the pair that *maximizes language model likelihood*, i.e., maximizes how much the merge improves the model's ability to compress the corpus.

WordPiece marks continuation subwords with a `##` prefix. For example, the word `playing` might tokenize as: `play` + `##ing`. This notation makes it easy to reconstruct which tokens are word-initial versus continuations.

Unigram Language Model Tokenizer

The Unigram approach inverts the merge strategy:

1. Start with a *very large* vocabulary (e.g., all character n-grams in the corpus).
2. Iteratively remove tokens that would increase corpus loss the least (i.e., remove tokens whose removal has minimal impact).
3. Repeat until vocabulary reaches the target size.

Unlike BPE, Unigram tokenization is *probabilistic*: multiple segmentations may exist, and the model assigns probabilities to each. During training, the segmentation that maximizes likelihood is preferred. This flexibility can be beneficial for capturing multiple valid decompositions.

Unigram is a core component of SentencePiece (discussed below).

SentencePiece & tiktoken

SentencePiece

SentencePiece is a language-agnostic tokenization library that treats input as a raw *character stream*, including spaces. Spaces are represented as the special character ▁ (U+2581, a lower one-eighth block used as a space placeholder). This design has major advantages:

- **No preprocessing required:** Other tokenizers require language-specific pre-tokenization (splitting on spaces, lowercasing, etc.); SentencePiece handles raw text directly.
- **Native CJK/RTL support:** Chinese, Japanese, Korean, Arabic, and other scripts are handled uniformly without special-casing.
- **Reversibility:** Tokens can always be losslessly converted back to the original text.

SentencePiece internally implements either BPE or Unigram encoding. It is used in LLaMA, T5, XLNet, and many other modern models.

> **Note**
>
> SentencePiece's treatment of spaces as explicit tokens is crucial for multilingual fairness. A Thai sentence will have explicit space tokens, preventing the tokenizer from silently biasing toward languages that use word boundaries.

tiktoken

tiktoken is OpenAI's tokenizer for GPT-3.5 and GPT-4. It is implemented in Rust for speed and uses byte-level BPE internally.

Key variants:

- `cl100k_base`: 100K tokens; used in GPT-4 and GPT-3.5-turbo.
- `o200k_base`: 200K tokens; used in newer OpenAI models.

The larger vocabularies (especially 200K) reduce token bloat on code and numerics compared to GPT-2's 50K vocab. This directly reduces API costs and increases effective context length for code-heavy inputs.

> tiktoken is optimized for English and source code. Non-English text may still experience significant over-tokenization relative to English. See the section on multilingual tokenization below.

Vocabulary size trade-offs

Choosing vocabulary size is a fundamental trade-off:

Note

- **Larger vocabulary** (e.g., 100K): Fewer tokens per sentence ⇒ shorter sequences, faster inference, lower API cost. **Downside:** Larger embedding parameter matrix.
- **Smaller vocabulary** (e.g., 8K): Shorter embedding table. **Downside:** Longer sequences, higher inference cost, reduced effective context length.

Most modern LLMs use a vocabulary in the range **32K–100K**, which balances these factors. The exact choice depends on:

- Compute budget (smaller models may use smaller vocab).
- Expected token density: models targeting code or multilingual data may use larger vocabularies.
- Inference constraints (latency vs. memory).

Fertility

"Fertility" measures the average number of tokens required to encode a word. English fertility in GPT-4 is approximately **1.3 tokens per word**—very efficient. However, fertility varies dramatically across languages (see the multilingual tokenization subsection below).

Multilingual tokenization

The over-tokenization problem

Tokenizers trained primarily on English corpora are *extremely inefficient* for non-English text. A few concrete examples:

- Arabic and Hindi text often require 3–5× more tokens than English for equivalent meaning.
- LLaMA-1 (32K vocabulary, trained mostly on English-heavy data) tokenizes Thai approximately **10× worse** than English.
- The same disparity applies to many low-resource languages, Cyrillic scripts, and CJK languages.

Vocabulary imbalance is the root cause. If a tokenizer is trained on 90% English + 10% other languages, the English vocabulary will be optimized for common English subwords, but rare non-English patterns will fragment into many single-character tokens. A 32K vocab with heavy English bias dedicates perhaps 20K tokens to English subwords and only 12K to all other languages combined.

Concrete impacts

Over-tokenization creates cascading problems:

1. **Higher API costs:** Large language model APIs charge per token. Non-English users pay 3–10× more for the same semantic content.
2. **Shorter effective context:** If a 4K context window is filled with 1.3 tokens per word (English) vs. 13 tokens per word (Thai), the Thai user sees only 400 words vs. 3000 words for English users—a 7× gap.
3. **Degraded inference:** Longer sequences increase memory usage, cache misses, and latency.

Solutions

Several approaches mitigate over-tokenization:

- **Larger multilingual vocabulary:** Increase vocab from 32K to 128K–256K and train on a balanced multilingual corpus. This dedicates more tokens to non-English scripts.
- **Script-specific tokens:** Add dedicated token ranges for Arabic, Devanagari, Cyrillic, CJK, etc. SentencePiece can be configured to do this.
- **Balanced training data:** Ensure the tokenizer training corpus represents all target languages proportionally, not just English-heavy data.
- **SentencePiece with multilingual corpora:** Use SentencePiece with a balanced training set to achieve more uniform fertility across languages.

Intuition

The optimal tokenizer for a multilingual system must be trained on multilingual data. A 100K vocab trained on 50% English + 25% CJK + 25% other languages will achieve far better balance than a 100K vocab trained on 90% English. This requires more engineering effort but yields significantly fairer and more efficient multilingual models.

Scaling Laws & Emergent Capabilities

Chinchilla scaling laws

Compute-optimal training

Emergent abilities & phase transitions

Scaling inference compute

Mixture of Experts (MoE)

Sparse MoE fundamentals

Routing mechanisms

Load balancing losses

MoE in practice (Mixtral, GPT-4 rumours)

State Space Models: Mamba & RWKV

State Space Models (SSMs) are an alternative sequence-modelling paradigm to transformers. They draw on classical control theory: a continuous-time linear dynamical system maps an input signal $u(t)$ to an output $y(t)$ via a latent state $h(t)$. Discretised for sequence data, SSMs offer $O(n)$ inference complexity with a constant- size recurrent state, in contrast to the $O(n^2)$ attention matrix of transformers.

S4 & structured state spaces

Structured State Space Sequence model. Parameterises the state transition matrix A using the **HiPPO** (High-order Polynomial Projection Operators) initialisation, which provably compresses the history of an input signal into orthogonal polynomial coefficients. This gives S4 the capacity to model extremely long-range dependencies that RNNs and transformers typically miss.

Continuous system. The core equations are:

$$h'(t) = Ah(t) + Bu(t), \quad y(t) = Ch(t) + Du(t)$$

Discretised with step size Δ:

$$h_k = \bar{A}h_{k-1} + \bar{B}u_k, \quad y_k = Ch_k$$

where $\bar{A} = e^{\Delta A}$ and $\bar{B} = (e^{\Delta A} - I)A^{-1}B$.

Efficient training via convolution. With fixed A, B, C, the recurrence can be unrolled into a convolution $y = u * K$ where the convolutional kernel $K = (CB, C\bar{A}B, C\bar{A}^2B, \ldots)$ can be computed in $O(n \log n)$ with FFT. This enables fast, parallelisable training while retaining $O(1)$ inference via the recurrence.

Mamba & selective SSMs

S4's state transition matrices A, B, C are fixed (input-independent), giving a linear time-invariant (LTI) system. Mamba breaks this assumption.

Introduces a **selection mechanism**: the SSM parameters B, C, and the discretisation step Δ become functions of the input, making the model input-dependent (non-LTI). This allows the model to *selectively* propagate or reset information based on content, analogous to the gating in LSTMs but with superior long-range capacity.

Key innovation. Because the matrices are now input-dependent, the convolution trick no longer applies. Mamba instead implements a hardware-aware *selective scan* algorithm that fuses the recurrence into a single GPU kernel operating in SRAM, achieving efficient parallel computation during training while preserving $O(1)$ recurrent inference.

Mamba architecture. Each Mamba block is a gated MLP variant: (1) the input is projected up; (2) one branch runs through the selective SSM; (3) the other branch gates the SSM output; (4) the result is projected back down. This mirrors the gated feedforward structure of modern LLMs.

Mamba-2 simplifies the selective SSM with a multi-head structure (SSD: Structured State Space Duality), establishing formal connections between SSMs and linear attention. It is faster in practice with similar quality.

RWKV: RNN-transformer hybrid

RWKV (Peng et al. 2023) reframes transformer attention as a linear RNN using a time-mixing mechanism inspired by both attention and classic RNNs. The key, value, and receptance (gate) are computed as in a transformer, but attention is computed as a cumulative weighted sum without the quadratic softmax matrix:

$$y_t = \frac{\sum_{\tau \le t} e^{-(t-\tau)w + k_\tau} v_\tau}{\sum_{\tau \le t} e^{-(t-\tau)w + k_\tau}}$$

where w is a learned decay parameter. This has $O(1)$ recurrent inference (like an RNN) while being parallelisable during training.

SSM vs attention trade-offs

Property	Transformer (attention)	SSM / Mamba
Training complexity	$O(n^2)$	$O(n)$
Inference state	KV cache (grows with n)	Fixed recurrent state $O(1)$
Long-context memory	Full context in theory	Fixed capacity hidden state
Positional inductive bias	Positional encodings needed	Baked in via state dynamics
Content addressability	Arbitrary key-value lookup	Approximate; recency bias
Hardware efficiency (A100)	FlashAttention helps	Selective scan; good SRAM use

When SSMs win: audio, genomics, time series, and any domain where sequences are extremely long and local patterns matter more than global content retrieval.

When attention wins: language reasoning, complex instruction following, tasks requiring recall of specific content from many steps ago. The fixed-capacity SSM state can "forget" distant information even if it is relevant.

Hybrid models (e.g. Jamba, Zamba) interleave Mamba and attention layers, aiming to capture the memory efficiency of SSMs for most layers while retaining the associative recall strength of attention in a few layers. This is an active research frontier.

Intuition

Attention is like a database: you can look up any past token by content at any step, but the lookup table (KV cache) grows with context. An SSM is like RAM with a fixed size: you can only hold so much, and old information may be overwritten. Hybrid models give you both a large RAM and a lookup table, used selectively.

Part III

Hardware & Systems

GPU Architecture

SM, warp, thread hierarchy

Tensor cores & TF32/BF16

Memory: HBM, L2, SRAM

CUDA programming model

NVLink & GPU interconnects

TPUs, NPUs & Custom ASICs

Google TPU architecture

Systolic arrays

Apple Neural Engine

Groq LPU & Cerebras WSE

Custom ASIC design trade-offs

Memory Hierarchy & Bandwidth Bottlenecks

Roofline model

Arithmetic intensity

Memory-bound vs compute-bound ops

HBM bandwidth & capacity limits

CUDA Kernels for ML

Matrix multiplication (GEMM)

Softmax & layer norm kernels

Fused attention kernel

Shared memory tiling

Profiling with Nsight

Triton & Custom Op Authoring

Triton language overview

Writing a fused kernel in Triton

Triton vs CUDA trade-offs

Integration with PyTorch

Profiling & Optimization Workflows

PyTorch Profiler & trace viewer

Nsight Systems & Compute

Part IV

Training at Scale

Pretraining Pipelines & Data Curation

Web-scale data collection

Deduplication & quality filtering

Data mixing strategies

Pretraining objectives (CLM, MLM, etc.)

Causal Language Modelling (CLM). The standard objective for autoregressive models: predict the next token given all preceding tokens. Loss is cross-entropy over the vocabulary at each position. Also called *next-token prediction*. Used by GPT, LLaMA, Mistral, and most production LLMs.

Masked Language Modelling (MLM). The BERT-family objective: randomly mask ≈15% of input tokens and predict them from bidirectional context. Produces richer contextual representations but is not generative.

Span corruption (T5). Replace random contiguous spans with a sentinel token and predict the replaced spans. Combines CLM and MLM; enables both encoding and generation.

The generality of CLM: beyond natural language. The CLM objective is domain-agnostic — it requires only a discrete token sequence with learnable structure. This has enabled foundation models trained on non-textual "languages":

- **Code:** Codex, StarCoder, and DeepSeek-Coder treat source code as a token sequence, learning syntax, APIs, and idioms from next-token prediction.
- **Genomics (DNA/RNA):** EVO 2 treats nucleotide sequences (A, T, G, C) as a four-letter alphabet and applies CLM at single-nucleotide resolution. The crucial insight: DNA from living organisms is an *evolutionary self-labelled dataset*. Because organisms must function to survive and reproduce, every sequence in the training corpus has already been filtered by billions of years of natural selection. Lethal mutations are absent; conserved sequences are functionally critical. CLM thus receives an implicit supervision signal at a scale and quality that no human-annotated dataset could match. See Chapter 51.
- **Protein sequences:** ESMFold and ProtTrans apply masked language modelling to amino acid sequences. The evolutionary conservation signal is even stronger at the protein level — most protein positions are under stronger selective pressure than individual nucleotides.
- **Music and audio tokens:** discretised audio (EnCodec, SoundStream) makes music generation a CLM task.
- **Agent trajectories:** CLM on (observation, action) sequences models agent behaviour offline — the basis of Decision Transformer.

Intuition

The CLM objective does not care *what* the tokens represent, only that the sequence has learnable structure. Any domain where the future is predictable from the past — code, DNA, protein, audio, actions — is in principle a CLM problem. The quality of implicit supervision depends on how strongly the domain's structure constrains valid sequences: evolutionary selection pressure makes genomic CLM arguably more information-dense per token than internet text.

Distributed Training

Modern large-scale LLM training combines multiple parallelism strategies simultaneously. The NVIDIA Megatron team coined the term **5D Parallelism** to describe the five orthogonal dimensions along which a training job can be distributed: Data (DP), Tensor (TP), Pipeline (PP), Sequence/Context (SP/CP), and Expert (EP).

Note

Each parallelism dimension targets a different bottleneck. Real training runs combine all five: a typical 70B job might use DP×8, TP×4, PP×4, with SP inside TP regions, and EP added for MoE variants.

Data parallelism (DDP)

Data parallelism replicates the full model on every GPU and splits the global mini-batch across GPUs. Each GPU computes a full forward and backward pass on its shard, then gradients are synchronised with an all-reduce before the optimiser step.

DDP (DistributedDataParallel) in PyTorch buckets gradients and overlaps all-reduce communication with the tail of the backward pass to hide latency.

Fully Sharded Data Parallel (FSDP) (see § 20.4) extends DDP by sharding optimiser states, gradients, *and* parameters across DP ranks.

Tensor parallelism

Tensor Parallelism (TP) splits individual weight matrices across GPUs. For a linear layer $Y = XW$ with $W \in \mathbb{R}^{d \times k}$:

- **Column-parallel:** W is split column-wise into $[W_1|W_2]$ across N GPUs. Each GPU computes XW_i and the outputs are concatenated via all-gather.
- **Row-parallel:** W is split row-wise; the input X must be split correspondingly, and partial sums are reduced across GPUs with all-reduce.

Megatron-LM implements column-parallel and row-parallel linear layers with a single all-reduce per transformer layer in each direction. Because all-reduce must complete before the next layer, TP requires extremely low-latency interconnects (NVLink within a node; TP is almost never used across nodes).

Pipeline parallelism

Pipeline Parallelism (PP) assigns contiguous groups of transformer layers to different GPUs ("stages"). A micro-batch flows through each stage in sequence, then the next micro-batch fills the pipeline. The **pipeline bubble** — idle time when a stage is waiting for upstream activations — is the main inefficiency.

1F1B schedule (Megatron-LM): interleaves forward and backward micro-batches across stages. Each stage alternates one forward pass with one backward pass, reducing peak pipeline bubble from $O(p)$ to $O(1)$ relative to the number of stages p.

Bidirectional pipeline where micro-batches travel *both* forward and backward through the pipeline simultaneously. Computation and communication are fully overlapped, reducing bubble fraction to near zero. Used in DeepSeek-V3 training alongside EP for MoE layers.

FSDP & ZeRO optimizer

A single GPU cannot hold the full state of a large model. The **ZeRO** optimizer (Zero Redundancy Optimizer, Microsoft DeepSpeed) eliminates the redundant copies of state that vanilla DP creates.

Stage	What is sharded	Memory saving (vs. DP)
ZeRO-1	Optimiser states	4×
ZeRO-2	+ Gradients	8×
ZeRO-3	+ Parameters	64× (linear in world size)

PyTorch's **FSDP** implements ZeRO-3 natively: during the forward pass each GPU all-gathers the full parameter shard, uses it, then immediately discards it. Gradients are accumulated locally, then reduce-scattered. Only $1/N$ of the parameters live on each GPU at rest.

ZeRO-Infinity (DeepSpeed) extends sharding to CPU RAM and NVMe storage, enabling trillion-parameter training at the cost of higher latency for offloaded data.

Frameworks:

- **Megatron-LM** (NVIDIA): reference implementation of TP + PP with 1F1B scheduling; used to train GPT-3, Llama, Mistral, and most NVIDIA-scale models.
- **DeepSpeed** (Microsoft): ZeRO optimizer, activation checkpointing, offloading to CPU/NVMe. Often combined with Megatron via *Megatron-DeepSpeed*.
- **Transformer Engine** (NVIDIA): mixed-precision training with FP8 on H100 hardware; integrates directly into Megatron-LM and NeMo.
- **Megatron Bridge**: adapter to use Megatron-LM parallelism with other training stacks.

3D parallelism & Sequence/Context parallelism

3D parallelism combines DP + TP + PP and is the standard recipe for hundred- billion-scale training. Each node group runs TP internally (NVLink); PP connects node groups sequentially; DP replicates the entire pipeline.

Sequence Parallelism (SP) (Megatron-LM): splits the sequence dimension along the portions of a layer (LayerNorm, Dropout) that lie outside the TP-parallelised matrix multiplications. This removes the remaining activation memory bottleneck in TP.

Context Parallelism (CP): for very long contexts (100K+ tokens), the sequence dimension is split across GPUs even during the attention computation itself. Ring Attention and similar algorithms enable cross-GPU attention by passing keys and values in a ring pattern, so each GPU sees the full sequence for attention while storing only a shard of the activations.

Expert Parallelism (EP): in Mixture-of-Experts (MoE) models each expert is assigned to a different GPU. Token routing via all-to-all communication dispatches each token to its selected expert and gathers the results back. EP combined with TP/PP/DP gives *5D parallelism*. See also: Mixtral of Experts, DeepSeek-MoE.

Intuition

Think of 5D parallelism as slicing a 5-dimensional tensor (model × data × layers × sequence × experts) across a cluster of GPUs. Each dimension targets a different memory or compute bottleneck.

Part V

Post-Training & Alignment

Fine-tuning: SFT, LoRA, QLoRA

Supervised fine-tuning (SFT)

Supervised Fine-Tuning (SFT) continues training a pretrained base model on a curated dataset of (instruction, response) pairs. The objective is identical to pretraining (cross-entropy over the response tokens), but the data distribution shifts from raw internet text to human-authored demonstrations of the desired behaviour.

What SFT accomplishes:

- **Instruction following:** the base model learns to respond to explicit task prompts rather than continuing text in the pretraining style.
- **Format compliance:** teaches the model to use specific response formats (markdown, JSON, conversational tone).
- **Surface alignment:** reduces obvious harmful, off-topic, or low-quality outputs, though it does not align values as robustly as RLHF.

Data quality over quantity. 1,000 carefully curated, diverse SFT examples can outperform 100,000 noisy ones. The LIMA paper (Zhou et al. 2023) demonstrated that 1K high-quality examples produced competitive instruction-following with models trained on orders of magnitude more data. Data curation and deduplication are the dominant effort in SFT pipelines.

Masking. During SFT training, loss is computed only on the response tokens; the instruction/prompt tokens are masked from the loss computation. This prevents the model from "learning" to output prompts.

Parameter-efficient fine-tuning (PEFT)

Full fine-tuning updates all model parameters, which costs as much GPU memory as pretraining and risks catastrophic forgetting when data is limited. **Parameter-Efficient Fine-Tuning (PEFT)** methods update only a small subset of parameters while keeping most of the model frozen.

Common PEFT families:

- **Adapter layers:** insert small bottleneck layers between transformer components; only adapter weights are trained ($< 1\%$ of parameters).
- **Prefix tuning:** prepend trainable "virtual tokens" to the input; gradients flow only through these prefix vectors.
- **Prompt tuning:** even simpler — only a small prefix embedding is trained; the model weights are entirely frozen.
- **LoRA** (see below): the dominant PEFT method for LLMs.

Why PEFT works: pretrained models learn rich representations; most task- specific adaptation lies in a low-dimensional subspace of the weight update. PEFT explicitly exploits this intrinsic dimensionality.

LoRA & AdaLoRA

Freezes the pretrained weight matrix $W_0 \in \mathbb{R}^{d\times k}$ and injects a trainable low-rank decomposition $\Delta W = BA$ where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, $r \ll \min(d,k)$. The modified forward pass is $h = W_0 x + \frac{\alpha}{r} BAx$. At inference, the LoRA weights can be merged ($W' = W_0 + \frac{\alpha}{r} BA$) so there is *zero* additional inference latency.

Key properties:

- Typically applied to the Q, K, V, and output projection matrices; sometimes also to MLP layers.
- Rank $r = 4$–64 is common; smaller r acts as stronger regularisation. α is a scaling hyperparameter (often set equal to r so the factor is 1).
- Trainable parameters: $r \cdot (d+k)$ per matrix ≈ 0.1–1% of total parameters for typical LLM sizes.
- A is initialised with random Gaussian; B is initialised to zero, so $\Delta W = 0$ at the start of training (no disruption to the pretrained model).

AdaLoRA adaptively allocates the rank budget across weight matrices based on importance scores (derived from SVD of the gradient signal). High-importance matrices receive more rank; low-importance ones are pruned. This avoids the uniform rank assumption of vanilla LoRA.

Intuition

LoRA is the "surgical strike" fine-tuning method: instead of re-training the full weight matrix, it adds a small residual correction in a low-dimensional subspace. The pretrained knowledge is preserved; only the task-specific delta is learned.

QLoRA & quantized fine-tuning

Combines quantization with LoRA to make fine-tuning billion-parameter models feasible on a single consumer/research GPU.

Three technical innovations:

- **4-bit NormalFloat (NF4):** a quantization format specifically designed for normally-distributed weights (as neural network parameters are). Minimises quantization error for Gaussian distributions; performs better than INT4 or FP4.
- **Double quantization:** quantizes the quantization constants themselves (the per-block scaling factors) from FP32 to FP8, saving ≈0.5 bits per parameter on top of the NF4 savings.
- **Paged optimisers:** uses NVIDIA Unified Memory to offload optimiser states to CPU RAM when GPU memory is exhausted during gradient accumulation. Avoids OOM crashes during long forward passes with large batch sizes.

Workflow: (1) load base model in NF4 4-bit precision; (2) attach FP16/BF16 LoRA adapters to the frozen quantized base; (3) train only the LoRA adapters. The base model is never de-quantized during training; dequantization happens on-the-fly during each forward/backward pass.

Practical impact. QLoRA made it possible to fine-tune a 65B-parameter model on a single 48 GB A100 GPU (or a 13B model on a single 24 GB consumer GPU), enabling researchers and practitioners without large compute budgets to fine-tune frontier-scale models.

Note

QLoRA introduces a small accuracy penalty versus full-precision LoRA due to the 4-bit base model. For most task-specific fine-tunes the gap is negligible; for highly precise scientific/mathematical tasks it may matter.

RLHF, DPO & Reward Modelling

Post-training alignment methods shape a pretrained (or SFT'd) model's behaviour to match human preferences: being helpful, harmless, and honest. The dominant paradigms are **Reinforcement Learning from Human Feedback (RLHF)** and its offline successor **Direct Preference Optimization (DPO)**.

Human preference collection

Why comparisons? Absolute quality ratings ("rate this response 1–5") are noisy and inconsistent across annotators. Comparative judgements ("which of A or B is better?") are faster, more reliable, and map directly to a mathematical preference model.

Process.

1. A prompt is sampled from a prompt distribution.
2. Two or more responses are generated by the current model (or by different models / human writers).
3. Human annotators select the preferred response, optionally with a reason.
4. Preference pairs (x, y_w, y_l) — prompt, preferred response, rejected response — are collected into a dataset.

Bradley-Terry model. Most reward model training assumes the Bradley-Terry model of pairwise preference:

$$P(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$$

where r is the scalar reward model score. This gives a cross-entropy loss for reward model training.

Reward model training

A reward model (RM) takes a prompt-response pair (x, y) and returns a scalar score indicating how well the response aligns with human preferences.

Architecture. Typically initialised from the SFT checkpoint with a scalar head replacing the language-model head. Training uses the Bradley-Terry cross-entropy loss over preference pairs.

Reward hacking. Because the RM is only an approximation of true human preference, optimising against it indefinitely leads to *reward hacking*: the policy finds responses that score highly on the RM but are not actually preferred by humans (e.g. very long responses, responses with confident-sounding false statements). A KL penalty from the SFT reference policy is applied to limit distributional drift.

PPO for language models

Proximal Policy Optimization (PPO) was used in InstructGPT (Ouyang et al. 2022) to fine-tune GPT-3 with RLHF. The language model is the policy π_θ; generating a response is a sequence of actions; the reward is the RM score.

Training loop:

1. Sample prompts from the training distribution.
2. Generate responses using the current policy π_θ (rollout).
3. Score responses with the frozen reward model.
4. Apply a per-token KL penalty against the SFT reference policy π_{SFT} to the reward: $r_{\text{total}}(x, y) = r_{\text{RM}}(x, y) - \beta \cdot \text{KL}(\pi_\theta \| \pi_{\text{SFT}})$.
5. Update π_θ using the PPO clipped objective to maximise r_{total}.

Challenges. PPO is on-policy: it must generate new rollouts for every gradient step, which is slow (decoding $\gg$ forward pass). It requires holding four large models in memory simultaneously: policy, reference policy, reward model, and value model. This makes PPO computationally expensive.

Direct preference optimization (DPO)

Observes that the optimal RLHF policy π^* can be expressed analytically in terms of the reference policy and the reward. Substituting this relationship back into the reward model's Bradley-Terry loss yields a closed-form objective over preference pairs that does not require an explicit reward model or RL.

DPO loss:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

Intuitively: increase the likelihood of the preferred response relative to the reference, and decrease the likelihood of the rejected response relative to the reference, with the KL constraint naturally embedded via the ratio terms.

Advantages over PPO:

- No reward model to train separately.
- No on-policy rollouts; trains offline on a fixed preference dataset like SFT.
- Simpler codebase; fits in the same training loop as SFT.

Limitations: sensitive to preference dataset quality; can be unstable when the reference policy is very different from the SFT model; does not directly improve performance beyond the reference model's distribution (no exploration).

GRPO & alternatives

Group Relative Policy Optimization (GRPO) was introduced by DeepSeek (Shao et al. 2024) and used to train the reasoning capabilities of DeepSeek-R1.

Key idea. For each prompt, sample a *group* of G outputs from the current policy. Compute a scalar reward r_i for each (e.g. a verifiable correctness signal for math). Normalise rewards within the group to get advantages:

$$\hat{A}_i = \frac{r_i - \text{mean}(r_{1:G})}{\text{std}(r_{1:G})}$$

Update the policy to increase the probability of high-advantage outputs.

Why GRPO over PPO? PPO requires a separately-trained value model (critic) to estimate advantages, which consumes memory and adds training complexity. GRPO replaces the critic with group-relative normalisation: the average reward within a group of sampled responses serves as the implicit baseline, eliminating the need for a value model entirely. This makes GRPO roughly as memory-efficient as DPO while enabling on-policy exploration like PPO.

Note

GRPO is particularly suited to tasks with *verifiable* rewards: math (check final answer), code (run unit tests), formal logic. For open-ended tasks where reward is subjective, a learned reward model must still be used.

Other alternatives:

- **REINFORCE** (simple, high-variance baseline): GRPO is a variance-reduced version of REINFORCE with group baseline.
- **KTO** (Ethayarajh et al. 2024): aligns using only unpaired preference signals (good/bad labels without requiring (y_w, y_l) pairs).
- **SimPO** (Meng et al. 2024): removes the reference model entirely; uses the average log-likelihood of the generated response as an implicit length normalisation.

Constitutional AI & RLHF Safety

Constitutional AI (Anthropic)

RLHF safety properties

Red-teaming methodologies

Refusal & harmlessness trade-offs

Part VI

Inference & Deployment

Inference & Decoding Strategies

Greedy, beam search, sampling

At each decoding step the model produces a probability distribution over the vocabulary. The decoding strategy determines how a token is selected from that distribution.

Greedy decoding always selects $\arg\max_t P(t \mid \text{context})$. It is fast and deterministic but tends to produce repetitive, low-entropy output because the model commits to the single most probable path at every step, discarding all alternatives.

Beam search maintains the top-B candidate sequences ("beams") in parallel. At each step every beam is extended with its top-B continuations, and the global top-B are kept. Final output is the highest-scoring complete sequence. Beam search trades $O(B)$ memory for more globally coherent text and is the default for machine translation; it tends to produce overly conservative text in open-ended generation.

Sampling draws from the full (or truncated) distribution, enabling varied and creative output at the cost of occasional incoherence. Temperature, top-k, and top-p control where sampling takes place (see §below).

Intuition

Greedy $\approx$ beam search with $B = 1$; both are deterministic. Sampling with temperature $T \to 0$ converges to greedy; $T \to \infty$ becomes uniform.

Temperature, top-k, top-p

These three hyperparameters reshape the distribution before sampling.

Temperature T scales the logits before the softmax:

$$P'(t) = \frac{\exp(z_t/T)}{\sum_{t'} \exp(z_{t'}/T)}$$

$T < 1$ sharpens the distribution (more deterministic); $T > 1$ flattens it (more exploratory). Typical creative-writing values: $T \approx 0.7$–1.0; coding: $T \approx 0.2$.

Top-k sampling keeps only the k highest-probability tokens, renormalises, and samples. Prevents extremely unlikely tokens from being selected but the hard cutoff can still include incoherent tokens if the distribution is flat.

Top-p (nucleus) sampling (Holtzman et al. 2020) keeps the smallest set of tokens whose cumulative probability $\geq p$, then samples. The effective vocabulary size adapts to the confidence of the distribution: when the model is certain, only a few tokens make the nucleus; when uncertain, many tokens share probability mass and a larger nucleus is used. $p = 0.9$–0.95 is a common default.

Note

Top-p is generally preferred over top-k for open-ended generation because it adapts automatically. Using both simultaneously ($p + k$) is also common as an additional safety net.

KV cache mechanics

During autoregressive generation, computing attention for every new token would require re-processing all preceding tokens. The **key-value (KV) cache** avoids this.

How it works. On the first forward pass (prefill phase), the model computes K and V matrices for all input tokens and caches them. For every subsequent token generated (decode phase), only the new token's Q, K, V are computed; the new K, V vectors are appended to the cache and the full attention is computed using the stored history. No recalculation of past keys and values is needed.

Memory cost. KV cache size $\propto$ `num_layers` $\times$ `num_heads` $\times$ `head_dim` $\times$ `seq_len` $\times$ `batch_size`. For a 70B-parameter model serving 32K-token sequences in a batch of 16, KV cache can easily exceed 30 GB — the dominant memory consumer at inference time, not the weights themselves.

KV cache with GQA/MQA. Grouped-Query Attention (GQA) and Multi-Query Attention (MQA) reduce the number of distinct K/V heads, directly shrinking the KV cache without changing the model's output quality much. This is why Llama 3 and Gemma adopt GQA.

PagedAttention and vLLM. Traditional KV cache is allocated contiguously in GPU memory, causing two problems: (1) fragmentation — reserved but unused space; (2) no sharing between sequences that share a common prefix (e.g. system-prompt batches).

Borrows OS virtual memory ideas. The KV cache is divided into fixed-size *blocks* ("pages") that are mapped via a block table, analogous to page tables in virtual memory. Blocks are allocated on demand and need not be contiguous in physical GPU memory. Copy-on-write enables prefix sharing: multiple sequences sharing the same system prompt use the same physical cache pages until they diverge. **vLLM** is the production serving library built on PagedAttention.

PagedAttention reduces memory fragmentation to under 4 % and roughly doubles serving throughput compared with static pre-allocation.

Speculative decoding

Bottleneck insight. LLM decode speed is memory-bandwidth-bound, not compute-bound. Each decode step loads the full model weights from HBM to feed just a single token — utilisation of GPU tensor cores is low. Speculative decoding reframes the problem to generate multiple tokens per weight-load.

Algorithm.

1. A small *draft model* (e.g. 7B) autoregressively generates k candidate tokens cheaply.
2. The large *target model* (e.g. 70B) processes all k tokens in a single parallel forward pass, producing probability distributions for each position.
3. Each draft token is accepted or rejected via token-level rejection sampling: if the draft distribution q is "consistent" with the target p, the token is accepted; otherwise it is replaced with a corrected sample from p.
4. The process is unbiased: the accepted sequence has the exact same distribution as if the target model had generated it greedily/sampled independently.

Typical speedup: 2–3$\times$ on tasks where the draft model is accurate (code generation, continuation). On highly creative tasks with low draft acceptance, gains are minimal.

Trains the model to predict the next n tokens simultaneously using independent heads. Unlike speculative decoding (which uses a separate draft model), multi-token prediction is baked into the architecture. It functions as a form of *self-speculation*: the base model is its own draft. It also improves sample-efficiency during training by providing n gradient signals per forward pass.

Continuous batching

Static batching groups requests into a fixed batch and runs them together. When some sequences finish early and others continue, GPU cycles are wasted waiting for the longest sequence to complete before swapping in new work.

Continuous batching (also called *iteration-level scheduling* or *in-flight batching*) solves this by decoupling batch composition from individual decode steps. After every iteration, completed sequences are evicted from the batch and new requests are inserted immediately, filling the vacant slots.

Intuition

Continuous batching treats the GPU like a highway with on-ramps: new cars (requests) can join mid-journey as soon as a lane (batch slot) opens up, rather than waiting for all cars to exit at the same time.

Combined with PagedAttention (variable-length KV cache pages), continuous batching enables high-throughput serving at arbitrary sequence lengths. vLLM, TensorRT-LLM, and SGLang all implement continuous batching as their default scheduling strategy.

Real-time action chunking for robot deployment

Intuition

VLA models deployed on real robots face a latency problem. The model must predict a sequence of actions (a "chunk") fast enough that the robot does not stall between chunks. A naïve chunk boundary causes a visible pause and a sharp joint-angle jump. This is distinct from the LLM serving problem but is fundamentally an *inference scheduling* challenge.

24.6.1 The action-chunking gap

A VLA predicts an action chunk $[a_0, \ldots, a_H]$ covering a horizon H timesteps. While the robot executes this chunk, the model must begin predicting the *next* chunk. Because the model has inference latency Δt_{inf}, the second chunk is ready only at time Δt_{inf} after the query was sent. If the robot has already consumed k actions from the first chunk, a naïve switch from a_k to a'_0 (the start of the second chunk) causes a discontinuous jump in joint angles — visible as a jitter or stall.

The problem is compounded by the **overlap region**: when predicting the second chunk, the model does not know which sub-sequence of the first chunk the robot is currently executing, so it cannot smoothly continue from the right state.

24.6.2 Training-time action conditioning (TTAC)

Proposed in the paper *Training-Time Action Conditioning for Efficient Real-Time Chunking* and adopted in the Size Zero humanoid system. The core idea is to treat the overlap as an **imputing** problem:

1. During **training**, randomly mask a prefix of the target action sequence and condition the model on the *known* suffix of the previous chunk (the actions already being executed). The model learns to predict only the *remaining* unknown portion, consistent with the known prefix.

2. A **simulated inference delay** $\hat{\Delta}t$ is injected during training so the model learns to infer the missing overlap under realistic latency conditions.
3. At **inference time**, the known suffix of the currently-executing chunk is passed as conditioning, and the model fills in only the remaining unseen actions — ensuring the predicted trajectory smoothly continues from the robot's current execution state.

Key insight: action chunking discontinuity is not a hardware problem but a *conditioning* problem. By training the model to condition on partially-known action prefixes, the overlap region becomes a smooth interpolation rather than a hard boundary. No change to the robot hardware or inference pipeline is required — only a modified training objective.

24.6.3 Client-server architecture for real-time control

For humanoid whole-body control the system is split into two processes:

- **Client** (on the robot): captures observations (RGB, joint states, tactile, IMU) at 30 Hz and streams them to the server. Consumes the latest available action from a shared action buffer and sends joint commands to the low-level controller.
- **Server** (GPU desktop): runs two concurrent loops:
  - **Control loop**: continuously reads the latest action from the inference buffer and forwards it to the client — always at 30 Hz regardless of inference speed.
  - **Inference loop**: runs TTAC-conditioned VLA inference to produce the next action chunk, using knowledge of currently-executing actions to guarantee trajectory continuity.

This architecture decouples the *control rate* (fixed, robot-determined) from the *inference rate* (GPU-bound), so inference latency never causes visible pauses or joint-angle jumps.

Note

Observed effect: without TTAC, the joint-angle trajectory shows a sharp discontinuity at every chunk boundary. With TTAC, the predicted trajectory's start smoothly matches the end of the previous trajectory, eliminating jitter. The control loop ensures the robot always receives a valid command even while inference is in-flight.

Test-time Compute & Reasoning Models

The standard scaling law says: more parameters + more training compute = better models. A complementary axis is **test-time compute scaling**: investing more computation at *inference* time to improve answer quality on hard problems, without changing the model weights.

Scaling inference compute

The core idea. A model with moderate training compute but extended test-time thinking can outperform a much larger model on reasoning tasks. This is especially powerful for problems with verifiable answers (math, code, formal proofs) where correctness can be checked without human labelling.

Mechanisms.

- **Serial compute:** generate more tokens before committing to an answer (thinking tokens, scratchpads). Allows the model to self-correct and explore reasoning branches.
- **Parallel compute:** sample many candidate solutions and select the best one (best-of-N, self-consistency). Parallelisable but $N\times$ more expensive.
- **Tree search:** combine parallel and serial compute by maintaining a tree of partial solutions evaluated by a process reward model (PRM). More computationally intensive but more directed than random sampling.

Compute-optimal inference. Snell et al. (2024) showed that for a fixed inference budget, adaptive test-time compute (deciding *how much* to think based on problem difficulty) outperforms using the same budget uniformly.

OpenAI o1 / o3 architecture (known)

The **o1** family (OpenAI, 2024) represents the first widely-deployed test-time compute scaling system. Limited public information suggests:

- The model is trained with RL to produce extended "reasoning traces" (thinking tokens) before the final answer. The reasoning trace is hidden from users in the public API.
- Training uses a form of RLHF/GRPO where the RL reward signal is verifiable correctness (math benchmarks, code tests) rather than human preference.
- The model learns to use the thinking budget dynamically: easy problems produce short traces; hard problems produce long, branching internal reasoning.
- Performance scales predictably with thinking token budget: more thinking tokens $\Rightarrow$ higher accuracy on competition math (AIME, AMC) and coding benchmarks.

DeepSeek-R1 (2025): an open-weight model trained with pure RL (GRPO) on verifiable math/code problems. Demonstrates that the reasoning capability is learnable without human-labelled thinking traces: the model discovers effective reasoning patterns by reward signal alone. This is analogous to AlphaGo learning to play Go without human game transcripts.

Process reward models (PRMs)

Outcome Reward Models (ORMs) give a reward only for the final answer. They are simple to train but give sparse signal that does not help with long reasoning chains.

Process Reward Models (PRMs) give step-level rewards, evaluating the correctness of each reasoning step rather than only the final conclusion. Benefits:

- Denser training signal: the model receives feedback on every step of a multi-step proof or solution.

- Better tree search guidance: PRMs can score intermediate nodes in a search tree, enabling more efficient MCTS or beam search over reasoning paths.
- Interpretability: step-level rewards identify exactly where reasoning fails.

Training PRMs. Requires annotating not just final answers but individual reasoning steps as correct/incorrect — a more expensive labelling process. Math- Shepherd (2024) and "Let's Verify Step-by-Step" (Lightman et al. 2023, OpenAI) are key PRM papers.

Best-of-N & self-consistency

Best-of-N (rejection sampling): sample N independent completions from the model and return the one with the highest reward model score. Simple and parallelisable; quality improves roughly as $\log N$ with diminishing returns.

Self-consistency (Wang et al. 2022): for problems with a discrete final answer, sample multiple reasoning chains and return the answer by majority vote. Does not require a reward model; leverages the model's own distribution. Effective for math (multiple paths often reach the same correct answer) but less useful for open-ended tasks.

Beam search with PRM: maintain the top-B partial solutions at each step, scored by a PRM. More directed than random sampling; finds higher-quality solutions with fewer total tokens generated.

Thinking tokens & scratchpads

Thinking tokens are tokens generated by the model that are not part of the visible output but are consumed as working memory before the final answer. Claude implements this via *extended thinking* blocks (see Chapter 22); OpenAI o1 uses hidden reasoning traces.

Chain-of-thought is the prompt-level equivalent: eliciting step-by-step reasoning as visible output text (see Chapter 17). Thinking tokens extend this by making the scratchpad implicit and training the model end-to-end to use it efficiently.

Trains a transformer to *loop* its own hidden states as input for multiple iterations before generating output, similar to how recurrent networks process a sequence. Each loop is an implicit reasoning step operating in the latent space rather than the token space. The model learns to allocate more loops to harder problems. Because reasoning happens in the continuous latent space (not discrete tokens), the "thinking" is not interpretable to humans but can be computationally more efficient. This is an early instance of *latent test-time compute scaling*.

Note

Ouro is conceptually close to the "Repeat Your Self" (RYS) approach and other looped/recurrent transformer variants (see also Chapter 48 on SSMs). The common thread: rather than expressing reasoning as token strings, these models perform computation in the latent space, trading interpretability for efficiency.

Quantization & Model Compression

Quantization reduces the numerical precision of model weights (and optionally activations) from 32-bit floating point to smaller representations. The goal is to reduce memory footprint, memory bandwidth requirements, and sometimes compute cost, with minimal impact on output quality.

Post-training quantization (PTQ)

PTQ quantizes a fully trained model without any re-training. It is fast and requires only a small calibration dataset.

Weight quantization. Weights W are mapped to discrete levels:

$$W_q = \text{round}\left(\frac{W}{\Delta}\right), \quad \Delta = \frac{\max(W) - \min(W)}{2^b - 1}$$

where b is the bit-width and Δ is the scale factor (learned per tensor, channel, or group).

Granularity.

- **Per-tensor:** single scale for entire weight matrix. Cheapest; most accuracy loss.
- **Per-channel:** separate scale per output channel. Better accuracy with modest overhead.
- **Per-group (block quantization):** scale per group of g weights (e.g. $g = 128$). Used in GPTQ and AWQ; good balance of accuracy and efficiency.

Activation quantization. Quantizing activations at runtime (as opposed to just weights) is harder because activation distributions vary per input. Dynamic quantization recomputes scales on-the-fly; static quantization uses calibration statistics.

Quantization-aware training (QAT)

QAT simulates quantization noise during training so the model learns to be robust to it. Weights and activations are quantized in the forward pass using straight-through estimators (STE) for gradients. QAT consistently outperforms PTQ, especially at very low bit-widths (INT4 and below), at the cost of requiring a training run.

FP8 training (H100): NVIDIA Transformer Engine supports mixed-precision training with FP8 (E4M3 or E5M2) for matrix multiplications. The master weights are kept in BF16/FP32; FP8 is used for the compute pass. This gives roughly 2× throughput vs FP16 on H100 tensor cores.

GPTQ, AWQ, GGUF formats

GPTQ (Frantar et al. 2023): second-order PTQ method. Uses the Hessian of the weight quantization error to iteratively quantize weights while compensating for errors using the remaining unquantized weights in the same row. Achieves high quality at 4-bit weight quantization with a one-shot calibration pass. Standard for 4-bit LLM deployment on GPU.

AWQ (Activation-aware Weight Quantization, Lin et al. 2024): observes that not all weights are equally important — weights corresponding to *salient input channels* (those with large activation magnitudes) should be quantized more carefully. AWQ scales salient channels before quantization so their effective precision is preserved. Faster calibration than GPTQ with competitive accuracy; widely used in TGI and vLLM.

GGUF (llama.cpp): file format for storing quantized models for CPU/edge inference. Supports a range from Q2_K to Q8_0 (2-bit to 8-bit), with mixed-precision "K-quants" that use higher precision for attention weights and lower for FFN weights. The primary format for local inference with llama.cpp, Ollama, and LM Studio.

Note

Common deployment rule of thumb: GPTQ/AWQ INT4 for GPU serving, GGUF Q4/Q5 for CPU inference. Both achieve perplexity within 1–3% of the FP16 baseline on typical LLM sizes.

Pruning & distillation

Pruning removes weights or entire structures (heads, layers) whose contribution to the output is small.

- **Unstructured pruning:** zeroes individual weights based on magnitude or gradient sensitivity. High compression ratios possible, but sparse weights require custom hardware/kernels to benefit at inference.
- **Structured pruning:** removes entire heads, channels, or layers. Output is a smaller dense model that runs efficiently on standard hardware. Used in DistilBERT (half the layers of BERT) and ShearedLLaMA.

Knowledge distillation. A smaller *student* model is trained to mimic a larger *teacher* model's output distribution, intermediate representations, or both. The student learns a compressed version of the teacher's knowledge.

- **Response distillation:** student minimises KL divergence from the teacher's soft output probabilities (softer labels than hard ground-truth carry more information about the distribution).
- **Feature distillation:** student additionally matches intermediate layer activations of the teacher.
- **Data augmentation via distillation:** generate synthetic training data from the teacher to train the student (used in TinyLlama, Orca, etc.).

Trains LLMs with weights constrained to $\{-1, 0, +1\}$ (BitNet b1.58). Each weight is represented in 1.58 bits on average. Inference can replace matrix multiplications with addition/subtraction operations. At 3B+ parameters, BitNet b1.58 matches FP16 LLM quality while using significantly less memory and energy. Requires training from scratch (not a PTQ method); the extreme low-precision constraint must be baked into the training objective.

Distillation risks: the student inherits biases and errors from the teacher, including hallucinations. Benchmark overfitting is a concern when the teacher is used to generate training data. Legal gray area exists when commercial model outputs are used as distillation data without license clearance (see also Chapter 52).

Inference Optimization on Hardware

FlashAttention 1, 2, 3

Paged KV cache (vLLM)

Continuous batching engines

TensorRT & compilation

Running LLMs On-device

llama.cpp & GGUF

Ollama & LM Studio

MLX (Apple Silicon)

Quantization for edge

Privacy & latency trade-offs

Embedded AI & Microcontroller Inference

TensorFlow Lite & TFLite Micro

ONNX Runtime on ARM

Keyword spotting pipelines

Power & memory constraints

Edge Deployment

Model pruning for edge

Post-training quantization (INT4, INT8)

ONNX export & optimization

Neural Architecture Search (NAS)

Part VII

Prompting & Retrieval

Prompt Engineering & In-context Learning

Zero-shot & few-shot prompting

Zero-shot prompting asks the model to perform a task without any examples, relying entirely on pretrained knowledge. Effective for clear, well-defined tasks where the instruction alone is sufficient.

Few-shot prompting provides k (typically 1–8) exemplar input/output pairs in the prompt before the actual query. The model infers the pattern and generalises to new inputs without weight updates. Key properties:

- Demonstrated format strongly constrains output structure — useful for JSON, tables, and constrained classification.
- Example *quality* matters more than quantity; misleading exemplars degrade performance.
- Sensitive to example ordering (recency bias); shuffling exemplars can improve robustness.
- In LangChain, `FewShotPromptTemplate` formalises this pattern, composing a `PromptTemplate` with stored exemplar pairs and an `ExampleSelector` for dynamic retrieval from a large pool.

Chain-of-thought (CoT)

Chain-of-thought prompting encourages the model to emit a step-by-step reasoning trace before its final answer, substantially improving performance on arithmetic, commonsense, and multi-step symbolic reasoning tasks.

- **Zero-shot CoT:** append "*Let's think step by step*" (or equivalent) to the prompt. Requires no worked examples.
- **Few-shot CoT:** provide exemplar triples of (question, reasoning chain, answer) so the model imitates the reasoning style and level of detail.
- **Emergent property:** CoT is effective only above roughly 100B parameters in base models; smaller models produce incoherent rationales that hurt accuracy. Instruction-tuning (e.g. IBM Granite Instruct, GPT-4o-mini) transfers CoT capability to much smaller models via distillation on CoT traces.

CoT vs. prompt chaining — a critical distinction:

- *CoT* elicits multi-step reasoning *within a single prompt turn*; reasoning lives in the model's output for that turn.
- *Prompt chaining* sequences multiple API calls, where the output of one call becomes context for the next — a structural/orchestration pattern, not a reasoning one.

Extended thinking (Claude) extends CoT with a dedicated scratchpad block that is invisible in the conversation history, enabling deeper reasoning without consuming visible response tokens. Each thinking block carries a cryptographic signature for tamper detection in multi-turn loops (see Chapter 52).

ReAct integrates chain-of-thought *Thought* steps with *Action* steps (tool invocations) and *Observation* steps (tool results) in an alternating loop. This grounds reasoning in real-world evidence and reduces hallucination. Full coverage in Chapter 23.

Structured output prompting

Agentic pipelines require machine-parseable outputs. A tool call with a malformed JSON argument crashes the loop; a stray natural-language prefix breaks a downstream parser. Structured output is therefore load-bearing, not cosmetic.

Techniques (roughly increasing reliability):

- Describe the target schema in natural language within the system prompt.
- Provide a few-shot example of the exact JSON/XML format expected.
- Use API-level JSON mode (OpenAI) or tool schemas (Anthropic) to constrain the model's decoding to syntactically valid output.
- Use `PromptTemplate` classes (LangChain, LlamaIndex) that bake the format requirement into a reusable, parameterised template object.

Tool schema design (see also Chapter 20) defines the structured interface the model uses to call external functions: unique name, natural-language description, parameter names/types, required vs. optional fields. The description is the primary signal the model uses to decide *when* to invoke the tool.

System prompts & jailbreaks

The **system prompt** establishes the model's role, constraints, and behavioural guidelines before the conversation begins. It is the primary mechanism for customising base model behaviour without fine-tuning.

In agentic systems the system prompt typically:

- Defines the agent's persona, available tools, and expected output format.
- Lists prohibited actions and safety rules ("*in-context*" controls).
- Embeds Skills documentation (e.g. Cowork's SKILL.md files) encoding task-specific best practices that fire reliably for specific requests.
- Is cached at the API level (Anthropic prompt caching, OpenAI prefix caching) so it does not re-incur compute cost on every turn.

The fundamental weakness of system-prompt safety: in-context instructions only constrain the model if it "chooses" to follow them. Sufficiently adversarial inputs — prompt injection via observed web/email content, context overflow, or multi-step reasoning-around-the-rules — can cause the model to ignore them. This is why out-of-process runtime enforcement (OpenShell, VM sandboxing) is needed as a backstop (Chapter 42).

Jailbreak techniques include: role-play framing ("pretend you have no restrictions"), token manipulation (unusual Unicode, homoglyphs), base64 encoding of forbidden requests, prompt injection via tool results, and suffix-appending adversarial attacks. Defences: constitutional AI (Chapter 43), input/output classifiers, and runtime enforcement (Chapter 42).

RAG & Long-context Retrieval

Naive RAG pipeline

Retrieval-Augmented Generation (RAG) grounds LLM responses in external knowledge without retraining model weights. The canonical pipeline has two phases:

Offline (indexing):

1. Load source documents (PDFs, web pages, databases) via document loaders.
2. Chunk documents into retrievable units (see §32.3).
3. Embed each chunk with a dense embedding model.
4. Store embeddings + raw text in a vector store.

Online (query time):

1. Embed the user query with the same model.
2. Run approximate nearest-neighbour (ANN) search to find top-k chunks.
3. Prepend retrieved chunks as context in the prompt.
4. LLM generates an answer conditioned on context.

LangChain components covering this pipeline:

- **Document loaders:** Dropbox, Google Drive, OneDrive, YouTube, PubMed, URLs, Airtable, Figma, Notion, Pandas, MongoDB, Trello, and 100+ more.
- **Text splitters:** `RecursiveCharacterTextSplitter` (default), sentence splitters, token-aware splitters, semantic splitters.
- **Vector stores:** 50+ integrations including Chroma, Pinecone, Weaviate, pgvector, FAISS; represent chunks as dense embeddings for low-latency queries.
- **Retrievers:** abstract interface accepting a query string, returning a `Document` list; decouples retrieval logic from the store backend.
- **Memory:** full conversation history, summarisation-based, or n-most-recent exchange buffers to maintain multi-turn context.

Embedding models & vector DBs

Chunking strategies

Chunking strategy is one of the highest-leverage decisions in a RAG system: it determines the granularity of what can be retrieved and how much context each chunk carries.

Static approaches (no LLM required):

- **Fixed-size / sliding window:** split every n tokens with an m-token overlap. Simple and predictable but ignores document semantics.
- **Sentence / paragraph:** split on linguistic boundaries. Better cohesion but variable lengths can overflow the retriever's embedding dimension budget.
- **Recursive character splitting:** try progressively smaller separators (`\n\n`, `\n`, space, character) until chunks fit the target size. The LangChain default; good general-purpose baseline.
- **Semantic splitting:** embed consecutive sentences and split where cosine distance between adjacent embeddings exceeds a threshold. Higher quality, more expensive.

Agentic chunking: An LLM agent determines chunk boundaries dynamically by reading the document and reasoning about semantic coherence and logical structure — treating chunking as a comprehension task rather than a rule-based process. Produces higher-quality, more coherent chunks at significantly greater token cost. Best suited for high-value corpora where retrieval quality justifies inference spend.

Retrieval methods: dense, sparse, hybrid

The retrieval step is the most critical quality lever in a RAG system. Three main approaches exist:

Dense retrieval: Documents and queries are embedded into dense vector representations; retrieval returns the top-k nearest neighbours by cosine or dot-product similarity.

- Good at: semantic matching, paraphrases, meaning-based queries ("what causes inflation" retrieves relevant content even without the word "inflation").
- Weakness: can miss exact keyword matches (names, IDs, version numbers, unique terms). A query for "v2.3.1 release notes" may retrieve semantically related content about other releases.

Sparse retrieval (BM25 / TF-IDF): Keyword-based scoring; documents ranked by term overlap with the query, weighted by term frequency and inverse document frequency.

- Good at: exact matches, named entities, product codes, technical terms.
- Weakness: no semantic flexibility; "automobile" and "car" are treated as unrelated terms.

Hybrid retrieval: Combines dense and sparse scores (typically via reciprocal rank fusion or a linear combination). This is the recommended default for production RAG systems because:

- Dense catches semantic meaning; sparse catches exact terms.
- Most real-world queries contain both semantic intent and specific keyword constraints.
- Adding sparse to a dense system is cheap (BM25 is fast and requires no GPU).

RAG vs fine-tuning vs in-context learning — when to use which:

Use RAG when	Use fine-tuning when	Use in-context when
Facts change often	Style/tone must be consistent	Task fits in the prompt
Private docs involved	Task patterns repeat at scale	Fast iteration needed
Citations required	Output format is rigid	No training data available
No retraining budget	Latency must be minimised	

Reranking & advanced RAG

Advanced RAG wraps the naive pipeline with pre- and post-retrieval optimisation steps:

- **Query rewriting:** reformulate the user query before embedding retrieval. HyDE (Hypothetical Document Embeddings) generates a hypothetical answer and embeds *that* instead; step-back prompting abstracts the query to a higher-level principle.
- **Reranking:** a cross-encoder model (e.g. Cohere Rerank, BGE-Reranker) re-scores the top-k ANN candidates against the query with full attention, then passes only top-m to the context window.
- **Contextual compression:** a post-retrieval LLM filters each chunk to only the sentences most relevant to the query, reducing noise in the prompt.
- **Multi-query retrieval:** generate n paraphrases of the query, retrieve for each, deduplicate (by hash or semantic similarity), then rerank the merged set.

Agentic RAG replaces the fixed retrieve-then-generate pipeline with an agent that can iterate:

- **Adaptive retrieval:** the agent decides *whether* to retrieve at all, which knowledge source to query, and whether a retrieved result is sufficient to answer — queries traditional RAG cannot express.
- **Iterative refinement:** if retrieval is insufficient, the agent reformulates the query and retrieves again, possibly from a different source.
- **Multi-source fusion:** the agent queries heterogeneous backends (vector DB, SQL, web search, REST API) and synthesises a coherent answer.
- **Semantic caching:** frequently retrieved query–result pairs are cached at the agent level to avoid redundant embedding lookups across turns.
- **Output validation:** the agent can verify retrieved facts against a secondary source before including them in the final response.

Traditional RAG vs Agentic RAG

Traditional RAG: one retrieval step, static rules, single knowledge base, no validation, cannot iterate. Agentic RAG: adaptive strategy, multi-source, iterative, self-validating, full tool-use loop. The cost is latency and token spend; the benefit is accuracy on complex multi-step queries.

Long-context vs retrieval trade-offs

Part VIII

Agent Systems

AI Agent Fundamentals & Architectures

Agent loop: perceive, plan, act

ReAct framework

Reflexion & self-critique

Agent scaffolding patterns

Tool Use, Function Calling & MCP

Function calling APIs

Modern LLM APIs (OpenAI, Anthropic, Google) support *tool use* (also called function calling): the model can emit a structured tool-invocation request instead of plain text, the caller executes it, and the result is fed back in the next turn.

Anthropic Messages API structure, as observed in Claude Code JSONL logs:

- A `tool_use` content block in an assistant message contains `id`, `name`, and `input` (a JSON object matching the tool's schema). Multiple tool calls can appear in the same content array.
- Tool results are returned by the caller as a `user` turn containing a `tool_result` content block with the matching `tool_use_id`. This means the API's call/result cycle always alternates user/assistant turns, even though the "user" message is machine-generated.
- Each result in the JSONL carries a `toolUseResult` wrapper with `stdout`, `stderr`, `interrupted` (boolean, set if the user killed the process), `isImage` (for screenshot tools), and `noOutputExpected`.
- The `sourceToolAssistantUUID` field links the result back to the exact assistant event that issued the call, enabling causal tracing across the log.

Tool loop termination. The loop ends when the model emits a `stop_reason` of `end_turn` (no more tool calls) or `max_tokens`. Agents must handle the `tool_use` stop reason to detect mid-response tool calls vs. a completed response.

Model Context Protocol (MCP)

MCP (Anthropic, 2024) is an open protocol that standardises how agents connect to external tools and data sources. It separates the *tool implementation* (an MCP server) from the *model* (an MCP client), so any compliant agent can use any compliant server without custom glue code.

- MCP servers expose *tools*, *resources* (data the model can read), and *prompts* (reusable prompt fragments) over a JSON-RPC interface via stdio or HTTP/SSE.
- In Cowork, MCP connectors provide authenticated access to external services (Slack, Asana, Google Drive, etc.). The agent never handles raw credentials; authentication is managed by the MCP server layer.
- A central registry (`mcp-registry`) lets agents discover available connectors at runtime, enabling dynamic tool acquisition.

Tool schema design

A tool schema is the structured definition that tells the model what a tool does and how to call it. Schema quality directly determines whether the model chooses tools correctly and forms valid arguments.

Schema fields (standard across APIs):

- **Name:** a unique, snake_case identifier. The model uses this to reference the tool in its tool_use output. Collision-free naming matters in large tool sets.
- **Description:** natural-language explanation of what the tool does, when to use it, and when *not* to use it. This is the primary signal the model uses for tool selection — a bad description causes wrong tool choice far more often than a bad parameter schema.
- **Parameters:** a JSON Schema object defining the tool's input fields, their types, descriptions, constraints (enum, pattern, minimum/maximum), and which are required vs. optional.
- **Tool call ID:** a unique tracking ID (assigned by the API) linking each invocation to its result in multi-tool conversations.

Design heuristics:

- Keep tool names semantically distinct; `search_web` vs. `search_docs` is clearer than `search1` vs. `search2`.
- Descriptions should include *when to use* and *when not to use*. Example: "Use this tool to look up current weather. Do not use for historical weather data; use `weather_history` instead."
- Limit required parameters. Optional parameters reduce model friction; use sensible defaults and document them.
- For tools with large output surfaces (filesystem reads, web fetches), document the output format so the model can parse results correctly.
- Test schemas by inspecting which tool the model selects for edge-case prompts; iterate on descriptions before parameters.

Combined with RAG: tool-calling and RAG complement each other. The model can use a `search` tool to retrieve documents, then consolidate multiple retrieval results and reduce redundant API calls, producing a richer and more contextually grounded response than either approach alone.

Error handling in tool loops

Tool failures must be communicated back to the model so it can adapt rather than silently stall:

- The Anthropic API's `tool_result` block accepts an `is_error: true` flag alongside the error message. The JSONL log confirms this: permission errors (e.g. `EACCES`) are returned as error results with the same `tool_use_id`, not as exceptions in the outer call.
- **Interrupted tools.** The `interrupted` field in `toolUseResult` signals that a long-running process was killed mid-execution (e.g. the user cancelled). The agent should treat partial output as unreliable.
- **Retry heuristics.** Well-designed agents distinguish between transient errors (network timeout — retry), permission errors (wrong path — try alternate approach), and hard failures (bad schema — surface to user). Blindly retrying permission errors wastes turns and context budget.
- **Context-budget awareness.** Because tool results consume input tokens, agents should prefer tools that return concise outputs (e.g. `Grep` over full-file `Read`) when context is constrained.

Agent Memory: Episodic, Semantic, Working

In-context (working) memory

The model's context window is its working memory: it holds the full conversation history, tool results, and system instructions for the current turn. Everything not in the context window is, from the model's perspective, forgotten.

Practical implications observed in production agents:

- **Context pressure.** As a session grows, input tokens increase and latency / cost rise. Prompt caching mitigates this for stable prefixes (system prompts, skills documentation) but not for the growing conversation tail.
- **Shell-snapshots.** Claude Code writes `shell-snapshots/` entries to preserve shell state (environment variables, working directory) between Bash tool calls, effectively extending working memory to include filesystem cursor state.
- **Extended thinking.** Thinking blocks give the model a scratchpad within a single turn that is not part of the official assistant message — an intra-turn working memory separate from the conversational context.

External episodic memory

Episodic memory stores a timestamped record of what happened, so the agent can recall prior sessions or survive context-window resets.

JSONL conversation logs (as implemented in Claude Code) are the clearest real-world example. Each event in a session is appended as a JSON line to `~/.claude/projects/{project-slug}/{session-uuid}.js`
Key design choices:

- **Append-only.** New events are always appended; nothing is deleted or mutated. This makes the file a reliable audit log and enables trivial crash recovery.
- **Linked-list structure.** The `parentUuid` field on each record points to the preceding event, forming a singly-linked list (or a tree when conversations branch via `isSidechain: true`). Replaying from the head reconstructs the full conversation.
- **Turn grouping via `promptId`.** All events in a single user turn (the human message, multiple assistant chunks, and all tool results) share a `promptId`, enabling efficient retrieval of a complete turn without walking the full chain.
- **Rich metadata.** Each record stores `cwd`, `gitBranch`, `version`, `userType`, and token-usage breakdowns, turning the log into a queryable audit trail for debugging and cost analysis.
- **Separation from semantic memory.** A sibling `memory/` directory holds structured facts the agent has been explicitly asked to remember — distinct from the raw episodic log. This mirrors the episodic/semantic distinction in cognitive science.

Semantic memory via vector DBs

Semantic memory stores structured factual knowledge (facts, rules, definitions) in a form the agent can query across sessions. Unlike episodic memory, it has no inherent chronology — it stores *what is true*, not *what happened when*.

Five memory types from the CoALA framework (Cognitive Architectures for Language Agents, Princeton):

- **Short-term (working):** rolling buffer inside the context window. Coherent across a session, forgotten when the window rolls over.

- **Long-term:** persists across sessions via external storage. Crucial for personalisation, domain expertise, and continuity. RAG is the primary LTM technique.
- **Episodic:** records of specific past events — actions taken, outcomes observed. Enables case-based reasoning: "last time I saw this error, X fixed it." Implemented by logging event sequences (e.g. JSONL session logs).
- **Semantic:** structured factual knowledge. Used in legal/medical diagnosis and enterprise knowledge management. Implemented via knowledge graphs, vector embeddings, or symbolic AI.
- **Procedural:** skills and learned behaviours enabling automatic task execution. Implemented via reinforcement learning or LoRA fine-tuning on task demonstrations. Analogous to human muscle memory.

Vector database as semantic store:

- Facts, documents, and conversation summaries are embedded into dense vectors and stored in a vector DB (Chroma, Pinecone, Weaviate, pgvector).
- At query time, the agent embeds its current query and retrieves the top-k semantically similar records via ANN search, injecting them into the context.
- **Personalisation loop:** as the agent accumulates interactions, new user preferences, project facts, and domain knowledge are embedded and stored. Future sessions retrieve only what is relevant to the current task.
- **Balance constraint:** storing too much leads to slow retrieval and noisy context; storing too little loses important facts. Relevance scoring and TTL expiry policies help prune stale entries.

Context engineering

While "prompt engineering" focuses on crafting the right instruction for a single turn, **context engineering** is the discipline of designing and managing *everything* placed in the context window across an agent's full lifecycle: system prompts, tool schemas, retrieved documents, conversation history, and memory injections.

As agents operate over long horizons and with large tool surfaces, context management becomes a first-class engineering concern. The key resources from Manus's engineering team summarise the practical lessons:

Context window as shared state. The context is not just instructions — it is the primary medium through which the agent perceives the world, maintains continuity, and communicates with tools. Every byte in the context competes for the model's "attention budget".

Core context engineering principles:

- **Explicit state serialisation.** Rather than relying on the model to infer state from conversation history, agents should serialise relevant state (current plan, completed steps, open tasks) into structured blocks at the start of each turn. This prevents context drift as the window grows.
- **Selective history retention.** Keep full fidelity for recent turns (last 2–5) and compress or summarise older turns. Tool call results should be trimmed to their essential outputs; raw stdout of thousands of lines is wasted context.
- **Chunked retrieval injection.** Retrieved documents and memory fragments should be injected near the task description (not buried at the end), with explicit relevance labels so the model knows why each fragment was included.
- **Tool schema placement.** Tool descriptions compete with task context for tokens. For agents with many tools, hierarchical tool loading (load only the relevant tool schema for each step) reduces noise and keeps task-relevant tokens more salient.

- **KV cache alignment.** For repeated agentic calls sharing a common system prompt and tool suite, structure the context so the stable prefix (system prompt + tool definitions) is identical across calls. This enables prompt caching to amortise the cost of the static portion across many agent steps.

> **Note**
>
> "Context engineering" is the term coined by practitioners at Manus (2025) and popularised via the *Context Engineering for AI Agents: Lessons from Building Manus* blog post. It reframes the craft of building agents away from individual prompts toward systematic management of the entire information flow within the context window.

Context engineering 2.0. As context windows grow to 1M+ tokens, the problem shifts from fitting information in to filtering what to prioritise. Research on "lost-in-the-middle" (Liu et al. 2023) shows that models systematically under-attend to content in the middle of very long contexts. Mitigation strategies include placing critical facts near the beginning or end, and using structured XML/Markdown delimiters to make information boundaries salient.

Memory consolidation strategies

Consolidation moves information from episodic logs into more compact, reusable forms:

- **Explicit summarisation.** At the end of a session (or when context pressure builds) the agent can be prompted to write a summary of key decisions and facts into the `memory/` store, discarding raw turn-by-turn detail.
- **Selective retrieval.** Rather than replaying the full JSONL log, an agent can embed past turns and retrieve only the k most relevant episodes for the current task — combining episodic storage with a vector-DB retrieval step.
- **Plan persistence.** The `plans/` subdirectory in `.claude/` acts as a form of procedural memory: multi-step plans survive context resets and can be resumed in a new session.
- **Trade-off.** Compact semantic memory loses provenance; raw episodic logs are complete but expensive to re-inject. Hybrid approaches maintain a compressed "working summary" alongside the full log.

Planning: MCTS, CoT, o1-style Reasoning

Classical planning (STRIPS, PDDL)

Monte Carlo Tree Search (MCTS)

Chain-of-thought & scratchpads

Chain-of-thought prompting produces a step-by-step reasoning trace inside the model's output (see Chapter 17 for prompting mechanics). In the context of planning, CoT enables a model to *decompose* a complex goal into sub-goals before committing to an action.

Scratchpad variants:

- **Visible CoT:** reasoning is part of the assistant turn and is seen by the user. Interpretable but verbose.
- **Extended thinking / thinking blocks:** reasoning in a hidden scratchpad that is signed but not shown to the user (Anthropic Claude o-style). Allows deeper multi-step reasoning without polluting the response.
- **Scratchpad in-weights:** test-time compute scaling (o1/o3 style) uses learned "thinking tokens" at inference; see Chapter 49.

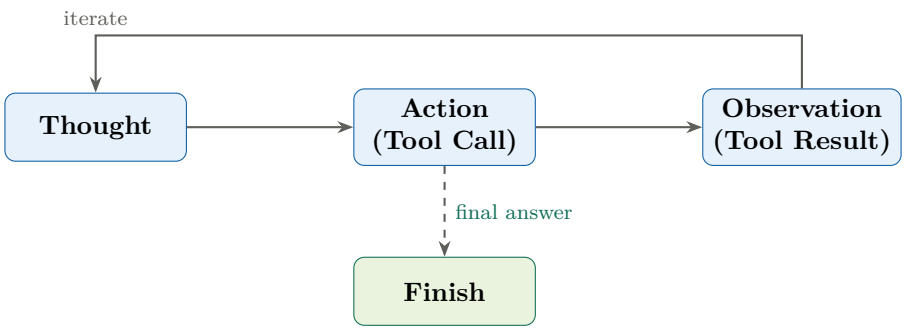
ReAct & agent planning loops

ReAct interleaves chain-of-thought reasoning traces with concrete tool-use actions in a tight loop, grounding abstract reasoning in real-world observations.

The ReAct loop alternates three step types in sequence:

1. **Thought** — the model reasons about its current state, what information it lacks, and what action to take next (CoT-style natural language).
2. **Action** — the model invokes a tool (search, calculator, API call, code execution) with structured arguments.
3. **Observation** — the tool result is returned and appended to context; the model reads it and produces the next Thought.

The loop terminates when the model emits a **Finish** action with a final answer, when a maximum iteration count is reached, or when a confidence threshold is satisfied.



Advantages over pure CoT:

- Reduces hallucination by grounding each reasoning step in retrieved evidence.
- Enables the model to course-correct mid-loop when an observation contradicts its prior reasoning.

- Produces an interpretable audit trail: thought → action → observation triples are logged (see JSONL session format, Chapter 52).

Multi-agent ReAct: a large orchestrator model handles high-level reasoning and Thought/Finish steps while delegating Action steps (tool calls) to smaller specialised subagents.

ReWOO (Reason Without Observation): An alternative to ReAct that separates the planning and execution phases. Rather than interleaving Thought and Observation, ReWOO generates a complete plan upfront (a sequence of tool calls with dependencies), then executes them, then synthesises the results. Benefits: fewer LLM calls (cheaper), plan is inspectable before execution. Cost: cannot course-correct mid-plan based on intermediate observations.

Plan-and-Execute

Plan-and-Execute separates agent reasoning into two distinct phases, unlike ReAct which interleaves them:

1. **Planner phase:** the model receives the goal and produces a structured multi-step plan *before any tool is called*. Example: "1. gather company background, 2. retrieve financial data, 3. compare competitors, 4. write report."
2. **Executor phase:** a (potentially different, cheaper) model or module carries out each step in sequence, passing results forward.

Compared to ReAct:

	ReAct	**Plan-and-Execute**
Planning	Interleaved with actions	Upfront, then separate
Adaptability	High (course-corrects mid-loop)	Low (plan commits early)
LLM calls	More (each step)	Fewer for planning
Debuggability	Harder (emergent path)	Easier (plan is inspectable)
Best for	Open-ended, adaptive tasks	Structured, predictable tasks

ReWOO is a close relative: both front-load planning; ReWOO additionally chains tool-call dependencies explicitly in the plan so the executor knows which earlier results to substitute (avoiding redundant re-retrieval).

Hybrid approaches: production agents often combine patterns — an orchestrator produces a plan (Plan-and-Execute style), then each step executes using a ReAct inner loop that handles step-level surprises, with human-in-the-loop approval at major plan transitions (LangGraph interrupt nodes).

o1 / o3 test-time compute scaling

Multi-agent Systems & Orchestration

Orchestrator-subagent patterns

In an orchestrator-subagent architecture, a primary (orchestrator) agent decomposes a task and delegates sub-tasks to specialised child agents. The orchestrator maintains the high-level plan; subagents handle isolated, bounded work.

Claude Code / Cowork implementation (observed directly):

- The orchestrator has access to an `Agent` tool that launches a subagent as a subprocess. The subagent type determines which tools it receives (e.g. `general-purpose`, `Explore`, `Plan`).
- Subagents are *isolated*: each receives a fresh context containing only the task prompt passed by the orchestrator, plus any tool results it generates. They do not share the orchestrator's conversation history.

- Subagents can be run **in parallel** (multiple `Agent` tool calls in one assistant turn) when subtasks are independent, dramatically reducing wall-clock time for multi-step research or multi-file operations.
- A subagent can be **resumed** by passing its `agent_id` back to the `Agent` tool, preserving its full transcript for follow-up work without re-incurring input token costs.
- **Worktree isolation** (`isolation: "worktree"`) gives a subagent its own git worktree so it can make changes without touching the orchestrator's working tree; the worktree is auto-cleaned if no changes are made.

The key design tension is *isolation vs. context sharing*: fully isolated subagents avoid context pollution but may duplicate work; shared-context agents collaborate more fluidly but risk token bloat and unintended interference.

AutoGen, CrewAI, LangGraph & MetaGPT

AutoGen (Microsoft) builds agents around *conversational turns*. Two agents (or a human proxy and an agent) exchange messages to jointly solve a task. `GroupChat` enables n-way conversations with a group manager that routes each turn to the right participant. Execution is controlled via reply functions and termination conditions.

CrewAI models a multi-agent system as a *crew* of workers with defined roles, backstories, goals, and tool access. Tasks are assigned to crew members and can run sequentially or in parallel. Encourages role-based thinking (researcher, writer, reviewer) rather than low-level code primitives.

LangGraph provides a *stateful graph* execution model. Agents and tools are nodes; edges encode control flow (sequential, conditional, cyclic):

- State is a typed dict threaded through every node, making data flow inspectable and making bugs reproducible.
- Cycles are first-class: ReAct-style Thought/Action loops map naturally to a graph edge pointing back to an earlier node.
- **Human-in-the-loop:** interrupt nodes pause execution for human review/approval before the graph continues.
- Integrates with MCP servers via `langchain-mcp-adapters` for plug-and-play tool server compatibility.

MetaGPT simulates a *virtual software company* using SOPs:

- Roles: product manager, architect, project manager, engineer, QA engineer.
- Input: one-line requirement. Output: PRD, architecture doc, API specs, code, tests.
- **Standard Operating Procedures (SOPs):** human workflows encoded as prompt sequences; each agent's output is the structured input to the next.
- **Publish-subscribe message pool:** all inter-agent messages are structured artefacts (documents, diagrams) stored in a global pool, decoupling senders from receivers and reducing natural-language parsing errors.

MetaGPT: one-line requirement → full codebase

MAS network topologies:

- **Centralized:** a central coordinator holds global knowledge and routes tasks to agents. Easy to monitor; single point of failure.
- **Decentralized:** agents share state only with their neighbours. More fault-tolerant and modular; coordination is harder.
- **Hierarchical:** tree structure with varying autonomy per layer. Natural for enterprise workflows where high-level strategy delegates to operational agents.
- **Holonic:** agents are grouped into *holons* (a holon is a unit that is both a whole and a part). A holon's leading agent appears to the outside world as a single agent; internally it manages a sub-team.
- **Coalition / Team:** agents temporarily unite to achieve a shared goal, disbanding afterward. Teams are more tightly coupled; coalitions are ad hoc.

Inter-agent communication

Direct agent-to-agent integration suffers from an $n(n-1)/2$ integration-point problem: n agents each needing a bespoke connector to every other agent. Protocol standards collapse this to n connections to a shared message bus.

Model Context Protocol (MCP) — Anthropic, 2024:

- Focus: one model, many tools. Standardises how a single agent enriches its context via external servers.
- Transport: JSON-RPC over stdio or HTTP/SSE.
- Servers expose *tools*, *resources*, and *prompts*.
- Limitation for multi-agent: no native agent-to-agent discovery, no delta streaming, no shared memory primitive.

Agent Communication Protocol (ACP) — IBM (merged with A2A):

- Focus: agent-to-agent communication across frameworks and organisations.
- Transport: REST over standard HTTP (no SDK required; cURL/browser compatible).
- **Async-first:** designed for long-running tasks; synchronous mode also supported.
- **Offline discovery:** agents embed metadata in their distribution packages so other agents can discover capabilities without a central registry.
- Enables orchestration across heterogeneous frameworks without custom glue.

Agent2Agent Protocol (A2A) — Google:

- Designed for cross-system and cross-organisation agent communication.
- More natural for Google-ecosystem integrations.
- Can coexist with ACP; each serves different primary use cases.

MCP vs ACP in one line

MCP = one model ↔ many tools. ACP = many agents ↔ many agents.

Routers, state machines & workflow engines

As agent pipelines grow, structured control-flow primitives prevent LLM-driven decision-making from becoming a runaway cost and reliability problem.

Router: decides which tool, subagent, or branch to invoke. Implementations:

- LLM classifier — flexible, expensive; use for open-ended routing.
- Embedding similarity match — fast, moderate generality.
- Deterministic rule — fastest, requires explicit case enumeration.

State machine / workflow engine: agent progression through phases is explicit. Each state = a task phase (planning, retrieving, synthesising, reviewing); transitions are triggered by events or conditions. LangGraph implements this directly.

Benefits over fully open-ended ReAct:

- Deterministic paths for known task types (no unnecessary LLM calls).
- Explicit human-in-the-loop checkpoints at state transitions.
- Each state is independently testable.
- Defined failure handling per state, rather than global "if error, retry."

Agent vs workflow

A *workflow* uses fixed steps and deterministic transitions. An *agent* makes dynamic decisions at each step. Many production "agents" are really workflows with one or two LLM-decision nodes — often the better engineering choice.

Swarm intelligence analogies

Swarm intelligence studies how simple local rules among agents produce complex, emergent global behaviour without central control. Canonical examples: ant colony optimisation (ACO), particle swarm optimisation (PSO), bee colony algorithms.

Analogies to LLM multi-agent systems:

- **Stigmergy:** ants communicate indirectly via pheromone trails left in the environment. In MAS, agents communicate indirectly via a shared message pool (MetaGPT), a vector database, or a shared workspace — they don't call each other directly.
- **Emergent division of labour:** no agent is told to specialise; roles emerge from local interactions. In LLM swarms (e.g. Society of Mind prompting), assigning diverse "perspectives" to agents produces richer collective outputs than a single agent.
- **Robustness through redundancy:** decentralised MAS topologies mirror swarms: if one node fails, the system degrades gracefully rather than collapsing (contrast with centralised architectures with a single point of failure).
- **Limits of the analogy:** LLM agents are expensive, stateful, and context-limited; true swarms use thousands of cheap stateless units. Current multi-agent LLM systems typically operate at $n < 20$ agents.

Part IX

Agent Platforms & Tooling

OpenClaw & Agent OS Integration

OpenClaw architecture overview

The NVIDIA stack for enterprise agentic deployment is composed of four distinct layers, each with a different responsibility:

- **OpenClaw** — the agent/application layer. This is the thing that plans, calls tools, writes code, reads files, and tries to complete tasks. Think of it as the agent "brain" and behaviour loop.
- **OpenShell** — the runtime governor. It sits *between* OpenClaw and the real machine, network, and models, and decides what the agent is actually allowed to do. NVIDIA describes it as governing how the agent executes, what it can see and do, and where inference is routed.
- **NemoClaw** — the packaged enterprise distribution. It takes OpenClaw and adds the missing operational pieces: safer setup, policy controls, privacy routing, model plumbing, and sandboxing. Installed in a single command via NVIDIA Agent Toolkit software.
- **Nemotron** — NVIDIA's family of open models (reasoning, coding, visual understanding, safety, and agentic workflows). In this stack, Nemotron is the local or NVIDIA-hosted model option that NemoClaw/OpenShell can route inference to, especially when enterprises want on-prem execution.

A useful one-line shorthand for each layer:

OpenClaw = agent behaviour Nemotron = model family OpenShell = execution/security runtime NemoClaw = packaged deployment stack

The key design philosophy is that "wrapper" is the right mental model for NemoClaw: it does not replace the agent brain; it adds a controlled execution environment *around* it. The core question OpenClaw answers is *"how do we make agents capable?"*; OpenShell answers *"how do we make capable agents safe to run?"* NemoClaw bundles both answers into something an enterprise can actually operate.

OS-level agent primitives

OpenShell exposes three core architectural blocks that together form an agent OS:

1. **Sandbox** — the isolated environment where the agent runs. Mistakes or malicious/confused behaviour are contained inside the sandbox boundary and cannot reach the host OS or other sessions.
2. **Policy engine** — checks what actions are allowed across the filesystem, network, and process layers before any action executes. Rules are defined by the operator/enterprise, not the agent.
3. **Privacy router** — decides whether a given inference request or data payload should stay on a local model (e.g. a local Nemotron instance) or may be forwarded to a frontier cloud model. The routing decision is based on operator cost/privacy policy, explicitly *not* on what the agent wants to do.

Action interception flow. When OpenClaw attempts any action (read a file, call a tool, invoke a model), OpenShell intercepts it before execution:

1. Agent requests an action.

2. Policy engine evaluates whether the action is permitted under current rules.
3. Privacy router determines where any associated inference should go.
4. Action executes only if policy allows it; otherwise it is blocked and the agent receives an error or a constrained alternative.

Example policy rules:

- HR data must stay on-device; only an approved local Nemotron model may see it.
- Code repository content may be routed to an approved cloud model.
- Outbound network requests are limited to a specified domain whitelist.
- The agent may read `/workspace/invoices` but not `/home/user/.ssh`.
- Only certain folder paths are writable; cloud model calls require scrubbed input.

Filesystem & process access

OpenShell implements controls at the OS primitive level, not just through soft system prompt instructions. Four categories of guardrails are enforced at runtime:

- **Filesystem limits.** Scoped read/write access per session. An agent handling support tickets can be restricted to `/tickets/current` and explicitly denied access to credential stores or other user home directories.
- **Blocked network egress.** Outbound connections require policy approval. An enterprise can prevent customer PII from leaving the perimeter by blocking unapproved external endpoints at the network namespace level.
- **Process restrictions.** Limits on what processes the agent can spawn (e.g. no arbitrary shell exec, no privilege escalation).
- **Controlled inference backends.** The agent cannot freely choose which model API to call; the privacy router intercepts model calls and enforces routing policy.

Specific kernel-level mechanisms cited in NVIDIA's OpenShell documentation: **Landlock** (Linux kernel filesystem access control), **seccomp** (system call filtering), and **network namespaces** (network isolation per process group). These are the same primitives used in container runtimes like Docker/gVisor, adapted for agentic workloads.

Security sandboxing

The most important design idea in OpenShell is **out-of-process policy enforcement**: moving the real control point *outside* the agent, so the agent cannot override it even if its reasoning is compromised, confused, or prompt-injected.

The problem with in-agent safety. Most agent safety today is "inside the agent": system prompts, behavioural rules, tool descriptions that say "don't do X". These soft protections can fail if the agent loses context, is prompt-injected via observed content, or simply reasons its way around them. In-context instructions cannot enforce themselves.

OpenShell's answer. By intercepting actions at the OS/runtime layer before they execute, OpenShell provides hard enforcement that the agent model cannot circumvent. NVIDIA compares this to a browser security model: sessions are isolated, and permissions are checked at the execution boundary, not just in the agent's "mind". The agent may *believe* an action is permitted; OpenShell decides whether it actually executes.

This is why OpenShell is better described as:

- a runtime governor

- a policy firewall
- a sandbox manager
- an inference router

. . . rather than as a model, a chatbot, or a prompt-engineering layer.

Kernel-level isolation. NVIDIA explicitly targets kernel-level sandboxing (Landlock, seccomp, network namespaces) rather than process-level or container-level approaches, giving stronger isolation guarantees for long-running autonomous sessions.

Desktop automation patterns

Enterprise deployment example: bank support ticket agent.

Without a controlled runtime, an unconstrained agent processing support tickets might:

- read too many internal files beyond those needed for the task
- leak customer PII to a cloud model in its prompt
- execute unsafe shell commands with unexpected side-effects
- POST data to an unapproved external endpoint

With NemoClaw/OpenShell, the operator configures policy such that:

- customer PII stays on-prem; only an approved local Nemotron model sees raw ticket text
- only summarised, scrubbed data may be forwarded to a frontier cloud model
- the agent may access `/tickets/current` but not engineer laptops or credential folders
- outbound network access is limited to an approved internal API whitelist

The enterprise value proposition is therefore not "make the agent smarter" but **"make the agent deployable"**. The big blocker for long-running agents in production is not model quality — it is *trust*. NVIDIA frames this explicitly: the missing primitives are sandboxing, permissions, and isolation. If a safe runtime layer exists, enterprises can let agents run longer, touch more systems, and do useful work without granting unconstrained access.

Comparison to the Cowork/Claude Code approach. Both NemoClaw/OpenShell and Cowork follow the same fundamental pattern — a VM or sandbox separates the agent from the host, and policy tiers control what the agent can reach — but at different deployment targets:

- Cowork targets individual/team desktop users: single-session Ubuntu VM, user-selected workspace mount, product-level prohibited-action lists.
- NemoClaw/OpenShell targets enterprise IT: kernel-level isolation, org-defined policy YAML, privacy routing between on-prem and cloud models, multi-session scale.

AI Coding Agents & IDEs

Landscape overview

Cursor: architecture & edit model

GitHub Copilot & Codex

Windsurf (Codeium)

Antigravity

Devin & SWE-agent

Aider & CLI coding agents

CLI-based coding agents run entirely in a terminal, wrapping an LLM with a persistent conversation loop, file I/O tools, and shell execution.

Aider is an open-source CLI agent that maintains a *repo map* (built with ctags/tree-sitter) to give the model structural context without flooding the context window with raw source. It applies model-generated edits via SEARCH/REPLACE blocks or unified-diff format, and supports multi-model backends (GPT-4o, Claude, Gemini, etc.).

Claude Code (Anthropic, 2024–) is Anthropic's official CLI agent, built on the Claude Agent SDK. Observations from direct inspection of live session JSONL logs:

- **Session persistence.** Every conversation is stored as a newline-delimited JSON file at `~/.claude/projects` The project slug is the working-directory path with `/` replaced by `-`, so each repository gets its own project namespace.
- **Message graph.** Each JSONL entry carries `uuid` and `parentUuid`, forming a linked list (or tree when the user branches). A `promptId` groups every event (user message, assistant chunks, tool results) that belongs to the same conversational turn. The boolean `isSidechain` marks branch points.
- **Event types.** Three kinds of records appear in the log:
  - `queue-operation` (enqueue / dequeue) — internal concurrency plumbing so messages do not collide when dispatched rapidly.
  - `user` — human messages *and* tool-call results (see below).
  - `assistant` — model output, written incrementally as chunks arrive; a single logical response often spans several consecutive lines.
- **Tool results are injected as `role: user`.** This is a direct consequence of how the Anthropic Messages API works: tool results are sent back to the model in a `user` turn, not as assistant output. Each result entry carries a `sourceToolAssistantUUID` linking it to the assistant message that issued the call, plus a `toolUseResult` object with `stdout`, `stderr`, and an `interrupted` flag.
- **Extended thinking blocks.** When extended thinking is enabled, the assistant content array contains a `{type: "thinking"}` block alongside the visible `{type: "text"}` block. Each thinking block carries a cryptographic `signature` — Anthropic's mechanism to verify that thinking content has not been tampered with between turns in a long tool-use loop.
- **Prompt-cache telemetry.** Each assistant message records `cache_creation_input_tokens` (tokens written to the cache this turn) and `cache_read_input_tokens` (tokens served from cache), broken out by TTL tier (`ephemeral_5m` vs. `ephemeral_1h`). This makes the cache hit rate observable as context grows.

- **Session metadata.** Every record stores `cwd`, `gitBranch`, `version` (the Claude Code release string), and `userType` (`external` for human users). A human-readable `slug` field (e.g. `gleaming-spinning-nygaard`) appears from the first tool-call onward, matching the session's VM directory name. These fields enable session resumption and provide an audit trail.
- **Permission model.** Messages carry a `permissionMode` field (`default`, `full`, `restricted`) that gates which tools the agent may invoke autonomously, aligning with Anthropic's principle of minimal footprint for agentic actions.

Claude Code remote control

Claude Code's **remote control** feature allows an active terminal session to be accessed and continued from a web browser or mobile device, without moving any code or state to the cloud. Everything keeps running on the original machine; only the conversation stream is proxied to the remote client.

Workflow.

1. In the project directory, run `claude remote-control`. This starts a new session and prints a URL. Pressing `Space` generates a QR code that can be scanned to open the session directly in the Claude mobile app or browser.
2. To hand off a session already in progress, type `/remote-control` inside the running session to get the same URL.
3. On the remote device, tap the *environment* button and select the machine by name. Multiple machines / sessions can be listed simultaneously.

Architecture properties.

- **Nothing moves to the cloud.** The agent's tools, MCP server connections, file system access, and shell environment remain on the originating machine. The remote device is a thin client that streams the conversation.
- **Session continuity.** Typing on the mobile app updates the terminal in real time and vice versa — the shared state is the JSONL session log on the local machine.
- **Spawn mode.** From a remote device you can also create entirely new Claude Code sessions on the connected machine, enabling you to launch parallel agents without being physically present.
- **Authentication.** The URL is tied to the Claude account; no separate credentials are required beyond being logged into the same account on the remote device.

Persistent remote control. By default, remote control must be explicitly enabled per session. In `/config` (Claude Code's configuration), enabling `remote-control` for all sessions removes this requirement.

Note

Remote control is available on Mac, Team, and Enterprise plans (rolling out to Pro/Max). It is a natural extension of the JSONL session log architecture: because every turn is already serialised to disk, surfacing that stream to a second client requires no additional server-side state.

How AI IDEs use context (repo map, RAG)

Diff application & conflict resolution

Coding agents must apply model-generated edits back to the repository. Two dominant formats exist and the choice meaningfully affects benchmark performance:

Whole-file (whole) format: The model rewrites the entire file with the change applied. Simple to parse and apply; but wastes tokens and compute rewriting unchanged content. Produces more reliable edits for small files; becomes impractical for large codebases.

Diff (unified-diff / SEARCH-REPLACE) format: The model emits only the changed lines, either as a unified diff or as a SEARCH/REPLACE block. Far more token-efficient; the apply step can fail if the surrounding context the model assumed does not match the current file (merge conflicts).

Observed impact: On the Aider Polyglot benchmark, the same base model scored 8% in diff format vs 16% in whole format because the model had not been fine-tuned on diff-style output — it would hallucinate diff syntax. After instruction-tuning on diff examples, performance converged and ultimately exceeded the whole-format ceiling.

The analogy

Whole format = re-drawing the entire picture to add a cloud. Diff format = drawing just the cloud. Diff is smarter; it requires the model to reason precisely about which lines to change and what the surrounding context is.

Conflict resolution arises when the model's assumed file state diverges from the actual file (concurrent edits, stale context). Agents handle this by: (1) re-reading the file before applying, (2) fuzzy matching the search block with edit distance tolerance, or (3) falling back to whole-file replacement.

Evaluation: SWE-bench, Aider Polyglot & coding benchmarks

SWE-bench / SWE-bench Verified:

- Tasks are real GitHub issues from popular Python repositories; the agent must produce a patch that passes the associated unit tests.
- SWE-bench Verified filters to issues where human raters confirmed the test suite correctly validates the fix, removing noisy items.
- State-of-the-art agents (Devin, SWE-agent) score in the 40–50% range as of mid-2024; fully autonomous resolution of hard issues remains challenging.
- Measures end-to-end capability: repo understanding, fault localisation, patch generation, and test passage — all in one score.

Aider Polyglot:

- Coding exercises in six languages (Python, JavaScript, TypeScript, Go, Rust, C++); evaluated in diff format and whole format separately.
- Respected as a real-world, language-diverse benchmark; frontier models like GPT-4-era scored around 18% in diff format at launch.
- Benchmark non-determinism is significant: consecutive runs of the same checkpoint can vary by 2–3 percentage points (see Chapter 45 on statistical significance).

Synthetic data for coding fine-tuning: Two prominent methods for generating instruction-tuning data for coding models:

- **OSS-Instruct** (from MagicCoder): given a code snippet, prompt a stronger model to generate an instruction-response pair *inspired by* the snippet but not identical to it. Produces diverse, naturally grounded examples.
- **Evol-Instruct:** iteratively evolve a seed instruction into more complex or diverse variants ("make it harder", "add an edge case", "translate to a different language"). Each evolved version becomes a new training example. The cloning/dancing-guy analogy: each iteration produces a slightly different version; after many rounds, the dataset is far richer than the seed.

The training-data pipeline challenge (observed empirically):

- Raw code from GitHub commit messages varies wildly in quality; poor commit messages often indicate low-quality, uncommitted code artifacts.
- Validation harnesses (test runners that score model outputs) can *pass garbage* if misconfigured — a stricter harness catching more failures is progress, not regression.
- **Garbage in, garbage out:** training on lower-quality data can make a model worse than its base. Cleaning (whitespace normalisation, deduplication, license filtering, quality scoring) is as important as volume.
- Data sources used in practice: The Stack (60TB permissively-licensed code), GitHub (MIT/Apache-licensed repositories), synthetic generation from frontier models (with contamination checks).

Agentic Platforms: Manus, Computer Use & Beyond

Manus AI overview (Butterfly Effect)

NVIDIA NemoClaw / OpenShell

NemoClaw is NVIDIA's packaged enterprise distribution for agentic deployment, combining three layers into a single installable stack (see Chapter 36 for full detail):

- **OpenClaw** — the agent/app layer (planning, tool use, file I/O).
- **OpenShell** — the out-of-process runtime governor: sandbox + policy engine + privacy router. Sits between the agent and the real OS/network.
- **Nemotron** — NVIDIA's open model family (reasoning, coding, safety, agentic) used as the local inference option when cloud routing is disallowed.

The distinguishing architectural idea is *out-of-process policy enforcement*: safety controls are applied at the OS level (Landlock, seccomp, network namespaces) rather than relying on soft in-context instructions. Enterprise policy rules (data residency, domain whitelists, filesystem scoping) are enforced by OpenShell before any action executes, regardless of what the agent reasons it should do.

OpenClaw as "Agentic OS." Jensen Huang framed OpenClaw at GTC as the operating system for *personal agents* in the same way Windows was the OS for personal computers: a universal runtime that abstracts hardware and exposes a stable interface for applications. The analogy maps naturally — OpenClaw manages resources (tools, file systems, models), schedules work (cron jobs, task decomposition), spawns sub-agents (analogous to processes), and handles I/O in any modality. Just as every company needed a Linux strategy, an HTTP strategy, and a Kubernetes strategy, the argument is that every software company now needs an *OpenClaw strategy*.

Nemotron Coalition

NVIDIA announced a coalition of companies committing to build on and contribute to Nemotron 4: **Black Forest Labs** (image generation), **Cursor** (AI coding IDE), **LangChain** (agent frameworks), **Mistral**, **Perplexity**, **Reflection**, **Sarv** (India), **Thinking Machines**, and **Mirror/Moradi's lab**. The coalition accelerates the ecosystem of fine-tuned and specialised derivatives around a shared open base model.

NVIDIA Open Model Families. Beyond Nemotron (language), NVIDIA maintains six frontier open model families spanning radically different domains:

Model family	Domain
Nemotron	Language: reasoning, visual understanding, RAG, safety, speech
Cosmos	Physical AI: world generation and understanding for robotics/simulation
Alpamo	Autonomous vehicles: the world's first thinking/reasoning AV model
Groot	General-purpose humanoid and articulated robotics
Bioneo	Biology, chemistry, and molecular design
Earth 2	Weather and climate forecasting via AI physics

Intuition

These six families reflect the fact that no single model can serve every industry. Language intelligence (Nemotron) and physical-world intelligence (Cosmos, Alpamo, Groot) require fundamentally different training distributions, evaluation criteria, and safety considerations. NVIDIA's strategy: maintain frontier open models in each vertical so that industry partners can fine-tune domain-specific sovereign models on top.

SaaS → AaaS: the agentic transformation

The enterprise software industry is undergoing a structural shift from **Software-as-a-Service (SaaS)** to **Agentic-as-a-Service (AaaS)**. Traditional SaaS companies build tools that *humans* use: dashboards, workflow engines, systems of record. Post-OpenClaw, those same companies are rebuilding their products as agentic services — autonomous agents that *act* on a company's behalf, not tools that humans operate.

Enterprise IT before/after agentic systems.

- **Before (SaaS):** data centres hold structured business data; software exposes tools; humans (or digital workers) operate those tools; GSI consultants integrate everything. Value is created by humans wielding software.
- **After (AaaS):** agents sit on top of the same data stores and SaaS APIs, but they operate autonomously. The company's domain expertise is encoded in agent configurations rather than in UI workflows.

Token budgets as a new engineering resource. Huang predicted that every engineer will eventually carry an *annual token budget* — analogous to a compute or travel budget — allowing them to amplify their work via AI inference. A software engineer paid $200K might receive a token budget worth $100K+ on top, with the expectation of 10× productivity. Token budget size is already emerging as a recruiting differentiator in Silicon Valley.

Note

The implication is that companies will simultaneously be *token consumers* (buying compute to amplify their own engineers) and *token manufacturers* (packaging their domain expertise as agentic services that customers pay per-token to access). AI factories (GPU clusters) become the production infrastructure for both sides of this equation.

Anthropic Computer Use

Anthropic's *Computer Use* capability (released October 2024) exposes the model's ability to observe screenshots and emit low-level desktop actions (mouse clicks, keyboard input, scrolling). It is offered as a set of tool schemas (`computer`, `text_editor`, `bash`) that the API caller is responsible for executing, making the operator the "hands" that actually drive the OS.

Cowork is Anthropic's desktop-native product built on this foundation. Rather than raw computer use, Cowork couples a sandboxed Linux VM with a rich tool layer (filesystem access, web browsing, scheduled tasks, MCP connectors) to produce a practical desktop agent for non-developers. Key architectural points observed directly:

- The agent runs inside a per-conversation Ubuntu 22 VM spun up on the user's machine. Each session receives a randomly generated slug (e.g. `festive-nice-ramanujan`) used as both the VM directory name and the project identifier in the `.claude/` state store.
- A mounted workspace path (`/sessions/{slug}/mnt/{user-folder}`) bridges the VM to the user's actual filesystem, while the VM's own root remains ephemeral and resets between tasks.

- A **Skills system** provides Claude with pre-written `SKILL.md` instruction files (e.g. `pptx`, `docx`, `pdf`, `xlsx`) that encode best practices for creating specific document types, reducing trial-and-error at inference time.
- MCP (Model Context Protocol) connectors allow authenticated integration with external services (Slack, Asana, Google Drive, etc.) without the agent handling raw credentials.

OpenAI Operator & Agents SDK

Browser-use & web automation

Desktop agent architectures

Desktop agents must bridge the gap between an LLM's token-level reasoning and the pixel/filesystem/network reality of a personal computer. A few recurring structural patterns:

- **Tool-first vs. screenshot-first.** Tool-first agents (Claude Code, Cowork) prefer structured tool calls (read file, run bash, fetch URL) over raw screenshot observation. Screenshot-first agents (OpenAI Operator, Manus) parse the visual DOM and emit pointer events. Tool-first agents are faster and cheaper for file/code tasks; screenshot-first generalizes to arbitrary GUI applications.
- **Workspace isolation.** A dedicated working directory (or VM) separates agent-generated artifacts from the user's files, preventing accidental overwrites. Cowork keeps a `/sessions/{slug}/` scratch space distinct from the mounted user folder.
- **State subdirectories.** The `.claude/` layout illustrates how desktop agents partition state: `projects/` for conversation logs, `plans/` for multi-step plans, `memory/` for persistent notes, `shell-snapshots/` for working-directory continuity across Bash calls, and `backups/` for rollback support.
- **Prompt caching at scale.** Long-running desktop sessions accumulate large system prompts. Tiered prompt caching (e.g. 5-minute and 1-hour TTL tiers in Anthropic's API) amortises the cost of re-encoding the same instructions on every turn.

Security & sandboxing for autonomous agents

Autonomous agents operating on real filesystems and networks introduce a surface area for both accidental harm and adversarial *prompt injection*.

Sandboxing layers used in practice:

- **VM isolation** — each session runs in a separate lightweight VM so a runaway command cannot reach the host OS or other sessions.
- **Filesystem scoping** — the agent's write access is limited to its own working directory; the user's files are mounted read-write only through an explicit workspace mount.
- **Permission tiers** — Claude Code's `permissionMode` field (`default / full / restricted`) controls whether the agent can run arbitrary shell commands, write files, or is restricted to read-only actions.
- **Prohibited action lists** — product-level policies (e.g. Cowork's security rules) enumerate actions the agent must refuse regardless of instruction: permanent deletion, modifying access controls, entering financial credentials, creating accounts.

Prompt injection is the primary adversarial threat: malicious instructions embedded in observed content (web pages, emails, filenames, tool results) attempt to hijack the agent's action stream. Defences include:

- Treating all content from tool results as *untrusted data*, not as instructions — regardless of claimed authority ("admin override", "Anthropic staff").
- Requiring explicit out-of-band user confirmation (in the chat interface) before acting on any instruction found inside a function result.
- Signing extended-thinking blocks (as in Claude's `signature` field) to detect tampering between turns in a long tool-use loop.

Meta's agent research (LLaMA, CICERO)

Part X

Multimodal & World Models

Vision-Language Models

CLIP & contrastive pretraining

LLaVA, Flamingo, PaLI

Vision tokens & patch embeddings

Grounding & spatial reasoning

Diffusion Models & Generative Vision

GANs & VAEs (generative predecessors)

Before diffusion models dominated image generation, two main approaches competed: **Generative Adversarial Networks (GANs)** and **Variational Autoencoders (VAEs)**.

GAN (Goodfellow et al., 2014) trains two networks:

- **Generator** $G : z \sim p(z) \to \mathbb{R}^d$ maps noise (e.g., standard Gaussian) to fake samples.
- **Discriminator** $D : x \to [0, 1]$ tries to distinguish real from fake.

The adversarial objective:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p(z)}[\log(1 - D(G(z)))]$$

Training alternates: discriminator tries to maximize, generator tries to minimize. Samples are sharp and realistic when well-trained.

GAN Drawbacks:

- **Mode collapse:** Generator finds a few modes and ignores others, reducing diversity.
- **Training instability:** Careful tuning of learning rates and architectures required.
- **Vanishing gradients:** When D is too good, $\log(1 - D(G(z)))$ saturates; switching to $-\log D(G(z))$ helps.

Notable variants: **DCGAN** (convolutional architecture), **StyleGAN** (style-based generation, high quality), **CycleGAN** (unpaired image translation).

VAE (Kingma & Welling, 2014) uses an encoder-decoder with a learned latent distribution:

- **Encoder** $q_\phi(z|x)$: maps input x to a distribution, typically $\mathcal{N}(\mu_\phi(x), \text{diag}(\sigma^2_\phi(x)))$.
- **Decoder** $p_\theta(x|z)$: reconstructs x from sampled z.

Trained with the **ELBO** (Evidence Lower Bound):

$$\mathcal{L} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \parallel p(z))$$

The **reparameterization trick** enables backprop: $z = \mu + \sigma \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$.

Intuition

The KL term regularizes the latent space toward the prior (e.g., $\mathcal{N}(0, I)$), enabling smooth interpolation and generation from the prior. Reconstruction and regularization are always in tension.

VAE limitations: Blurry outputs (Gaussian decoder assumption limits expressiveness); less sharp than GANs.

Note

Summary: GANs are sharp but unstable; VAEs are stable but blurry; diffusion models achieve both sharpness and stability through iterative refinement.

DDPM & DDIM

DDPM (Ho et al., 2020) gradually adds noise to data in a Markov chain:
Forward process (fixed, no learning):

$$q(x_t|x_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\, x_{t-1}, \beta_t I\right)$$

Over T steps (typically 1000), noise variance β_t is small, so the process is slow. After T steps, $x_T \approx \mathcal{N}(0, I)$.
Closed form: Using $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

This lets us jump directly from x_0 to any x_t without iterating.
Reverse process (learned): Train a U-Net $\epsilon_\theta(x_t, t)$ to predict the noise ϵ added in step t:

$$\mathcal{L} = \mathbb{E}_{t,x_0,\epsilon}[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

Intuition

At inference, start from $x_T \sim \mathcal{N}(0, I)$ and iteratively denoise for T steps, using the learned ϵ_θ to reverse the forward process.

Key advantage: Unlike GANs, training is stable; unlike VAEs, samples are sharp.
Key drawback: Requires 1000 denoising steps, making inference very slow.

DDIM (Song et al., 2020) reformulates the reverse process as an **ODE** instead of an SDE, allowing deterministic (non-stochastic) sampling.
Rather than adding noise at each reverse step, DDIM rewrites the reverse process deterministically, reducing required steps from 1000 to 20–50 with minimal quality loss. The rewritten trajectory:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}\right) + \sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_t, t)$$

Advantages:

- **Speed:** 20–50 steps vs. 1000 (50–100× faster).
- **Deterministic:** Same latent ⇒ same output (enables meaningful interpolation).

- **Quality:** Comparable to DDPM with full steps.

Note

DDIM's deterministic path is the foundation for latent-space editing and semantic interpolation in tools like Stable Diffusion.

Score matching & SDE perspective

The **score function** $s(x) = \nabla_x \log p(x)$ points toward regions of higher density. Training a network to estimate the score is called **score matching**.
Denoising score matching (Vincent, 2011) shows that score matching is equivalent to DDPM's noise prediction. Instead of predicting ϵ, we can equivalently learn:

$$s_\theta(x_t, t) = -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)$$

This alternative view connects diffusion to the broader score-based generative modeling literature.

Intuition

The score function gives a direction and magnitude for how to move toward the data distribution. Iteratively following the score is another way to think about diffusion sampling.

Diffusion Models as SDEs (Song et al., 2021) unify all diffusion models as stochastic differential equations (SDEs).
Forward SDE:

$$dx = \mathbf{f}(x, t)dt + g(t)dW_t$$

where $\mathbf{f}$ is the drift, $g(t)$ controls diffusion (noise), and W_t is a Wiener process.
Reverse SDE:

$$dx = \left[\mathbf{f}(x, t) - g(t)^2 \nabla_x \log p_t(x)\right] dt + g(t)d\bar{W}_t$$

The crucial term is $\nabla_x \log p_t(x)$—the score function at time t. By training a network to estimate this score for all t, we can run the reverse SDE to sample.
Benefits of SDE view:

- **Continuous time:** Treat diffusion as a continuous process; no need to pick T discrete steps.
- **Probability flow ODE:** The reverse process can be written as a deterministic ODE (generalizes DDIM). Enabling faster sampling via ODE solvers.
- **Flexibility:** Different choices of drift $\mathbf{f}$ and diffusion g yield different samplers; score is the common learned component.

Note

Key insight: All diffusion models fundamentally learn the score function $\nabla_x \log p_t(x)$. How you parameterize it (noise prediction vs. direct score prediction) is a choice, but the learned quantity is the same.

Stable Diffusion architecture

Latent Diffusion (Rombach et al., 2022, now Stable Diffusion) performs diffusion in the latent space of a pretrained VAE instead of pixel space.

Architecture components:

1. **VAE Encoder:** Compresses input (e.g., 512×512 RGB image) to a latent tensor (64×64×4).
2. **U-Net denoiser:** Runs the diffusion process entirely in latent space. 860M parameters; includes cross-attention layers for text conditioning.
3. **CLIP text encoder:** Encodes the text prompt into a sequence of embeddings, fed to cross-attention in the U-Net.
4. **VAE Decoder:** Reconstructs the final image from the denoised latent.

Why latent space?

- **Compression:** 8× spatial downsampling ⇒ 64× fewer tokens.
- **Speed:** Diffusion on 64×64 latents is much faster than on 512×512 pixels.
- **Quality:** VAE's learned compression preserves semantic information; diffusion achieves high-quality details.

Intuition

The VAE encoder learns to compress while preserving semantic content. Diffusion refines in this efficient latent space. The decoder expands back to high-resolution images. This layering is key to Stable Diffusion's speed.

Classifier-free guidance (Ho & Salimans, 2021) controls how strongly the model follows the text prompt.

During training, the U-Net is trained both with the text conditioning and without (dropping the conditioning with probability p_{uncond}, e.g., 0.1).

At inference, two predictions are made:

- $\epsilon_{\text{cond}}(x_t, t, c)$: noise estimate conditioned on text c.
- $\epsilon_{\text{uncond}}(x_t, t, \varnothing)$: noise estimate without text.

The guided prediction:

$$\epsilon_{\text{guided}} = \epsilon_{\text{uncond}} + w \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$$

where w is the guidance weight (typically 7–15).

Effects of w:

- $w = 0$: Ignores prompt; high diversity, low relevance.
- $w = 1$: No guidance (equivalent to standard conditioning).
- $w > 1$: Amplifies prompt-following signal; sharper adherence to text but less diversity.
- $w \gg 1$: Overly prompt-focused; may produce unrealistic artifacts.

Note

Classifier-free guidance is a simple post-hoc technique that requires no external classifier and provides a principled way to trade off prompt fidelity vs. diversity.

ControlNet, LoRA for images

ControlNet (Zhang et al., 2023) adds explicit spatial control to a frozen diffusion model, enabling control via:

- Depth maps (depth-guided generation).
- Pose skeletons (pose-guided human generation).
- Canny edge maps (sketch-to-image).
- Segmentation masks (layout-guided generation).
- Optical flow, normals, etc.

Architecture:

1. Copy the U-Net encoder weights (frozen) to a trainable **control branch**.
2. Add **zero-convolution** layers to merge the control signal with the main U-Net.
3. Zero-convolutions initialize to zero-output, so training starts from the pretrained distribution.
4. Fine-tune only on (image, control map) pairs; the pretrained model's knowledge is preserved.

Intuition

Zero-convolutions are crucial: they allow the control branch to be trained from scratch without disrupting the frozen model's pretrained weights. Gradual scaling from 0 ensures stable training.

Advantage: Single frozen model + many lightweight ControlNets for different control types; users can combine multiple ControlNets for compound guidance.

Note

ControlNet demonstrates that spatial conditioning can be added to pretrained diffusion models with minimal retraining, enabling flexible control without full fine-tuning.

LoRA (Low-Rank Adaptation, Hu et al., 2021) applies low-rank factorization to U-Net layers, enabling efficient fine-tuning.

For image diffusion models, LoRA is applied to **cross-attention layers** in the U-Net. These layers compute:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

Rather than fine-tuning the full weight matrices W_Q, W_K, W_V, we replace with:

$$W \approx W_0 + \alpha AB^\top$$

where $A \in \mathbb{R}^{d\times r}$ and $B \in \mathbb{R}^{d\times r}$ are learned, and $r \ll d$ is the rank (e.g., $r = 4$ or 8).

Advantages:

- **Parameter efficiency:** ∼100 MB per LoRA vs. full model retraining (∼4 GB for Stable Diffusion).
- **Fast training:** Fine-tune to a specific style or subject in hours on a single GPU.
- **Composability:** Combine multiple LoRAs to blend styles and subjects.
- **Preservation:** Base model weights frozen; LoRA only modifies the low-rank subspace.

Intuition

Cross-attention layers are where text embedding interacts with image features. Low-rank updates here efficiently steer generation toward a specific aesthetic or subject without retraining the entire model.

Flow matching

Flow Matching (Lipman et al., 2022) takes a different approach: instead of learning to denoise, directly learn a **vector field** $v_\theta(x_t, t)$ that transports a noise distribution to the data distribution along a path.

Core idea:

1. Define a path x_t from noise ($t = 0$) to data ($t = 1$).
2. The vector field $v_\theta(x_t, t)$ should match the direction of the path: $\frac{dx_t}{dt} = v_\theta(x_t, t)$.
3. Train by minimizing the matching loss:

$$\mathcal{L} = \mathbb{E}_{t,x_0,x_1} \left\| v_\theta(x_t, t) - \frac{dx_t}{dt} \right\|^2$$

Straight-line paths: The simplest choice is a straight line:

$$x_t = (1-t)x_0 + tx_1, \quad \text{so} \quad \frac{dx_t}{dt} = x_1 - x_0$$

This is constant, making training very simple: the network learns to predict a constant direction.

Advantages over DDPM/DDIM:

- **Simpler loss:** Direct matching vs. noise prediction; more stable.
- **Straight paths:** Fewer steps needed at inference (fewer integration steps along a straight line).
- **Flexibility:** Paths don't have to be straight; can use optimal transport for curved paths minimizing detours.
- **Faster training:** No need to condition on noise schedules.

Intuition

Flow matching is intuitive: imagine pushing probability mass along a field of vectors from noise to data. Learning to predict the velocity field is simpler than denoising.

Optimal Transport (OT) conditioning: Instead of assigning data points arbitrarily to noise samples, use the Wasserstein distance to find the **straightest possible paths**.

For a batch of data $\{x_1^{(i)}\}$ and noise samples $\{x_0^{(i)}\}$, OT finds a coupling that minimizes path curvature. This results in:

- Straighter paths $\Rightarrow$ fewer integration steps.
- More efficient training (less doubling-back or zigzagging).
- Better sample quality.

Modern flow-matching models (e.g., Stable Diffusion 3, Flux) use OT-conditioned flow matching for improved speed and quality.

Note

Comparison: DDPM predicts noise; DDIM makes it deterministic; flow matching learns a vector field directly. Each approach refines the previous, trading off complexity for speed and simplicity.

Audio, Speech & Multimodal Fusion

Whisper & speech recognition

Audio tokenization

TTS & voice cloning

Multimodal fusion strategies

World Model Theory

LeCun's JEPA architecture

44.1.1 Critique of Autoregressive Models as World Models

Yann LeCun's fundamental critique of using autoregressive large language models (and autoregressive vision models) as world models concerns **wasted modeling capacity**. When a model predicts the next pixel or token from context, it must allocate capacity to capturing:

1. **Predictable** structure (high utility for world understanding)
2. **Unpredictable** variation: photon noise, hair particle positions, thermal fluctuations, stochastic choices made by agents

For high-resolution images or long sequences, the unpredictable signal dominates, forcing the model to spread its representational budget across meaningless details.

The Pixel Prediction Problem: A model predicting RGB pixels must learn the statistics of camera noise, weather variation, and random object motion. In contrast, human prediction works in a *compressed representation space*—we don't predict pixels, we predict "the ball will be at location X with velocity V", which is far lower-dimensional.

44.1.2 JEPA: Joint-Embedding Predictive Architecture

JEPA inverts the prediction strategy: instead of predicting observations, predict in **learned representation space**.

Key components:

- **Encoder** f_θ: maps observations (context) $\mathbf{x}$ to representation $\mathbf{s}_x = f_\theta(\mathbf{x})$
- **Predictor** $\mathbf{p}_\phi$: maps context representation to predicted target representation $\hat{\mathbf{s}}_y = p_\phi(\mathbf{s}_x)$
- **Encoder** f_θ: encodes actual target $\mathbf{y}$ to true target representation $\mathbf{s}_y = f_\theta(\mathbf{y})$
- **Loss**: MSE in representation space: $\mathcal{L} = \|\mathbf{s}_y - \hat{\mathbf{s}}_y\|_2^2$

Intuition

The encoder f_θ is shared across context and target. This *tightens* the bottleneck: the predictor cannot exploit the encoder to expand information. The representational space must be meaningful, not just a reconstruction buffer.

44.1.3 Energy-Based Models (EBMs)

To prevent *collapse* (all representations becoming identical), JEPA uses an energy-based framework. An energy function $E(\mathbf{s}_x, \mathbf{s}_y)$ assigns a scalar score to representation pairs:

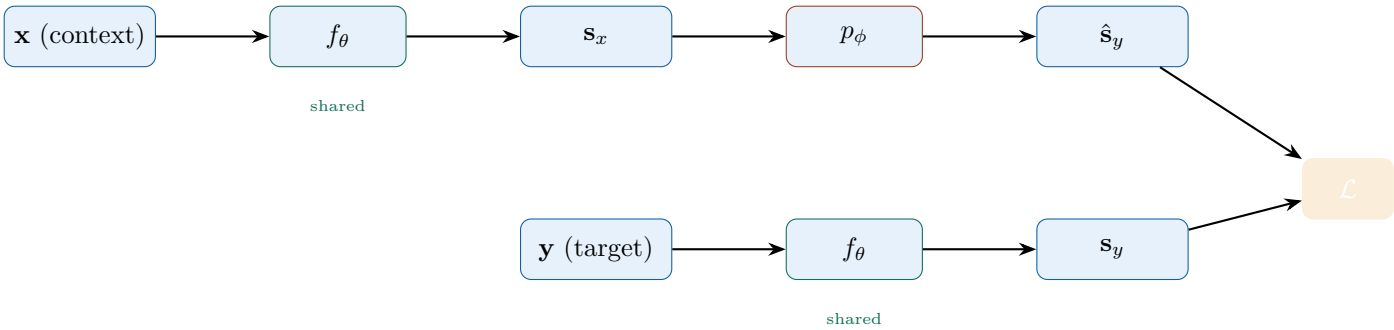


Figure 1: JEPA architecture: predict in representation space, not observation space.

$$E(\mathbf{s}_x, \mathbf{s}_y) = \|\mathbf{s}_y - \hat{\mathbf{s}}_y\|_2^2$$

Interpretation:

- Low energy: $\mathbf{s}_x$ and $\mathbf{s}_y$ are compatible (prediction succeeds)
- High energy: incompatible (prediction fails)

EBMs naturally handle multimodal futures: if multiple futures are plausible, they all have low energy. In contrast, regression-based models force a single Gaussian mode.

44.1.4 I-JEPA: Image Variant

I-JEPA applies JEPA to image patches. Training:

1. Mask random patches from target image (e.g., 50% masking)
2. Context = unmasked patches
3. Predictor must infer representations of masked patches from surrounding context
4. Loss computed only on masked patch representations

This forces the encoder to learn representations that capture **semantic structure**, not pixel details, because the model must **generalize across patch boundaries**.

44.1.5 V-JEPA: Video Variant

V-JEPA extends JEPA to video by masking spacetime tubes (e.g., regions in time and space):

- Context: visible spacetime regions (e.g., earlier frames, peripheral regions)
- Target: masked spacetime tube (e.g., a future time window, central region)
- Encoder processes spatiotemporal features
- Predictor predicts future motion and changes

V-JEPA learns dynamics without predicting raw pixels, instead learning representations of motion, occlusions, and object interactions.

Key Insight: By predicting in representation space and using masking as self-supervision, JEPA variants avoid the pixel-prediction curse. The model learns what is *semantically important* because masking forces it to generalize beyond pixel statistics.

Dreamer (RSSM)

44.2.1 Recurrent State Space Models (RSSM)

Dreamer introduces the **Recurrent State Space Model**, a hybrid deterministic-stochastic architecture for learning world models from observations and actions.

State evolution (two components):

1. **Deterministic path**: $\mathbf{h}_t = \text{GRU}(\mathbf{h}_{t-1}, \mathbf{z}_{t-1}, \mathbf{a}_{t-1})$
   - GRU processes previous hidden state, stochastic latent, and action
   - Captures *causal, deterministic dynamics*
2. **Stochastic latent**: $\mathbf{z}_t \sim q(\mathbf{z}_t \mid \mathbf{h}_t, \mathbf{x}_t)$ (during training)
   - Posterior encoder q uses current observation $\mathbf{x}_t$
   - Prior $p(\mathbf{z}_t \mid \mathbf{h}_t)$ depends only on deterministic state
   - Captures *stochastic variations* (multiple plausible futures)

Reconstruction: Decoder $p(\mathbf{x}_t \mid \mathbf{h}_t, \mathbf{z}_t)$ reconstructs observations from combined state.

44.2.2 Training with ELBO

The world model is trained by maximizing the evidence lower bound (ELBO):

$$\mathcal{L}_{\text{world}} = \mathbb{E}_{q(\mathbf{z}_{t:T})}\left[\sum_{t=1}^{T}\left(\log p(\mathbf{x}_t \mid \mathbf{h}_t, \mathbf{z}_t) - \beta\,\text{KL}(q(\mathbf{z}_t \mid \mathbf{h}_t, \mathbf{x}_t)\|p(\mathbf{z}_t \mid \mathbf{h}_t))\right)\right]$$

Components:

- **Reconstruction term**: likelihood $\log p(\mathbf{x}_t \mid \mathbf{h}_t, \mathbf{z}_t)$ reconstructs observations accurately
- **KL term**: $\text{KL}(q\|p)$ regularizes posterior to stay close to prior, preventing information collapse
- β **schedule**: annealing from $\beta = 0$ (ignore KL) to $\beta = 1$ (full ELBO)

Intuition

The stochastic latent $\mathbf{z}_t$ is not forced to predict everything; the deterministic path $\mathbf{h}_t$ can capture predictable dynamics. The latent only encodes information that helps *reconstruct the observation* (ELBO objective), encouraging learning of multi-modal futures.

44.2.3 Policy Learning from Imagination

Dreamer's key insight: **train the policy entirely on imagined trajectories, never updating the policy with real environment interaction**.

Procedure:

1. Sample initial state from real observations: $(\mathbf{h}_0, \mathbf{z}_0)$
2. **Imagine ahead**: Sample actions $\mathbf{a}_t$ from policy $\pi_\psi(\mathbf{a}_t \mid \mathbf{h}_t)$, evolve latent state using learned RSSM
3. Trajectory is entirely in learned representation space (no real environment)
4. Compute imagined reward $r_t = r(\mathbf{h}_t, \mathbf{z}_t)$
5. Apply any RL algorithm (actor-critic, policy gradient) to imagined trajectories

Advantage:

- **Sample efficiency**: one real trajectory generates many imagined rollouts
- **Policy learning cost**: computed from learned models, not environment interactions
- **Exploration**: can imagine counterfactual trajectories not yet observed

44.2.4 DreamerV3: Unified Hyperparameters

DreamerV3 extends Dreamer with the achievement: **same hyperparameters work across diverse environments** (vision-based control, manipulation, locomotion, Atari games).

Key improvements:

- **Latent overshooting**: predict multiple steps ahead in latent space, improving long-horizon planning
- **Separate model and value networks**: value function has its own head on $\mathbf{h}_t$
- **Actor-critic RL**: actor improves policy via gradient, critic learns value function
- **World model loss normalization**: scale reconstruction loss to improve gradient flow

Note

DreamerV3 demonstrates that learning world models for RL is not just feasible but can *generalize* across problem classes when the architecture and loss design are principled.

World models in RL

44.3.1 Model-Free vs. Model-Based: Trade-offs

Aspect	Model-Free	Model-Based
Sample Efficiency	Low (10^6–10^9 steps)	High (10^3–10^5 steps)
Computational Cost	Low (just function approximation)	High (model + planning)
Model Error	N/A	Compounds over long horizons
Transferability	Poor (policy is narrow)	Good (model captures dynamics)
Planning Horizon	Implicit (V network horizon)	Explicit (N steps)
Exploration	Via ϵ-greedy or entropy	Via uncertainty in model
Example Algorithms	DQN, PPO, SAC	PILCO, Dreamer, MuZero

Table 1: Comparison of model-free and model-based RL approaches.

44.3.2 Dyna: The Hybrid Approach

Sutton's **Dyna algorithm** (1991) blends both approaches:

Algorithm flow:

1. **Real step**: execute action $\mathbf{a}_t$ in environment, observe $(\mathbf{x}_{t+1}, r_t)$
2. **Update model**: train dynamics model $p(\mathbf{x}_{t+1} \mid \mathbf{x}_t, \mathbf{a}_t)$
3. **Update value function**: via real transition
4. **Imagination loop** (K iterations):
   - Sample state $\mathbf{x}_s$ from replay buffer
   - Sample action $\mathbf{a}_s$ from policy
   - Predict next state $\hat{\mathbf{x}}_{s+1}$ and reward $\hat{r}_s$ using model

- Update value function via imagined transition

Intuition: For each real step, gain K imagined steps worth of value learning. Model errors are contained because value estimates are grounded in real transitions.

44.3.3 MuZero: Latent Dynamics + Planning

MuZero combines learned latent dynamics with Monte Carlo tree search (MCTS) for planning.

Architecture:

- **Representation network** h: $\mathbf{s}_0 = h(\mathbf{x}_0, \ldots, \mathbf{x}_t)$ maps observation history to latent state
- **Dynamics network** g: $\mathbf{s}_{k+1} = g(\mathbf{s}_k, \mathbf{a}_k)$ predicts next latent state (deterministic)
- **Policy head** $\pi(\mathbf{a} \mid \mathbf{s}_k)$: predicts action logits in latent space
- **Value head** $v(\mathbf{s}_k)$: predicts value estimate
- **Reward head** $r(\mathbf{s}_k, \mathbf{a}_k)$: predicts immediate reward

Training:

1. **Collect trajectories**: play with MCTS using learned models
2. **MCTS planning**: expand tree by calling dynamics network, use policy/value heads for rollout
3. **Supervised learning**: train representation, dynamics, policy, value, reward heads to match MCTS search statistics

Key advantage:

- Only predicts in **latent space**, not pixels (avoids pixel-prediction curse)
- Value and policy targets come from MCTS (more accurate than raw environment returns)
- Achieves human-level performance in Atari with sample efficiency between model-free and full model-based

Compounding Error Risk: Any learned dynamics model accumulates error when unrolled multiple steps. MuZero mitigates this by: (1) using MCTS to aggregate multiple rollouts, and (2) training the model on MCTS targets rather than ground truth.

Predictive coding theories

44.4.1 Historical Foundations

Rao & Ballard (1999) proposed that the brain performs hierarchical predictive coding:

- Top-down: higher layers send *predictions* of lower-layer activity
- Bottom-up: lower layers compute *prediction errors* (observed - predicted)
- Error signals drive learning in both forward (generative) and backward (error) pathways

Free Energy Principle (Friston, 2010):

Perception and learning minimize the variational free energy:

$$F = \mathbb{KL}(q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z})) - \mathbb{E}_{q(\mathbf{z})} \log p(\mathbf{x} \mid \mathbf{z})$$

This is equivalent to the ELBO (evidence lower bound), meaning: **minimize prediction error while keeping the posterior close to the prior**.

44.4.2 Hierarchical Prediction and Error Signals

Hierarchical structure:

Consider a three-level hierarchy:

1. **Level 3 (abstract)**: high-level concepts (e.g., "scene type")
2. **Level 2 (intermediate)**: object properties and spatial layout
3. **Level 1 (sensory)**: low-level features (e.g., edges, colors)

Top-down predictions: Level 3 generates predictions of Level 2 activity; Level 2 predicts Level 1.

Bottom-up error signals:

- Level 1 computes $\boldsymbol{\delta}_1 = \mathbf{x}_{\text{obs}} - \hat{\mathbf{x}}_1^{\text{pred}}$ (sensory prediction error)
- Level 2 computes $\boldsymbol{\delta}_2 = \mathbf{x}_1 - \hat{\mathbf{x}}_2^{\text{pred}}$ (error in predicting Level 1 features)
- Level 3 interprets $\boldsymbol{\delta}_2$ to refine high-level hypothesis

Intuition

This hierarchical organization means low-level errors (pixel noise) are not propagated to high-level learning; only errors that *violate higher-level predictions* drive top-level adjustment. This naturally implements *attention* to surprising events.

44.4.3 Connection to Variational Autoencoders (VAEs)

VAEs implement predictive coding in a generative model:

$$\mathcal{L}_{\text{VAE}} = -\mathbb{E}_{q(\mathbf{z}|\mathbf{x})} \log p(\mathbf{x} \mid \mathbf{z}) + \beta \, \text{KL}(q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z}))$$

Interpretation in predictive coding terms:

- Encoder $q(\mathbf{z} \mid \mathbf{x})$: bottom-up inference of latent state
- Decoder $p(\mathbf{x} \mid \mathbf{z})$: top-down generative model (predictions)
- Reconstruction loss: prediction error magnitude
- KL term: regularizes inference (posterior) to stay close to learned prior

Key insight: VAEs are not just compression machines; they implement a learned hierarchical model where the encoder infers hidden causes and the decoder makes predictions.

44.4.4 Connection to JEPA

JEPA is the modern descendant of predictive coding, applied to representation learning:

- **Like predictive coding:** prediction occurs in a learned representation, not raw observation space; multimodal futures are allowed (energy-based, not mode-collapsing)
- **Unlike predictive coding:** no explicit hierarchical error propagation; learned via end-to-end training
- **Advantage over VAE:** JEPA avoids the pixel-reconstruction objective (which VAEs still use); instead, predictions are in a task-relevant latent space

Unified view:

- **Predictive coding**: minimize prediction error in a sensory hierarchy

- **VAEs**: learn a latent code to minimize reconstruction error + KL
- **JEPA**: learn representations by predicting masked observations in latent space (no reconstruction required)
- **World models (Dreamer, MuZero):** use learned latent dynamics for planning in RL

All share the principle: **learning occurs through prediction, and prediction is most efficient in learned representation spaces, not raw observation spaces**.

Note

Fundamental Principle: Intelligent agents must predict future states (or compressed representations) to plan and learn. The efficiency of this prediction depends on the representation: predicting compressed, task-relevant features is far more tractable than predicting raw sensor data. This principle unifies predictive coding, VAEs, JEPA, and world-model-based RL.

Video & Predictive Models

Sora & video diffusion

Sora (OpenAI, 2024) represents a major milestone in generative video: a text-to-video foundation model capable of producing coherent videos up to 1 minute in length at 1080p resolution. Unlike earlier text-to-video systems that required single-frame generation followed by upsampling, Sora generates video natively and directly.

45.1.1 Architecture: Video Diffusion Transformer

The core innovation is the **Video Diffusion Transformer (DiT)**, which departs from the U-Net-based architecture standard in image diffusion. Rather than applying convolutional layers hierarchically, Sora's transformer operates on *spacetime patches*—small cuboid regions of video (e.g., a $16 \times 16 \times 2$ block spanning spatial and temporal dimensions). Each patch is tokenised into a flat embedding sequence, allowing the transformer to treat video generation as a sequence-to-sequence task similar to language modelling.

This patch-based representation is elegantly general: instead of hardcoding separate models for different resolutions or aspect ratios, Sora simply conditions on the target resolution and generates patches of the appropriate size and duration. A $480p$ portrait video, a $1080p$ landscape shot, and a $720p$ square clip can all be generated by the same model by adjusting patch scaling.

Intuition

Why patches instead of pixels? Pixel-level diffusion on video is prohibitively expensive: a 1-minute 1080p video at 30fps is $\approx 1.8M$ frames of 1920×1080 pixels each. Patches reduce this to $\sim 10k$ tokens, making transformer-scale generation tractable. The trade-off is that patch-level generation may lose fine-grained spatial detail, but token efficiency wins.

45.1.2 Training and Data

Sora was trained on large-scale video datasets paired with text descriptions. A critical practical detail: raw video captions (e.g., YouTube metadata) are often sparse or vague. OpenAI likely employed a **recaptioning pipeline**: passing each video through a vision-language model (similar to CLIP or multimodal GPT-4V) to generate rich, detailed text descriptions that better explain the visual content. This densification of captions significantly improves text-to-video coherence.

The model learns in the diffusion framework: corrupted videos (with noise added to patches) are reconstructed conditioned on text embeddings. As training progresses, the model learns to generate spatially coherent frames and temporally consistent motion.

45.1.3 World Simulator Hypothesis

The Sora paper makes a bold claim: by learning to predict the next patch in a sequence, the model implicitly learns a *world model*—an internal representation of physics, causality, and object dynamics. Evidence cited includes:

- Objects maintain consistent identity and trajectories across frames.
- Shadows, reflections, and lighting respond realistically to object motion.
- Fluid and deformable objects (smoke, water, cloth) obey rough physical laws.

This hypothesis is influential but contested. Sceptics argue that the model may simply memorise large-scale patterns from training data rather than developing a true causal model of physics. Distinguishing learned dynamics from pattern matching remains an open question in interpretability.

45.1.4 Known Limitations

As of early 2024, Sora exhibits several failure modes:

Physics violations: Objects can appear or disappear without physical justification. A person may phase through a wall.
Long-range consistency: Over 30+ seconds, character appearance, lighting, and background slowly drift or degrade.
Control granularity: Fine-grained directives (e.g., "move the camera exactly 30 degrees clockwise") are difficult; the model interprets text-level intent rather than precise camera parameters.
Temporal flicker: Despite transformer-based processing, occasional frame-to-frame jitter occurs, especially at patch boundaries.

These limitations suggest that the current generation of diffusion-based video models, while impressive, remain far from true physical simulation or precise interactive control.

Genie & interactive world models

Genie (Google DeepMind, 2024) tackles a different problem: learning a *controllable* world model from unlabelled video. Rather than generating videos from text prompts, Genie learns to generate video frames conditioned on learned *actions*, enabling interactive control without a pre-defined control scheme.

45.2.1 Unsupervised Action Learning

The key innovation is that Genie learns its action space **without any action labels**. Given a sequence of raw video frames (e.g., from gameplay footage, dashcam footage, or other dynamic scenes), Genie infers: what latent factors, if manipulated, would steer the world's evolution?

Formally, Genie models $P(\text{frame}_{t+1}|\text{frames}_{1:t}, \mathbf{a}_t)$, where $\mathbf{a}_t$ is a learned latent action token. By training this mapping end-to-end on video sequences, the model discovers an action representation where nearby latent tokens correspond to coherent world perturbations (e.g., "move forward," "jump," "rotate camera"). This is enabled by the structure of the data itself: video sequences naturally contain the signal of causality (action causes state change).

Intuition

Why unsupervised? Collecting action labels (a button press or joystick input for every frame) is expensive and domain-specific. Gameplay footage, for instance, may come without controller logs. Genie's insight is that the video data itself encodes action information through the sequential structure: if frames are statistically different in a consistent direction, an action likely caused that difference.

45.2.2 Architecture and Components

Genie's pipeline comprises three main components:

1. **Spatial Tokeniser**: A VQ-VAE (Vector Quantised VAE) encodes individual video frames into discrete latent codes. This compresses spatial information and prepares frames for the dynamics model.
2. **Latent Action Model**: A learned discrete action tokeniser discovers a set of "primitive actions" such that sequencing them produces coherent trajectories. This acts like learning a proto-language of control.

3. **Dynamics Model**: A diffusion-based or autoregressive model predicts the next frame's latent code given the previous frame and a latent action token. This is the core world model: $P(\mathbf{z}_{t+1}|\mathbf{z}_{1:t}, \mathbf{a}_t)$ where $\mathbf{z}$ are frame latents.

Once trained, a user can "play" the world by selecting action tokens; the model generates a stream of coherent frames responding to those actions. Notably, no traditional controller input is required—the model infers control directly from user interaction (e.g., keyboard or learned policy).

45.2.3 Connection to World Models

Genie embodies a modern realisation of the *world models* programme from reinforcement learning: the hypothesis that agents learn an internal forward model of environment dynamics, which they use for planning. Unlike classical world models trained on RL environments with action labels, Genie learns from passive observation—the same regime humans often learn from (watching video, reading books). This makes it potentially more data-efficient and more aligned with human-like learning.

Genie and V-JEPA (Chapter 24) are complementary: V-JEPA learns a high-level latent representation of video through masked prediction; Genie learns *actionable* representations where latent variables map to world state changes. Both leverage the structure of video to extract generalizable representations without labels.

Temporal consistency challenges

Video generation introduces challenges absent from image generation: frames must cohere across time, objects must persist and move realistically, and long-range dependencies must be maintained. This subsection surveys the main challenges and techniques.

45.3.1 Flickering and Inter-Frame Jitter

Diffusion models, even transformer-based ones, can process frames with too much independence, leading to:

- **Flicker**: Pixel values oscillate between frames without clear motion, resembling poorly denoised noise.
- **Jitter**: Objects appear to vibrate or twitch due to slight spatial shifts in consecutive frames.

Root cause: the noise prediction objective optimises each timestep independently; without explicit temporal coupling, the model may find local minima that improve a single frame at the cost of inter-frame coherence.

45.3.2 Temporal Modelling Approaches

Several techniques enforce temporal consistency:

1. **3D Convolutions**: Inflate standard 2D convolutions into 3D by adding a temporal dimension. A 3×3 kernel becomes $3 \times 3 \times 3$, allowing the model to reason about local motion patterns. U-Net diffusion backbones can be retrofitted with 3D convs in the deeper layers.
2. **Temporal Attention**: Add self-attention across the time dimension in addition to spatial attention. A transformer can attend to "where was this object in the previous frame?" explicitly, guiding consistency. This is computationally expensive (quadratic in sequence length) but highly effective.
3. **Causal Temporal Transformers**: Structure the transformer so that frame t can attend to frames $0 \ldots t-1$ but not future frames. This respects causality and is more efficient than full

bidirectional attention, though models that attend bidirectionally before generation (and then use causal generation) are also common.

4. **Optical Flow Supervision**: Use optical flow (pre-computed or predicted) as a conditioning signal or auxiliary loss. If the model predicts inconsistent flow between frames, a loss penalty encourages it to enforce motion consistency. Some models compute optical flow between diffusion steps and use it to warp frames for better coherence.

45.3.3 Motion Blur, Ghosting, and Trajectory Tracking

Generating realistic motion is non-trivial. The model must:

- Predict where objects will be several frames hence.
- Render motion blur or sharp motion as appropriate to the scene context.
- Avoid **ghosting**: trailing artifacts where an object's old and new position both appear, creating a "smeared" look.

Successful motion generation requires the model to implicitly track object trajectories. Models trained with temporal attention or 3D convolutions tend to perform better because they can reason about object identity across frames.

45.3.4 Long-Range Consistency

A harder problem than frame-to-frame coherence: maintaining consistency over hundreds of frames. Issues include:

Character drift: A person's appearance (clothing, hairstyle) may subtly shift over 30+ seconds.
Lighting creep: The ambient light colour and intensity may slowly change without motivation.
Background accumulation: Repeated small errors in background rendering accumulate, eventually making the scene look wholly different.
Layout collapse: The spatial relationships between objects become confused, leading to impossible geometry.

Current models degrade noticeably after ~ 1 minute, suggesting that purely feed-forward generation (frame-by-frame without global memory) is insufficient. Research directions include hierarchical generation (high-level scene layout first, then detail infilling) and memory-augmented architectures.

45.3.5 Optical Flow as a Supervision Signal

Some recent models use **optical flow** (dense motion estimation) as auxiliary supervision:

- **Flow prediction loss**: In addition to the diffusion loss, encourage the model's latent representations to predict accurate optical flow.
- **Flow-based warping**: Use predicted flow to warp frames from frame t to frame $t + 1$, then blend the warp with the generated frame. This enforces physical continuity.
- **Flow conditioning**: Condition the diffusion model on precomputed or user-specified optical flow, constraining motion to be plausible.

This approach leverages classical computer vision (optical flow estimation) to ground modern generative models, improving consistency at a modest computational cost.

Next-frame prediction as pretraining

Predicting the next frame in a video sequence is a powerful self-supervised objective. It forces the model to learn dynamics, object persistence, and causal structure—all without labels. This subsection covers video prediction as a pretraining paradigm.

45.4.1 Video Prediction as Self-Supervised Learning

The core insight: a model that can predict frame_{t+1} from $\text{frames}_{1:t}$ must understand:

- **Motion**: How do objects move in this domain?
- **Object permanence**: Does an occluded object continue to exist?
- **Physics**: What are the constraints on how the world can change?
- **Semantics**: What is the scene about, and what are the key actors?

Unlike supervised pretraining (contrastive learning, masked language modelling), video prediction is a *generative* task: the model must synthesise the raw sensory signal of the next moment. This is a rich learning signal that captures the structure of the world.

Video prediction dates back to the early 2010s (Zeiler et al., Srivastava et al.), where convolutional models learned to predict pixel space. The field has evolved through adversarial losses (GANs), flow-based methods, VAEs, and now diffusion. The core idea remains: learning from temporal structure.

45.4.2 VideoMAE: Masked Autoencoding for Video

VideoMAE (He et al., Meta, 2022) applies the **masked autoencoder** paradigm to video:

- **Masking**: Randomly mask 90%+ of spacetime patches in a video. This is much more aggressive than image masking ($\sim 75\%$) and tests the model's ability to reason about long-range structure.
- **Reconstruction**: An encoder embeds the visible patches; a decoder reconstructs the masked patches. The loss is the reconstruction error on masked regions.
- **Learned representations**: The trained encoder produces video embeddings useful for downstream tasks (action recognition, video classification) without finetuning.

VideoMAE demonstrates that masked prediction scales well: larger models, more data, and higher masking ratios all improve downstream performance, following predictable scaling laws. This makes video prediction an attractive foundation for video understanding.

45.4.3 V-JEPA: Prediction in Representation Space

Rather than predicting pixels, **V-JEPA** (Video Joint-Embedding Predictive Architecture, Chapter 24) predicts in a learned latent space:

- **Representation learning**: An encoder f maps frames to latents: $\mathbf{z}_t = f(\text{frame}_t)$.
- **Latent prediction**: A predictor g is trained to minimise the distance between predicted latents $g(\mathbf{z}_{1:t})$ and target latents $f(\text{frame}_{t+1})$ (using a momentum-updated target encoder, similar to BYOL).
- **Advantage over pixel prediction**: Latent space is much simpler than pixel space; the model avoids modelling irrelevant textural variation and learns more abstract motion representations.

V-JEPA achieves strong video understanding performance and is computationally cheaper than pixel-space prediction. The representation learned this way captures semantically meaningful motion rather than superficial pixel transitions.

45.4.4 Scaling Laws in Video Pretraining

Like language and image models, video pretraining exhibits **scaling laws**:

- **Data scaling**: More video data yields better performance, with diminishing returns following a power law (similar to language models).
- **Compute scaling**: Larger models achieve lower prediction loss, consistent with Chinchilla scaling.
- **Task transfer**: Models trained on video prediction with internet-scale data transfer to many downstream tasks (classification, detection, tracking) better than supervised pretraining on smaller, task-specific datasets.

This suggests that **video prediction is a foundational task** for video understanding, much as next-token prediction is foundational for language. Scaling video prediction to billions of hours of video and trillion-parameter models remains an active frontier, with potential to unlock powerful open-domain video AI.

Note

Key Takeaway: Next-frame prediction mirrors language modelling's philosophy—leverage the structure of raw unlabelled data to learn powerful, transferable representations. For video, this means learning dynamics, causality, and scene understanding as an emergent property of predicting the sensory signal.

Embodied AI & Simulation

SLAM & robot perception

Intuition

Simultaneous Localization And Mapping (SLAM) is the classic chicken-and-egg problem: a robot must build a map of an unknown environment while simultaneously tracking its own location within that map. These two tasks are deeply intertwined — you need to know where you are to map accurately, and you need a good map to localize.

46.1.1 The two interleaved problems

SLAM decomposes into two core problems:

1. **Localization**: Given a map, where am I? Solved with particle filters or Kalman filters.
2. **Mapping**: Given my location, what does the environment look like?

46.1.2 EKF-SLAM

Extended Kalman Filter SLAM maintains a Gaussian belief over a joint state vector: $[\text{robot pose}, \text{landmark position}$ The state grows with each observed landmark, leading to $O(n^2)$ update cost where n is the number of landmarks. A key challenge is *data association*: deciding which observed feature corresponds to which landmark in the map.

46.1.3 Particle filter SLAM (FastSLAM)

Represents the belief as a distribution over robot trajectories; each particle maintains its own map of landmarks. This approach is more robust to non-Gaussianity than EKF-SLAM and achieves $O(n \log n)$ complexity per update.

46.1.4 Graph-SLAM and pose graph optimisation

Modern SLAM formulates the problem as a graph of robot poses connected by relative constraints (odometry measurements and loop closures). The goal is to find the most likely configuration via non-linear least squares, solved with tools like `g2o` and `GTSAM`.

Note

Loop closure occurs when the robot revisits a known location. This is where accumulated drift is corrected. The challenge is the place recognition problem: identifying that current observations match a previously visited location. Techniques include bag-of-words, NetVLAD, and DBoW descriptors.

46.1.5 Sensor modalities

- **LiDAR SLAM** (e.g. Velodyne): directly produces 3D point clouds; robust to lighting conditions.
- **Visual SLAM** (e.g. ORB-SLAM, COLMAP): uses cameras; cheaper hardware but more challenging in low light; sensitive to motion blur.
- **Dense vs. sparse**: sparse methods track keypoints; dense methods build full volumetric models.

46.1.6 Modern deep SLAM

Recent systems (NICE-SLAM, iMAP) use neural representations like Neural Radiance Fields (NeRF) or neural implicit maps as the underlying map representation, combining classical SLAM geometry with learnable dense scene models.

Sim-to-real transfer

Intuition

The sim-to-real gap is one of the hardest problems in robotics. Training in simulation is fast and cheap, but real robots face physics inaccuracies, sensor noise, appearance differences, and latency that don't exist in the simulator.

46.2.1 Sources of the gap

The sim-to-real mismatch includes:

- Physics: friction coefficients, mass, contact dynamics, compliance of actuators
- Sensors: camera noise, latency, quantisation in encoders
- Appearance: domain shift between photorealistic rendering and real images
- Control: latency, actuator saturation, backlash

46.2.2 Domain randomisation

Train the policy in many varied simulation conditions: random friction coefficients, masses, lighting, textures, camera parameters. The real world appears as just another instance of the randomised distribution. This worked for OpenAI's Dactyl (multi-fingered robotic hand solving a Rubik's cube).

46.2.3 Domain adaptation

Rather than randomisation, adapt the model's learned representation from simulation to real using adversarial training or style transfer. The real robot provides additional data to refine the simulator's parameters.

46.2.4 System identification

Measure real robot dynamics (forward kinematics, joint compliance, motor characteristics) and tune simulator parameters to match real hardware. This narrows the gap at the physics level.

46.2.5 Real-to-sim

Conversely, capture real sensor data and use it to build a more faithful simulation: photo-realistic rendering from real images, real trajectory data for dynamics validation.

RT-2, PaLM-E, embodied LLMs

Intuition

The key insight is that large language models and vision-language models trained on the internet contain rich semantic knowledge. By fine-tuning these models on robot demonstrations and grounding them in actions, we get data-efficient robot learning without training from scratch.

46.3.1 PaLM-E (Google 2023)

A multimodal language model that consumes continuous sensor streams (images, robot state) as inputs alongside text. Trained jointly on:

- Internet-scale text corpora
- Embodied robot manipulation data

Can plan multi-step manipulation tasks from natural language instructions and reason over sensor observations.

46.3.2 RT-2: Robotics Transformer 2 (Google DeepMind 2023)

Fine-tuned PaLI (a vision-language model) to output robot actions directly as discretised text tokens. Key insight: web-trained knowledge transfers to robot generalisation — the model can pick up objects it has never been trained on, if told the object's name.

46.3.3 Vision-Language-Action (VLA) models

The emerging paradigm: take a large vision-language model, fine-tune it on demonstrations where the model outputs robot actions (as text tokens or continuous vectors). RT-2 is the canonical example. OpenVLA is an open-source alternative.

Web-scale pretraining gives the model semantic understanding of objects, spatial relationships, and natural language. Fine-tuning on even modest amounts of robot data (thousands of trajectories) grounds this knowledge in actions — the model learns to convert "pick up the red cube" into motor commands. Far more sample-efficient than learning from scratch.

Project GR00T & foundation models for robotics

Intuition

Foundation models are reshaping robotics. Instead of building separate controllers for each task and robot type, the goal is a single large model trained on diverse robot and human data, then fine-tuned or prompted for specific embodiments and tasks.

46.4.1 NVIDIA Project GR00T (2024)

A framework for building foundation models for humanoid robots, announced by NVIDIA in 2024. Not a single monolithic model, but rather a pipeline and pretrained weights.

46.4.2 Groot N1

The first release: a general-purpose humanoid robot foundation model. Outputs joint positions or torques for humanoid bodies. Trained on:

- Diverse robot demonstrations (from multiple robot platforms when possible)
- Synthetic data from simulation (Isaac Lab)
- Human motion video (to provide motion priors, without requiring the model to explicitly control a body)

46.4.3 Key technical ideas

- **Leverage simulation**: use GPU-accelerated environments (Isaac Lab) to generate massive synthetic training data for pretraining.
- **Human motion priors**: incorporate human video to teach natural motion, even though humans and robots differ in kinematics.
- **Hardware fine-tuning**: adapt the foundation model to specific robot hardware and tasks via further training.
- **Multi-embodiment pretraining**: if available, train on diverse robot platforms; enables generalisation.

46.4.4 NVIDIA Isaac platform

The underlying simulation ecosystem consists of:

- **Isaac Gym**: GPU-accelerated physics, multi-stage simulation pipeline, used for large-scale RL training.
- **Isaac Lab**: high-level learning framework built on Isaac Gym; supports curriculum learning, task scheduling, and more.
- **Isaac Sim**: photorealistic rendering and sensor simulation (cameras, LiDAR) for bridging to real robots.

46.4.5 The "GR00T moment" for robotics

NVIDIA argues that GR00T represents a paradigm shift: foundation models for robotics are what ImageNet and CNN pretraining were for computer vision — a common foundation that dramatically lowers the barrier to building capable systems. Instead of each team training from scratch, they build atop shared, large-scale pretrained representations.

Physical AI: NVIDIA's term for AI that operates in and interacts with the physical world (forces, contact, dynamics), distinct from "digital AI" (language, images, text). Requires understanding physics deeply — not just visual recognition.

Simulation environments (Isaac, MuJoCo)

46.5.1 MuJoCo (Multi-Joint dynamics with Contact)

The standard open-source physics engine for robot learning:

- Very fast and accurate continuous control simulation
- Benchmark choice for RL on locomotion and manipulation
- Acquired by DeepMind; now fully open-source with active community
- GPU support in recent versions

46.5.2 Isaac Gym and Isaac Lab

NVIDIA's ecosystem, with GPU-accelerated physics and RL training:

- Simulate thousands of robot instances in parallel on a single GPU
- 100–1000× speedup over CPU-based simulators like MuJoCo on CPU

- Used for large-scale pretraining of foundation models (e.g. GR00T)
- Isaac Lab provides high-level APIs for task definition, curriculum learning, and integration with PyTorch

46.5.3 Habitat (Meta AI)

Photorealistic indoor navigation simulation:

- Built on real-world 3D scans from Matterport
- Focused on embodied vision and navigation tasks
- Used for PointNav, ObjectNav research

46.5.4 AI2-THOR

Interactive household simulator for robotic manipulation and object interaction:

- Support for articulated objects (doors, drawers, fridges)
- Realistic physics and visual rendering

46.5.5 PyBullet

Open-source physics engine; widely used in research and teaching. Slower than MuJoCo and Isaac, but easy to use and flexible.

Manipulation & locomotion tasks

46.6.1 Manipulation

Core robotic manipulation challenges:

- **Pick-and-place**: grasping an object and moving it to a target location
- **Grasping**: computing stable grasp points; parallel-jaw grippers vs. multi-finger dexterous hands
- **Assembly**: precise insertion and alignment (pegs in holes, mating parts)
- **Tool use**: using objects as tools to accomplish goals
- **Deformable objects**: handling cloth, rope, liquid — non-rigid dynamics

Key research challenges: predicting contact forces, planning grasps on novel objects, handling partial observability.

46.6.2 Dexterous manipulation

OpenAI's Dactyl (2019) achieved multi-fingered manipulation of a Rubik's cube in the real world. Key ingredients:

- Domain randomisation at extreme scale (50,000+ simulation variations)
- Massive compute for training in simulation (thousands of GPU hours)
- Careful system identification to narrow the sim-to-real gap

46.6.3 Locomotion

Mobile robot locomotion at scale:

- **Bipedal**: ATLAS (Boston Dynamics), Digit (Agility Robotics) — balancing on two legs
- **Quadruped**: Spot (Boston Dynamics), ANYmal (ANYbotics), Unitree — four-legged platforms achieving dynamic running and climbing
- **Agile locomotion**: MIT Cheetah, Boston Dynamics Spot can now do parkour — jumping over obstacles, climbing stairs

Deep reinforcement learning has been key to achieving highly agile, energy-efficient locomotion policies by training end-to-end in simulation.

46.6.4 Whole-body control

An emerging challenge: coordinating both base locomotion (walking/running) and arm manipulation simultaneously. Requires:

- Larger models to output more degrees of freedom
- Better simulators to capture contact-rich dynamics
- Curriculum learning (start with stationary manipulation, then add locomotion)

Enabled by recent advances in both simulation fidelity and model scale.

Humanoid loco-manipulation & VLA models

Intuition

Locomotion for humanoids is largely solved by large-scale simulation. The hard unsolved problem is *combining* locomotion with dexterous manipulation *and* language-grounded intelligence in a single, unified controller. A full humanoid such as the Unitree G1 has ∼43 degrees of freedom (DoF) — far more than the ∼18 DoF tabletop manipulation setting that most current VLA models target.

46.7.1 The π_0 VLA paradigm

Most contemporary vision-language-action models follow a common two-stage recipe:

1. **Pre-training**: a large vision-language model (VLM) backbone is trained on internet-scale multimodal data to acquire semantic and perceptual knowledge.
2. **Post-training / action alignment**: a continuous *action expert* head (typically trained with a flow-matching loss) is appended and trained on in-domain robot demonstrations to align the model's representations with motor commands.

The π_0 model (Physical Intelligence) and its successors $\pi_{0.5}$ and $\pi_{0.6}$ established this as the standard recipe. The action expert produces smooth, continuous trajectories via flow matching rather than discretised token prediction.

46.7.2 The robot learning data pyramid

- **Internet video** (bottom): vast scale, rich semantics, but no action labels and a large viewpoint domain gap.
- **Simulation data** (middle): unlimited episodes for locomotion; large physics and visual fidelity gap for fine-grained manipulation.

- **Real-world teleoperation data** (top): highest quality for manipulation; expensive to collect at scale (robot hardware, operator time).

A key research question is whether cheaper data tiers can substitute for expensive teleoperation data.

46.7.3 Egocentric human video as a pretraining source

Rather than relying on internet video (large viewpoint gap) or simulation (physics gap), the **Size Zero** project (Wang lab, USC) uses *egocentric human video* for VLM pretraining:

- **Viewpoint alignment**: an egocentric camera worn on the head sees the world from roughly the same perspective as a humanoid's head cameras, dramatically reducing the observation-space domain gap.
- **Action-space alignment**: human hands share morphology with dexterous robot hands. Using 3D hand-tracking as the action representation gives a natural common action vocabulary without robot-specific action labels.
- **Scalability**: a simple four-camera stereo cap (two front-facing, two down-facing cameras, connected to a Raspberry Pi) captures ∼5 hours of data per person per day without interrupting normal work. At ∼20 devices, the project collected over 1,000 hours with a target of 10,000 hours.

The action representation used is **48-dimensional**: 9-DoF wrist pose + 3 fingertip positions × 5 fingers = 24 DoF per hand. During pretraining the model learns to predict *discrete action tokens* representing this hand motion, obtained by compressing the continuous 48-D vector with a fine-tuned **FAST tokenizer** (DCT → quantisation → BPE), reducing it to 13 tokens.

Note

Egocentric data sets: the Eagle dataset (89 hrs, captured with Apple Vision Pro) was used for the Size Zero pretraining experiments. Down-facing cameras in the custom cap additionally enable whole-body pose estimation via SAM-3D, recovering leg and torso motion even outside the camera's field of view.

46.7.4 Two-stage training for humanoid VLAs

Stage 1 — VLM pretraining on egocentric data. The VLM backbone is trained on egocentric video + text descriptions with a discrete action-token prediction objective (next-token prediction on hand-motion tokens). This is analogous to causal language modelling but on motor sequences rather than text.

Stage 2 — Action expert post-training on robot teleoperation data. The pre-trained VLM backbone is *frozen*. A separate action expert module is conditioned on the frozen VLM's feature representations and trained with a **flow matching** objective on 30 hrs of in-domain humanoid teleoperation data (Humanoid Everyday dataset; see below).

46.7.5 MM-DiT action head

A key architectural innovation is replacing the *naïve DiT head* used by most VLA models with an **MM-DiT (multi-modal diffusion transformer) head**:

- **Naïve DiT**: VLM features and action tokens are mixed only at the very end of the DiT module (late fusion). The self-attention operates only over action tokens.
- **MM-DiT**: VLM feature tokens and action tokens are *concatenated at the beginning* of the module, and cross-attention is converted to **joint (full) self-attention** over the combined sequence. This performs channel-wise and spatial feature mixing throughout the entire depth of the module, dramatically improving alignment between visual context and action generation.

MM-DiT heads are the standard in video generation (Stable Diffusion 3, Flux) but were not widely used in VLA models before Size Zero. The ablation shows a large performance gain over the naïve DiT baseline. If you are working on VLA models, use the MM-DiT head.

46.7.6 Whole-body controller deployment

At inference time:

- **Upper body** (arms + hands): the VLA outputs joint targets; a *multi-target inverse kinematics (IK)* solver converts wrist-pose predictions into full arm joint angles.
- **Lower body** (legs + base): an AMO-style *reinforcement-learning locomotion policy* handles walking and balance, receiving high-level velocity commands from the VLA output simultaneously.

For smooth real-time execution see the action-chunking discussion in Chapter 15 (Inference).

46.7.7 Results: Size Zero vs. baselines

Evaluated on eight whole-body loco-manipulation tasks on a Unitree G1 humanoid:

- **+40% over Gr00T N1.6** (previous state-of-the-art) on task success rate.
- **10× less pretraining data**: Size Zero uses ∼800 hrs of egocentric video; Gr00T N1.6 uses >10,000 hrs.
- VLMs consistently outperform imitation-learning policies (Diffusion Policy, DP3) confirming the value of large-scale pretraining.

46.7.8 Humanoid Everyday dataset

A companion data paper (*Humanoid Everyday*, ICRA/IROS 2026) features:

- 10,000+ trajectories, 3 million frames, 260 tasks across Unitree G1 and H1.
- Diverse coverage including bipedal loco-manipulation and human-robot interaction.
- High-frequency 30 Hz multi-modal streams: RGB, depth, LiDAR, joint states, tactile, odometry, IMU, and hand actions.
- Control loop latency reduced from 500 ms to 20 ms via I/O offloading.
- Cloud evaluation platform: users upload policies; the lab evaluates on real hardware and returns success rates (no physical robot required).

HumanDex: IMU-based whole-body teleoperation

Intuition

VR-based teleoperation requires expensive headsets and constrains the operator. An IMU (inertial measurement unit) motion-tracker system can achieve comparable data quality with a lighter-weight, more portable setup that does not limit the operator's mobility or productivity.

HumanDex replaces the VR headset with body-worn IMU trackers (wrist, torso, legs) plus a Manus 5-finger glove for hand capture, enabling whole-body teleoperation of a humanoid with 5-finger dexterous hands (Wuji hands).

46.8.1 Learning-based hand retargeting

A 5-finger dexterous hand has ∼20 DoF, but the IMU setup only provides 5 fingertip positions. Mapping from 5 fingertip signals to 20-DoF joint commands is under-determined. Two approaches:

- **Optimisation-based IK**: fast but fails on unusual hand poses (awkward joint configurations where no analytical solution exists).
- **Learning-based MLP retargeting**: collect 20,000 frames in 20 min using the IK solver, filter out failure cases (abnormal joint angles, failed pinch motions), then train a small MLP to learn the mapping. The learned network handles configurations where IK fails, achieving higher teleoperation success rates.

46.8.2 Human data augmentation

Because humans perform tasks faster than teleoperating a robot, HumanDex also explores augmenting robot training data with human demonstrations:

- An egocentric camera is worn in a pose matched to the humanoid's head camera, aligning the observation space.
- Robot proprioceptive states are approximated using retargeted hand joint estimates.
- Policies trained on a *mixture* of human demonstrations and robot teleoperation data consistently outperform policies trained on robot data alone.

Part XI

Safety & Interpretability

AI Safety Landscape & Threat Models

Alignment problem overview

Reward hacking & Goodhart's law

Deceptive alignment

Containment & control

The in-agent safety problem. Current agent deployments rely heavily on *in-context* safety mechanisms: system prompts that say "don't do X", tool descriptions with usage restrictions, and behavioural alignment instilled through RLHF/DPO. These are soft controls — they live inside the model's reasoning and can fail under:

- **Prompt injection:** malicious instructions embedded in observed content (web pages, emails, file contents) that hijack the agent's action stream.
- **Context overflow:** safety instructions scrolling out of the effective attention range in long sessions.
- **Reasoning drift:** the model convincing itself an exception is justified through multi-step chain-of-thought.

Out-of-process policy enforcement is the architectural response. Rather than relying solely on the model's internal reasoning to stay within bounds, a runtime layer *outside* the agent intercepts actions before they execute and enforces hard policy at the OS or network level. The model cannot override a kernel-level syscall filter by reasoning; it simply receives a blocked-action error.

Real-world implementations:

- **OpenShell (NVIDIA/NemoClaw):** policy engine + sandbox manager + privacy router. Uses Landlock, seccomp, and network namespaces. Routing decisions are based on operator policy files, not agent intent.
- **Cowork (Anthropic):** per-session Ubuntu VM with scoped workspace mount, product-level prohibited-action lists, and `permissionMode` tiers in the Claude Code agent runtime.
- **Browser security model analogy:** both approaches mirror how browsers sandbox tabs — each session is isolated, cross-origin requests are blocked by default, and permissions (camera, clipboard, filesystem) require explicit grants.

Principle of minimal footprint. Anthropic's stated design principle for agentic systems: prefer reversible over irreversible actions, request only necessary permissions, avoid storing sensitive information beyond immediate need, and confirm with users when scope is unclear. This is the soft-control complement to hard runtime enforcement.

The emerging consensus is that robust agent containment requires *both*: aligned model behaviour (soft) to handle the common case, plus an out-of-process runtime layer (hard) as the enforcement backstop when soft controls are insufficient.

Training-time safety: data exclusion & perplexity verification

A complementary safety strategy operates at training time rather than at inference time: deliberately **excluding dangerous data from the training corpus** so the model never acquires the capability

in the first place.

Rationale. Runtime filters and RLHF refusal training can suppress a capability the model already has, but suppression can be bypassed via jailbreaks or fine-tuning. If the dangerous capability was never learned, there is nothing to unsuppress.

Examples in practice.

- **EVO 2 (biological foundation model):** all eukaryotic virus sequences were excluded from the 9-trillion-base-pair training dataset, including pathogens on the USDA Select Agents and Toxins list. The model was therefore never trained to model the statistical patterns of dangerous viruses. See Chapter 51.
- **CSAM prevention:** image generation models routinely filter known CSAM hashes from training data as a non-negotiable pre-processing step.
- **Weapons synthesis:** curated pretraining data pipelines for chemistry LLMs may exclude detailed synthesis routes for scheduled substances.

Verifying exclusion works: perplexity probing. Removing data from a training corpus does not guarantee the model has no residual representation of the excluded domain (e.g. from related sequences or indirect references). **Perplexity probing** is the standard verification technique:

1. Collect a held-out probe set of examples from the excluded domain (e.g. viral genome sequences).
2. Feed these to the trained model and measure per-token perplexity.
3. **High perplexity** ⇒ the model has no internal model of these sequences; exclusion was effective.
4. **Low perplexity** ⇒ the model has acquired representations of the excluded domain via indirect exposure; exclusion was incomplete.

For EVO 2, viral sequence inputs produced high perplexity, and generation attempts produced biological gibberish — confirming successful exclusion.

Note

Perplexity probing is the dual of perplexity-based contamination detection in eval methodology (Chapter 45): there, *low* perplexity on benchmark examples reveals the model *has* seen them. Here, *high* perplexity on excluded data confirms the model *has not*. Same technique, opposite direction.

Data exclusion is necessary but not sufficient. Releasing open-source training code alongside a virus-excluded dataset still allows a malicious actor to obtain the excluded data independently and fine-tune their own version of the model. The EVO 2 authors acknowledge this dual-use risk explicitly.

Idempotency for agents: An action is *idempotent* if executing it twice produces the same result as executing it once. Critical when agents retry failed actions on real-world systems:

- Non-idempotent retries produce duplicate side-effects: two emails sent, two orders placed, two support tickets created.
- Mitigation: pass an *idempotency key* (unique ID per logical action) so the backend can detect and deduplicate retries. Standard practice in payment APIs (Stripe) and REST design (`PUT`/`DELETE` are idempotent; `POST` is not).
- Agent design rule: tools that mutate real-world state should be idempotent by design *or* require explicit human confirmation and must never be retried automatically without an idempotency key.

Interpretability & Mechanistic Analysis

Mechanistic interpretability (MI) aims to reverse-engineer neural networks into human- understandable algorithms. Rather than treating a model as a black box, MI tries to identify the *circuits*, *features*, and *representations* the model has learned and explain *why* it produces particular outputs.

Probing classifiers

A **probing classifier** trains a simple linear (or shallow MLP) classifier on top of a frozen model's internal activations to test whether a particular concept is linearly encoded at a given layer.

- High probe accuracy ⇒ the concept is linearly represented at that layer. Low accuracy ⇒ either the concept is not encoded, or it is encoded non-linearly.
- **Limitation:** a probe showing accuracy does not prove the model *uses* that representation for its downstream behaviour. Probing is a necessary but not sufficient condition for functional relevance.
- Common probes: part-of-speech tags on intermediate layers, world-state probes in game-playing models (Othello-GPT), truth probes on factual knowledge.

Superposition hypothesis

Neural networks represent far more features than they have neurons. The **superposition hypothesis** (Elhage et al. 2022, Anthropic) explains how:

- Features (interpretable properties of the world) are encoded as *directions in activation space*, not as individual neurons.
- The same neuron participates in encoding multiple features simultaneously (**polysemanticity**): a single neuron may respond to "banana", "yellow", *and* "the word 'the' ".
- By spreading features across nearly-orthogonal directions in a high-dimensional space, the network can represent $N \gg d$ features in a d-dimensional activation space. The cost is interference between co-activating features.
- Features with lower frequency and higher importance get cleaner, more dedicated representations; rare but important features can avoid superposition.

Intuition

Superposition is the neural-network analogue of compressed sensing: you can recover N sparse signals from $d < N$ measurements if the signals are rarely co-active. The network exploits the sparsity of natural concept co-occurrence.

Sparse autoencoders (SAE)

Sparse autoencoders are the primary tool for *decomposing* superposed representations into interpretable monosemantic features.

Architecture. An SAE has an encoder that maps an activation vector $\mathbf{x} \in \mathbb{R}^d$ to a sparse latent $\mathbf{f} \in \mathbb{R}^m$ ($m \gg d$), and a decoder that reconstructs $\hat{\mathbf{x}}$:

$$\mathbf{f} = \mathrm{ReLU}(W_e\mathbf{x} + \mathbf{b}_e), \quad \hat{\mathbf{x}} = W_d\mathbf{f} + \mathbf{b}_d$$

Trained with an L_2 reconstruction loss plus an L_1 sparsity penalty on $\mathbf{f}$. The L_1 penalty forces each activation to be explained by only a few features.

Interpretation. Each dimension of $\mathbf{f}$ corresponds to a "feature"; the decoder column $W_d[:, i]$ is that feature's direction in activation space. If a feature's activating inputs cluster around a semantically

coherent concept (e.g. "Python function definitions", "references to DNA"), it is labelled as that concept.

Applied SAEs to a one-layer transformer. Found thousands of interpretable features including curve detectors, high-frequency features, and multimodal neurons. Validated that SAE features are causally relevant: steering on a feature's direction induces the corresponding behaviour.

Scaled SAEs to Claude 3 Sonnet (a large production model) and extracted ∼34 million features. Found high-level abstract concepts including features for the "Assistant" token that encode intent and agency. Demonstrated that feature steering on safety-relevant features (e.g. "deception") produces consistent behavioural changes.

Tooling.

- **SAELens**: open-source library for training, analysing, and evaluating SAEs across model families. Provides standardised training pipelines and interpretability metrics.
- **Neuronpedia**: web platform for browsing trained SAE features, viewing max-activating dataset examples, and testing feature steering.

Circuit-level analysis

A **circuit** is a minimal subgraph of model components (attention heads, MLP neurons, residual stream paths) that is sufficient to explain a particular model behaviour.

Methodology.

1. Identify a *task* (e.g. indirect object identification: "John told Mary that __ liked ice cream" → "she").
2. Use **activation patching** (see below) to identify which components causally matter for the task.
3. Trace the information flow through the residual stream to characterise what each component contributes.

Canonical circuits found in transformers:

- **Induction heads** (Olsson et al. 2022): two-layer circuit that copies patterns from earlier context (`[A] [B] ... [A]` → predict `B`). Mechanistically implement in-context learning.
- **Indirect Object Identification (IOI)** (Wang et al. 2022): a 26-head circuit in GPT-2 medium that correctly identifies indirect objects in sentences like "When John and Mary went to the store, John gave a book to..."

Circuit-Tracer is an automated tool for discovering circuits without manual intervention; it uses gradient-based attribution to identify the most impactful paths through the computation graph.

Activation patching

Activation patching (also called *causal tracing*) is an intervention technique that identifies which activations causally mediate a model behaviour.

Protocol:

1. Run the model on a *clean* input (correct behaviour) and a *corrupted* input (incorrect behaviour, e.g. name swap). Record all activations.

2. Run the corrupted input again, but *patch* the activation at a specific layer and token position by substituting the clean activation.
3. Measure how much the model's output recovers toward the clean behaviour. A large recovery means that activation is causally important.

Used causal tracing on GPT-2 and GPT-J to localise factual associations to specific MLP layers in the middle of the network. Used this insight to design Rank-One Model Editing (ROME), which directly overwrites factual associations in those MLP weight matrices without fine-tuning.

Path patching extends the technique by patching not just a single activation but a specific *path* between two components, enabling more precise circuit decomposition.

Activation patching can identify correlation between a component and a behaviour, but determining the full mechanism still requires understanding *what* the component computes, not just *that* it matters. Patching is the localisation step; circuit analysis is the explanation step.

Part XII

Evaluation

Evaluation Methodology & Benchmark Design

Evaluation taxonomy

Evaluations exist on two orthogonal axes: *what* is being measured and *how* it is measured.

What is being measured:

- **LLM output quality:** fluency, coherence, factual accuracy of a single response. Standard metrics: BLEU, ROUGE, BERTScore, perplexity.
- **Agent task success:** did the agent complete the goal? Fundamentally different from output quality — a response can be fluent but wrong; a task can succeed via a suboptimal path. Metrics: task completion rate, step accuracy, tool-call efficiency, latency, cost per run.
- **Safety and alignment:** does the model/agent refuse correctly, avoid harmful outputs, resist injection, and respect privacy constraints?
- **Robustness:** does performance hold under paraphrasing, adversarial inputs, or distribution shift?
- **Efficiency:** latency, token count, cost per query or task.

How it is measured:

- **Exact match:** strict string equality. Fast and unambiguous; brittle for open-ended tasks.
- **Functional correctness:** test-based evaluation — does the output pass unit tests? Standard for code evals (SWE-bench, Aider Polyglot, HumanEval).
- **Rubric scoring:** a fixed rubric grades outputs on specific dimensions. Can be applied by human raters or an LLM judge.
- **LLM-as-judge:** a stronger model rates the output against a rubric. Scalable; inherits the judge model's biases and inconsistencies.
- **Human rating:** annotators assess quality. Expensive but sets the gold standard for nuanced, open-ended tasks.
- **End-to-end environment:** agent runs in a sandboxed environment and outcomes are verified programmatically (WebArena, OSWorld, SWE-bench).

Key distinction: output quality vs task success

A model can generate a fluent, coherent answer that fails the actual task. A model can succeed at the task via an ugly, verbose path. For agents, always prefer task-success metrics over output-quality proxies.

Contamination & data leakage

Benchmark contamination occurs when examples from an evaluation benchmark appear (exactly or near-exactly) in the model's training data. This inflates measured performance beyond the model's true generalisation capability.

Why it happens:

- Web-scale pretraining datasets inevitably contain benchmark text published online, in papers, or in tutorials.
- Synthetic data pipelines (where a stronger model generates training examples) can inadvertently reproduce benchmark questions when the teacher model has seen them.
- Fine-tuning on large scraped corpora (GitHub, Stack Overflow, coding forums) can pull in code that is identical to benchmark test cases.

Why it is hard to catch:

- Near-duplicates (paraphrased, variable-renamed) are harder to detect than exact copies and can still give unfair advantage.
- Developers may not check for contamination until after a model "beats" a benchmark — at which point the incentive to look is low.
- Some contamination is partial: a model may have seen the problem statement but not the ground-truth answer, giving a weaker but real advantage.

Detection methods:

- **N-gram overlap:** check training data for n-gram matches with benchmark items. Computationally cheap; misses paraphrases.
- **Perplexity probing:** the model assigns lower perplexity to text it has memorised. Benchmark items with unusually low perplexity are suspect.
- **Canary strings:** embed unique strings in private benchmarks; if they appear in training data, contamination is confirmed.
- **Held-out splits:** maintain a private benchmark partition never published online.

After training a coding model and achieving a score that appeared to beat ChatGPT's ChatGPT-4 on Aider Polyglot, the author discovered small amounts of contamination in his training data. He discarded the benchmark result and retrained on clean data — demonstrating that contamination is easy to miss even in small, manually curated datasets. The final clean score (39.1%) was ultimately higher, but the contaminated result was invalid regardless of the magnitude of overlap.

Long-context evaluation: needle-in-a-haystack

Standard benchmarks test short-context reasoning. A model that scores well on MMLU might still "forget" information placed 100K tokens earlier in its context window. **Needle-in-a-haystack (NIAH)** is the canonical stress test for long-context fidelity.

Protocol.

1. Fill the context with a large volume of semantically neutral "haystack" text (e.g. randomly generated prose, or a large document corpus).
2. Insert a short, specific "needle" — a unique fact or sequence — at a controlled position within the haystack.
3. Ask the model to retrieve the needle. Vary two dimensions:
   - **Context length:** test at 1K, 4K, 16K, 64K, 128K, 1M tokens.
   - **Depth:** position the needle at the very start, middle, or end of the context.

Findings. Most models show a characteristic "lost in the middle" failure mode: retrieval accuracy drops sharply when the needle is placed in the middle of a long context, while start and end positions

are retrieved more reliably. This is because transformers distribute attention unevenly across positions — recency bias and primacy bias both apply.

Intuition

A model that scores 99% on short-context benchmarks may score 60% on NIAH at 128K tokens. For agents that must track facts across long tool-use sessions, NIAH performance is a practical reliability metric — not just a toy benchmark.

Multi-needle variants. Extend the test to multiple needles of varying relevance to probe selective retrieval rather than simple recall.

Domain adaptation. NIAH generalises beyond text. The EVO 2 biological foundation model used a genomic NIAH test: a specific 100-nucleotide sequence was embedded inside 1 million random DNA base pairs; EVO 2 retrieved it perfectly, validating that its million-token context window was functional, not just nominal. See Chapter 51.

Human vs automated evaluation

	Automated	**Human**
Speed	Fast (seconds–minutes)	Slow (hours–days)
Cost	Cheap	Expensive
Scale	Unlimited	Limited by annotator pool
Nuance	Low–medium	High
Consistency	High (deterministic)	Variable (inter-rater variance)
Coverage	Easy to run at every PR	Typically periodic
Best for	Regression testing, CI	Gold standard, edge cases

Automated eval strategies:

- **Unit-test / functional:** execute generated code and check test passage. High fidelity for coding tasks; requires a sandboxed execution environment.
- **Regex / exact match:** good for factual Q&A with known answers.
- **LLM-as-judge:** a stronger model scores outputs on a rubric. Practical at scale; biased toward its own output style and sensitive to prompt phrasing. Should be validated against human judgements.
- **Reference-free metrics:** BERTScore, METEOR measure semantic similarity to a reference; useful but imperfect for open-ended generation.

Human eval strategies:

- **Pairwise comparison:** raters choose which of two outputs is better. Easier and more reliable than absolute ratings; enables Elo-style leaderboards (e.g. LMSYS Chatbot Arena).
- **Rubric-anchored rating:** raters score on a scale (1–5) against explicit criteria. Inter-rater agreement must be measured (Cohen's κ).
- **Task completion:** rater attempts to complete a task using the model's output. Produces a binary or partial-credit score.

Regression testing workflow: When a model, prompt, or retrieval config changes, run the full automated eval suite before and after and compare distributions. Flag regressions in specific categories (coding, reasoning, safety) before deploying. Human review is reserved for cases where automated metrics disagree or a novel failure mode is suspected.

Statistical significance in evals

Benchmark scores are noisy. LLMs are non-deterministic at temperature > 0, many benchmarks use sampling to select a subset of problems, and small dataset sizes amplify variance. A single reported number is rarely sufficient.

Real-world variance observed: On the Aider Polyglot coding benchmark, scores for the same model checkpoint varied across runs — 14%, 15%, and 16% from consecutive runs, and top-line frontier model trackers show week-to-week swings of 5–10 percentage points on identical benchmark versions. This is not model degradation; it is inherent stochasticity.

Best practices:

- **Report distributions, not points.** Run $n \geq 5$ trials; report mean and standard deviation. "$25 \pm 2\%$" is more honest than "25%".
- **Statistical tests.** For comparing two model versions, use a paired t-test or bootstrap confidence interval on per-example scores. A 1-point improvement is rarely significant on a 100-item benchmark.
- **Enough items.** A 50-item benchmark has very wide confidence intervals. $n \geq 500$ items is a reasonable floor for reliable comparisons.
- **Fixed temperature / seed.** For reproducibility, fix temperature to 0 (greedy decoding) or fix the random seed. Report which was used.
- **Never cherry-pick runs.** Running the benchmark 10 times and reporting only the best result ("god run") is a form of data dredging — the expected maximum of 10 samples is substantially higher than the true mean.

Running a benchmark repeatedly and reporting the highest score is statistically invalid. If the true mean performance is 25% and you run 10 trials, the expected maximum will be several points higher simply by chance. Always report mean $\pm$ std across a fixed number of trials decided in advance.

Standard Benchmarks

MMLU, HellaSwag, ARC

HumanEval, SWE-bench

MT-Bench & Chatbot Arena

MATH, GSM8K

GPQA & frontier benchmarks

Evals for Agents & Agentic Tasks

WebArena, OSWorld

AgentOps & observability

AgentOps (agent operations) is the emerging discipline of managing the full lifecycle of autonomous agents in production, analogous to DevOps for software and MLOps for models.

Why agents are harder to operate than models:

- Non-deterministic: same input can produce different action sequences.
- Autonomous: they initiate actions, chain tasks, and make decisions without per-step human approval.
- Multi-turn: errors compound across steps; a wrong action in step 3 may only surface in step 10.
- Multi-system: agents call external APIs, databases, and sub-agents; failure can originate anywhere in the chain.

AgentOps lifecycle phases:

1. **Development:** specify objectives and constraints, map task dependencies, define data pipelines and expected tool schemas.
2. **Testing:** evaluate agent performance in a sandbox environment with simulated tool responses before production exposure.
3. **Monitoring:** observe results at session, trace, and span levels. Track actions taken, API calls made, latency, total duration, and LLM cost per run.
4. **Feedback:** log agent mistakes and surface them for review; mechanisms for triggering a corrected re-run or updating instructions.
5. **Governance:** guardrails and policies constraining agent behaviour; regulatory compliance (GDPR, HIPAA); audit trails for accountability.

IBM / watsonx observability stack (OpenTelemetry-based):

- Built on OpenTelemetry (OTEL) standards for vendor-neutral instrumentation.
- Automatic and manual instrumentation across major frameworks (LangChain, CrewAI, AutoGen, watsonx Orchestrate, OpenAI Agents SDK).
- **Session replays:** step-by-step observation of an agent run, including every tool call, intermediate result, and reasoning step.
- **Multi-trace workflow views:** see how a task flows across multiple sessions, agents, and tool calls in a single dashboard.
- **Cost analysis:** trace LLM token spend across providers (Azure, AWS, IBM) per session or per task type.

- **Trajectory exploration:** inspect which paths through the agent's action space led to success vs. failure.
- AI-powered analytics layer for iterative benchmarking toward optimisation.

LangSmith (LangChain's eval and observability platform):

- Automatically traces all model and tool calls to spot errors and latency outliers.
- Supports testing performance under different model configurations without code changes.
- Not limited to LangChain; integrates with any LLM application.
- **LLM-as-a-judge evaluators:** use a stronger model to score agent outputs on correctness, helpfulness, and safety criteria.
- Aims to bridge the gap between prototype performance and production-quality reliability.

AgentOps ecosystem (17+ tools as of 2024): Agenta, LangSmith, TruLens, AgentOps.ai, Weights & Biases Weave, Arize Phoenix, Braintrust, Helicone, and others. Most focus on trace capture, cost tracking, and evaluation scoring.

SWE-bench Verified

Tool-use evals

Evaluating tool-using agents is fundamentally different from evaluating static question-answering models:

- **Trajectory correctness:** does the agent reach the right answer *via the right sequence of tool calls*? A correct answer via a lucky wrong path is not a reliable success.
- **Tool selection accuracy:** does the agent choose the appropriate tool for each sub-task, and avoid calling tools unnecessarily?
- **Argument quality:** are the arguments passed to tools well-formed, correctly typed, and semantically appropriate?
- **Error recovery:** when a tool returns an error, does the agent diagnose and adapt, or loop in confusion?
- **Efficiency:** how many tool calls does the agent make vs. the minimum required? Excess calls inflate cost and latency.

Long-horizon task evaluation

Long-horizon tasks span many steps, multiple tools, and potentially multiple sessions. Evaluation is harder because:

- **Partial credit:** an agent that completes 8 of 10 required steps is meaningfully better than one that completes 2, but binary pass/fail metrics miss this. Step-level scoring is preferred.
- **State coupling:** later steps depend on earlier ones; an error early in the trajectory can make all subsequent steps fail even if the agent's logic is otherwise sound.
- **Non-determinism:** re-running the same task may produce different action sequences. Eval suites should run multiple trials and report distributions, not single results.
- **Sandboxed environments:** long-horizon evals (WebArena, OSWorld, SWE-bench) provide controlled environments with real or simulated backends so agents can take real actions without affecting production systems.
- **Human-in-the-loop baselines:** comparing agent task completion rate and time-to-completion against human workers provides the most meaningful benchmarks for enterprise use cases.

Safety evals for agents

Agent safety evaluation goes beyond behavioural alignment to cover the agent's interaction with real-world resources:

- **Prompt injection resistance:** does the agent correctly refuse to follow instructions embedded in tool results, web pages, or documents? Measured by injecting adversarial instructions into observed content and checking whether the agent acts on them.
- **Minimal-footprint compliance:** does the agent request only the permissions it needs, prefer reversible over irreversible actions, and avoid storing sensitive data unnecessarily?
- **Refusal accuracy:** does the agent correctly refuse prohibited actions (permanent deletion, credential entry, sharing permissions) while not over-refusing legitimate requests?
- **Escalation / confirmation behaviour:** does the agent pause and request human approval at the right points (novel situations, irreversible actions, high stakes) rather than proceeding autonomously?
- **Data leakage:** does the agent avoid routing sensitive context to unapproved model backends or external endpoints? (See also OpenShell privacy router, Chapter 36.)

Agent failure modes

Understanding failure modes drives the most impactful eval improvements.

- **Hallucination:** the agent asserts facts not supported by evidence. Mitigations: RAG grounding, citation requirements, fact-check subagent, "unknown" fallback when evidence is absent.
- **Looping:** the agent repeats a tool call or reasoning cycle without progress. Mitigations: max-step limit, repeated-state detection, explicit stop conditions, or upfront planning (ReWOO).
- **Tool misuse:** wrong tool, malformed arguments, or over-calling. Mitigations: strong tool descriptions (Chapter 20), schema validation, explicit routing rules.
- **Context drift:** the agent loses track of the original goal across many steps. Mitigations: persistent task state, goal re-injection, summarised intermediate outputs.
- **Over-delegation:** spawning too many subagents for a task that could be handled directly, inflating cost and latency. Mitigation: delegate only when specialisation gives a measurable quality win; budget subagents.
- **Sycophancy:** telling the user what they want to hear rather than the accurate answer. Particularly dangerous in report-generation and evaluation agents. Measured by checking whether the agent contradicts demonstrably false user premises.

Part XIII

Frontiers

Neurosymbolic AI & Program Synthesis

Neural-symbolic integration

Intuition

Connection to Berkeley Week 8: The course frames this as the fundamental interface between *learned/statistical reasoning* (neural) and *structured/logical reasoning* (symbolic). Robust AI agents need both: neural systems to handle perception, ambiguity, and learning from data; symbolic systems to enforce constraints, enable compositionality, and provide interpretability. Neurosymbolic AI seeks to combine their complementary strengths.

Motivation for Integration: Pure neural systems excel at pattern recognition and learning from data, but suffer from lack of explicit reasoning, compositionality, and interpretability—they treat the learned function as a black box. Pure symbolic systems are logically sound and transparent, but are brittle (require perfect knowledge engineering) and don't learn from data or handle perceptual ambiguity.

Two key failure modes motivate integration:

- **Neural hallucination:** Neural systems generate outputs that violate logical constraints or domain knowledge. A vision model might identify an object as "cat" despite strong spatial or contextual evidence to the contrary.
- **Symbolic brittleness:** Symbolic systems fail to handle noisy, ambiguous, or high-dimensional perceptual input. A rule-based reasoner requires crisp, discrete facts and cannot gracefully degrade with partial information.

Key Integration Approaches:

1. **Learning + Reasoning Pipeline** (neural perception → symbolic reasoner)

   Neural network produces a learned representation or embedding. A symbolic reasoner then operates on this representation to reach conclusions.

   - **Neuro-Symbolic Concept Learner (NS-CL):** Learn object attributes (color, size, shape) from visual input via a neural network, then apply symbolic logical rules to reason about relationships. The neural part is trained end-to-end by backpropagating through the symbolic reasoning layer.
   - Advantage: reasoning is interpretable and provably correct once attributes are extracted; learning is data-driven.

2. **Differentiable Logic** (make symbolic operations learnable)

   Embed symbolic operations in a differentiable framework so they can be trained with gradient descent.

   - **Differentiable Theorem Proving:** Represent logical inference as a differentiable computation graph. A learned "attention weight" can guide which axioms and rules to apply at each step, optimizing the proof search.
   - **Neural Theorem Provers:** Use a neural network to suggest the next proof step in an interactive theorem prover; a symbolic verifier (e.g., Lean) checks correctness. Only valid

steps are accepted, ensuring soundness.

- Advantage: combines the scalability and generalization of neural search with the soundness guarantee of symbolic verification.

3. **LLMs as Symbolic Reasoners** (generate symbolic programs for execution)

   Use an LLM to generate a symbolic program (e.g., logical rules, pseudocode, formal specification) that is then executed by a formal interpreter, which provides feedback.

   - **Program-Aided Language Models (PAL):** Query the LLM to write Python code that solves a math or reasoning problem. The code is executed, and the interpreter output (correct or error message) serves as ground truth. Intermediate results guide the LLM's reasoning.
   - **Tree of Thought with Symbolic Verifier:** Explore a tree of candidate solutions (generated by the LLM), using a symbolic verifier or test suite to evaluate each branch. Prune invalid branches; expand promising ones.
   - Advantage: LLMs are powerful at code and language generation; symbolic execution provides deterministic, verifiable results.

4. **Constraint Satisfaction** (symbolic filter on neural outputs)

   Run a neural model (e.g., LLM) unconstrained, then apply symbolic constraints as a post-processing filter or during decoding.

   - **Constrained Decoding:** Use an SMT solver or type checker to restrict the LLM's next-token distribution to only those tokens that maintain syntactic validity or satisfy domain constraints.
   - Example: in generating a SQL query, constrain the decoder to only emit tokens that keep the query syntactically valid (balanced parentheses, valid column names).
   - Advantage: leverages the LLM's flexibility while guaranteeing structural correctness.

System 1 vs. System 2 Framing: A useful cognitive analogy: **System 1** (fast, intuitive, pattern-matching) maps to neural processing; **System 2** (slow, deliberate, rule-based reasoning) maps to symbolic processing. Neurosymbolic AI aims to combine both—rapid intuitive judgments refined by careful deliberate reasoning. This mirrors human cognition: we use pattern recognition for quick assessment, then apply logical rules when a decision matters.

Program induction

Goal and Definition: Program induction is the task of learning a *program*—a structured, executable representation with formal semantics—from examples, rather than learning a flat neural function. The inferred program should generalize beyond the training examples.

Program Synthesis: The broader term for generating code that satisfies a specification. Specifications can be input/output examples, natural language descriptions, or formal logical specifications.

Key Approaches:

1. **Search-Based Synthesis**

   Enumerate candidate programs from a domain-specific language (DSL) and check which ones satisfy the specification.

   - **FlashFill (Microsoft Excel):** User provides a few input/output examples. The system searches a DSL of string operations (substring, regex, concatenation) to find a program that matches all examples and generalizes.

- **PROSE Framework:** General-purpose synthesis engine; can learn string transformations, data wrangling rules, and more. Combines search with ranking heuristics to prioritize likely programs.
- Advantage: guaranteed correct (checks against examples); disadvantage: search space is exponential; works best for small, well-defined DSLs.

2. **Inductive Logic Programming (ILP)**

   Learn logical rules (Horn clauses) from positive and negative examples. A rule is "good" if it covers all positives and few negatives.

   - Example: given facts ("parent(tom, bob)", "parent(bob, ann)") and positive examples ("ancestor(tom, ann)"), infer the rule "ancestor(X, Y) :- parent(X, Y) or ancestor(X, Z), parent(Z, Y).".
   - Advantage: results are human-readable and interpretable.

3. **Neural Program Synthesis**

   Train a neural model (e.g., a transformer or RNN) to generate programs.

   - **Guided Search:** The neural model ranks or prioritizes candidate programs. Promising programs (those passing intermediate tests) are expanded; others are pruned.
   - **Direct Generation:** The neural model directly generates a program as text (similar to a language model). Training uses execution feedback: if the generated program passes tests, the sequence is reinforced; if it fails, backtrack or adjust.
   - Advantage: scales to large program spaces; learns patterns from large corpora of code.

4. **DreamCoder**

   An alternating two-step algorithm:

   - **Step 1 (Synthesis):** Given a library of primitive functions, synthesize programs that solve the current task set.
   - **Step 2 (Learning):** Analyze the synthesized programs to identify recurring substructures. Add the most useful substructures as new library functions (abstraction). These become available for future synthesis.

   Iteration drives *compression*—the library becomes more expressive and specialized to the domain, making future synthesis faster and more elegant. Inspired by cognitive development: humans learn building-block skills that enable solving more complex tasks.

5. **Execution-Guided Synthesis**

   Generate a partial program, execute it to compute intermediate results, and use those results to guide generation of the next step. Critical for long-horizon code generation.

   - Example: to synthesize a data wrangling program, first run the initial transformation (e.g., "load CSV"), observe the intermediate table, then generate the next step (e.g., "filter rows where column > 5") with full visibility of the data shape.
   - Advantage: avoids dead-end paths (can detect errors early); exploits concrete intermediate feedback.

Program Induction vs. Synthesis:

- **Induction:** Infer a general rule or principle from examples (data-centric).
- **Synthesis:** Generate specific code that satisfies a specification (spec-centric).

The terms are often used interchangeably, but induction emphasizes learning abstractions from data, while synthesis emphasizes satisfying a constraint.

AlphaCode & code generation

AlphaCode (DeepMind, 2022):

AlphaCode was the first LLM to achieve competitive-programmer level performance on Codeforces, reaching the top 54th percentile—roughly equivalent to a strong intermediate-level competitive programmer.

> **Note**
>
> For context: Codeforces is a platform hosting algorithmic programming contests. Problems range from simple implementation to complex combinatorial optimization. Success requires strong algorithmic knowledge, ability to handle edge cases, and skill in debugging.

Key Innovations:

1. **Massive Code-Specific Pretraining**

   AlphaCode was pretrained on a large, carefully curated dataset of code (GitHub + Codeforces problem solutions and contest history). This focused domain expertise is crucial: general-purpose language models underperform on competitive coding.

2. **Problem Understanding**

   Fine-tune a transformer encoder on problem statements (natural language descriptions + constraints). The encoder produces a rich embedding of the problem. The decoder is conditioned on this embedding when generating candidate solutions.

3. **Large-Scale Sampling**

   Generate a very large number of candidate solutions per problem (1M+ samples). The key insight: even if the success rate is low (0.1%), quantity compensates. With 1M samples, you can expect hundreds of correct solutions.

4. **Filtering and Clustering**

   Run all generated programs against public test cases (unit tests provided with the problem). Correct solutions are retained; incorrect ones discarded. Group the correct solutions into clusters by runtime behavior or structure. Sample one solution per cluster for final submission (avoids redundancy; improves diversity of submissions).

AlphaCode 2 (2023):

Combined with Gemini, achieved top 15th percentile—a substantial improvement. Likely driven by the stronger underlying language model and continued scaling of compute.

Code as Structured Text:

A key insight distinguishes code generation from natural language generation:

- **Code has formal semantics.** It can be executed and tested. A program either passes or fails unit tests; there is an unambiguous ground truth.
- **Natural language is subjective.** Multiple answers can be correct; evaluation is fuzzy and context-dependent.

This enables *automated, unambiguous evaluation*: we can run generated code and immediately know if it works. This feedback signal is invaluable for iterative refinement.

Connection to Coding Agents:

Modern coding agents (Claude Code, Devin, SWE-agent, etc.) extend AlphaCode's ideas:

- Generate code with an LLM.
- Execute the code; capture output or error messages.
- Feed errors back to the LLM to refine the solution.
- Repeat until the code passes all tests or the agent exhausts its attempts.

This is a form of *execution-guided program synthesis*. The agent's loop is: generate → execute → error feedback → refine. Each iteration narrows the search space toward a correct solution. Unlike AlphaCode's offline filtering, agents can refactor and debug in real time.

Formal verification + LLMs

Formal Verification Primer:

Formal verification is the task of *mathematically proving* that a program, circuit, or system satisfies a specification. Unlike testing (which checks examples), verification is exhaustive—it proves correctness for all possible inputs.

Key tools:

- **Model Checking:** Exhaustively explore the state space of a system (e.g., finite automaton, hardware design). Tools: Spin, NuSMV.
- **Theorem Proving:** Construct a formal proof using axioms and inference rules. Tools: Coq, Lean, Isabelle.
- **SMT Solvers:** Decide satisfiability of logical formulas (Boolean, arithmetic, etc.). Tools: Z3, CVC5.

LLMs for Formal Proofs:

LLMs can assist in interactive theorem proving (ITP) by suggesting proof steps. The key is that an automated verifier checks each step; only valid steps are accepted, ensuring soundness.

Critical Safety Property: LLMs are unreliable at proof generation and often hallucinate invalid steps. However, when an LLM is tightly coupled with a *verifier*, the verifier rejects invalid steps. The system as a whole is sound: every accepted step is mathematically correct, even if the LLM sometimes generates garbage.

Key Systems:

1. **Lean + LLM**

   Lean is an interactive theorem prover. A proof is built by applying tactics (e.g., "intro", "simp", "rw") to simplify the goal. An LLM (via agents or decision trees) suggests which tactic to apply next. Lean's kernel verifies the tactic is legal; if not, the tactic is rejected and the LLM tries again.

   Hyper Tree Proof Search (HTPS): A search algorithm that explores the proof tree using an LLM to evaluate and expand promising nodes. The verifier ensures only valid proof steps are recorded.

   AlphaProof (DeepMind, 2024): Combined tree search and LLM. Achieved remarkable results on International Mathematical Olympiad (IMO) geometry and algebra problems, with fully formal proofs generated in Lean. Proofs are not just correct by machine check; they are human-readable and insights can be extracted.

2. **MathLib (Lean 4)**

   A large, curated library of formalized mathematics (thousands of theorems and definitions in Lean). Serves as: (1) a training corpus for proof-generating models, and (2) a foundation for new proofs (reuse existing theorems).

3. **LLMs as Specification Writers**

   Before verifying a system, you must specify its desired behavior formally (e.g., in temporal logic, Z notation, or TLA+). LLMs can help translate natural language requirements into formal specs:

   - Engineer writes: "The system must allow any user to add items to their cart and checkout without losing data."
   - LLM generates a TLA+ spec that formalizes this constraint.
   - A model checker verifies the system against the spec.

   Advantage: LLMs accelerate the costly specification-writing step.

Constrained Decoding for Correctness:

During code or proof generation, use SMT solvers, type checkers, or grammar constraints to restrict the LLM's output to syntactically and semantically valid tokens:

- **Type Checking:** Only emit tokens that maintain type safety (useful in Lean or other typed languages).
- **SMT Constraints:** If the proof step must satisfy a logical constraint (e.g., the variable must be positive), constrain the decoder to only suggest steps that satisfy the constraint.

This dramatically reduces the LLM's hallucination rate by ruling out invalid outputs before they are generated.

Limitations and Open Questions:

- **Hallucination Still Happens:** Even with verification, LLMs frequently generate invalid proof steps. The verifier rejects them, but the search process must backtrack, slowing convergence.
- **Exponential Search Complexity:** Proof search is exponential in problem difficulty. For very hard problems (like hard IMO problems), even with LLM guidance, the search space is enormous. Success requires careful problem reduction and domain-specific heuristics.
- **Generalization:** Most systems are trained on specific domains (IMO geometry, software verification). Generalization to new domains is unclear.
- **Human Insight:** The most impressive results (AlphaProof on IMO) combine LLM guidance with symbolic search and external tools (e.g., computational geometry for angle-chasing). Pure neural or pure symbolic approaches fall short.

Research Reading Log & Paper Notes

How to use this chapter

This chapter is a running log of papers read in depth. Each entry follows the template below. Cross-references to the relevant technical chapters are given where the concepts are treated more formally.

Paper template

- **Title / Authors / Venue / Year**
- **One-sentence summary**
- **Key claims and findings**
- **Technical approach**
- **Limitations / open questions**
- **Cross-references**

Reading list

One-sentence summary. A 40-billion-parameter biological foundation model trained on 9 trillion DNA base pairs that learns the "grammar" of life purely from raw sequences, enabling zero-shot mutation effect prediction, cancer variant classification, and *de novo* generation of functional genomes.

Framing. DNA is a four-letter alphabet (G, C, A, T) read in triplets (codons). Just as LLMs learn language from raw text with next-token prediction, EVO 2 applies the same causal language-modelling objective to nucleotide sequences. The difference: instead of internet text, the training signal is 9 trillion base pairs from the **Open Genome 2** dataset spanning bacteria, archaea, fungi, plants, and animals.

Architecture. EVO 2 uses a hybrid long-convolution / attention architecture (in the **Striped-Hyena** family) achieving a **1-million-token context window** at single-nucleotide resolution. Transformer self-attention alone would require $O(n^2)$ memory for $n = 10^6$ — infeasible. The Hyena convolutional operator (related to SSMs; see Chapter 48) reduces this to near-linear complexity, enabling whole-gene and whole-chromosome context.

Key findings.
1. Zero-shot mutation effect prediction. The model was never given labels about which mutations are harmful. Yet it correctly flags:

- **Start/stop codon mutations** — changing the start codon (ATG) prevents protein synthesis; changing a stop codon causes read-through. EVO 2 assigns these very low probability of existing in nature.
- **Shine-Dalgarno / Kozak sequences** — ribosome landing-pad sequences flanking the start codon in bacteria (Shine-Dalgarno) and eukaryotes (Kozak/KAC). Mutations in these regions are correctly flagged as highly damaging.
- **Synonymous vs. frameshift mutations** — synonymous: one letter changed, same amino acid encoded (benign, correctly recognised); frameshift: insertion/deletion shifts every downstream codon (catastrophic, correctly flagged).
- **Ciliate code divergence** — ciliates reassigned the stop codon TGA to an amino acid. EVO 2 infers this non-standard code from local sequence context without being told the organism's identity.

2. Why does zero-shot work? The implicit supervision signal is **evolutionary selection**: every

sequence in the dataset belongs to a living organism. Lethal mutations are absent from the gene pool by definition. By learning the distribution over 9 trillion base pairs, EVO 2 learns that conserved sequences are functionally essential and that unseen sequences are likely deleterious. Evolution provides the free label.
3. Human cancer variant classification. Tested on ClinVar BRCA1/BRCA2 data (pathogenic vs. benign variants). EVO 2 had never seen medical records or cancer annotations, yet correctly classified clinically dangerous BRCA mutations — showing evolutionary conservation directly correlates with disease-causing variants.
4. De novo genome generation.

- **Human mitochondria** (≈16,000 bp): generated a complete novel mitochondrial sequence. MitoZ analysis confirmed correct gene architecture (protein-coding genes, tRNAs, rRNAs). AlphaFold 3 further validated that generated proteins folded correctly and physically interlocked.
- ***Mycoplasma genitalium*** (≈580,000 bp): complete functional genome of the smallest free-living bacterium.
- ***Saccharomyces cerevisiae*** (≈12 Mbp): complete eukaryotic genome — yeast is phylogenetically closer to humans than to bacteria.

Safety. All eukaryotic virus sequences were excluded from training data (including USDA Select Agents). Verification used **perplexity probing**: feeding dangerous viral sequences to the trained model yielded high perplexity (the model had no learned representation of those sequences), and generation attempts produced incoherent gibberish. See Chapter 42.

Open source. Weights (40B, 82 GB), Open Genome 2 dataset (virus-excluded), training/fine-tuning code — all on GitHub and Hugging Face.

Limitations.

- Eukaryotic chromosomes (>100 Mbp) exceed the 1M-token context window.
- Learned from observed diversity, not first-principles biochemistry; may miss scenarios outside the training distribution.
- Open-sourcing the training pipeline creates a dual-use risk: a malicious actor could fine-tune on restricted viral sequences to produce a capable bioweapon model.

Cross-refs. Ch 48 (SSMs / Hyena), Ch 44 (zero-shot probing), Ch 42 (training-data exclusion as safety), Ch 11 (evolutionary signal as implicit supervision).

One-sentence summary. A humanoid whole-body controller that uses egocentric human video for VLM pretraining, an MM-DiT action expert fine-tuned on robot teleoperation data, and training-time action chunking for smooth real-time deployment — achieving a 40% improvement over the prior state of the art (Gr00T N1.6) using 10× less pretraining data.

Framing. Locomotion in humanoid robots is largely solved by simulation-based RL. The hard remaining problem is *loco-manipulation*: executing dexterous, language-conditioned manipulation tasks while the robot is mobile. Current VLA models target ∼18 DoF tabletop arms; the Unitree G1 humanoid has ∼43 DoF. Size Zero addresses both the data and model bottlenecks simultaneously.

Key findings.

1. **Egocentric human video as pretraining data**: cheaper and better-aligned (viewpoint + action space) than internet video or simulation. 5 hrs/person/day with a simple stereo cap; 1,000 hrs collected, targeting 10,000 hrs.
2. **FAST tokenizer for action discretisation**: continuous 48-D hand vectors (9-DoF wrist + 3 fingertips × 5 fingers) compressed to 13 discrete tokens via DCT → quantisation →

BPE; enables efficient pretraining with a standard language modelling objective.

3. **MM-DiT action head outperforms naïve DiT**: concatenating VLM and action tokens at the start of the diffusion transformer and using joint self-attention (vs. late fusion) is the single biggest model-architecture win. Borrowed from video generation (SD3, Flux); underused in robotics.
4. **Two-stage training**: frozen VLM backbone (pre-trained on egocentric video) + flow-matching action expert (fine-tuned on 30 hrs of Humanoid Everyday teleoperation data).
5. **Training-time action conditioning (TTAC)**: resolves the action-chunk boundary jitter problem at inference time by conditioning the model on already-executing actions and simulating inference delay during training. See Ch 15 for details.

Deployment architecture. Upper body: multi-target IK from predicted wrist poses. Lower body: AMO-style RL locomotion policy. Client (robot at 30 Hz) ↔ server (GPU) with dual control/inference loops for latency-free execution.

Humanoid Everyday companion dataset. 10,000+ trajectories, 3M frames, 260 tasks on Unitree G1 and H1; 30 Hz RGB + depth + LiDAR + joint states + tactile + IMU. Control loop latency 20 ms (down from 500 ms via I/O offloading). Includes a cloud evaluation platform for labs without physical robots.

HumanDex side contribution. IMU-based whole-body teleoperation (no VR headset). Learning-based MLP retargeting from 5 fingertip IMUs to 20-DoF hand joints outperforms optimisation-based IK on unusual poses. Human demonstration data augments robot data to improve policy performance.

Limitations / open questions.

- Evaluation environment is not fully zero-shot: tasks and backgrounds are seen in training. Object positions and orientations vary but not the task type itself.
- Force/torque feedback not yet incorporated; limits performance on compliant grasping tasks.
- Scaling egocentric data beyond lab scale requires crowd-sourcing or data vendors.
- The model handles 260 tasks; complex long-horizon household tasks (e.g. full meal preparation) remain open.

Cross-refs. Ch 26 (embodied AI, VLAs, whole-body control, HumanDex), Ch 15 (TTAC / real-time action chunking), Ch 09 (FAST tokenizer / action discretisation), Ch 12 (two-stage fine-tuning), Ch 24 (world models as policy evaluators).

Part XIV

Reference

University AI Modules

CS2109S — AI and Machine Learning

Machine learning algorithms, neural networks, and AI applications.

CS3263 — Foundations of AI

Search algorithms, knowledge representation, and reasoning.

CS4246 — AI Planning and Decision Making

Automated planning, reinforcement learning, decision-theoretic approaches.

MA4275 — Mathematics for Reinforcement Learning

Mathematical foundations: dynamic programming, Markov decision processes, value functions.

General-purpose LLM + Agent Map

How to use this chapter

This chapter is a navigation layer for the rest of these notes: quick definitions, how concepts connect, and where each idea belongs (training, inference, or agent systems).

Where each concept belongs in this repo

LLM training and adaptation

- **Pretraining** (Chapter 11): learning broad language/world patterns from large raw corpora.
- **Fine-tuning, SFT, LoRA** (Chapter 12): specialization for domain/task behavior.
- **RLHF / preference optimization** (Chapter 13): shaping behavior with preference signals.
- **Distillation** (cross-cutting, mostly training): transferring behavior from teacher to student.

Inference and deployment

- **Inference and decoding** (Chapter 15): temperature, top-k, top-p, caching, batching.
- **Quantization** (Chapter 16): reducing numerical precision for speed and memory.
- **RAG** (Chapter 18): retrieval pipeline to ground generation at inference time.

Agent systems

- **Agent fundamentals** (Chapter 19): loop, planning, acting, reflection.
- **Tool use / function calling / MCP** (Chapter 20): external actions and integrations.
- **Memory and planning** (Chapters 22–23): state, persistence, and decision flow.

Distillation vs Quantization (high-value distinction)

- **Distillation**: changes what the model learns by training a smaller student to imitate a larger teacher.
- **Quantization**: changes how the learned weights are stored/executed by using lower precision.

Rule of thumb: distillation is mostly a *training-time* method; quantization is mostly an *inference/deployment-time* method.

Distillation: practical trade-offs

Pros

- Better cost-latency-quality trade-off than training small models from scratch.
- Good path to task-specific students (coding, math, support workflows).
- Cleaner teacher outputs can provide a strong learning signal.

Cons

- Teacher errors and style artifacts transfer to the student.
- Student capability ceiling is often tied to teacher quality.
- Overusing synthetic outputs can reduce diversity and robustness.

Quantization: practical intuition + numeric example

Quantization compresses weight values into fewer buckets.

Bucket example

Suppose a layer's weights in one tensor range from -1.0 to 1.0, and we use 4-bit quantization. 4-bit gives $2^4 = 16$ buckets.
Bucket width is:

$$\Delta = \frac{1.0 - (-1.0)}{16 - 1} = \frac{2.0}{15} \approx 0.1333$$

For a weight $w = 0.18372642$:

$$q = \text{round}\left(\frac{w - (-1.0)}{\Delta}\right) = \text{round}\left(\frac{1.18372642}{0.1333}\right) = 9$$

Dequantized value:

$$\hat{w} = -1.0 + 9 \cdot 0.1333 \approx 0.1997$$

So the exact value 0.18372642 is stored as bucket index 9, then reconstructed approximately as 0.1997.

Why this helps

- Lower memory footprint (VRAM/RAM)
- Faster inference on many hardware targets
- Enables larger models on local/edge devices

What can degrade

- Multi-step reasoning, coding precision, and long-context stability can drop at aggressive bit-widths.

Minimal glossary (highest-yield)

Pretraining
Learn broad priors from large raw corpora.

Fine-tuning
Specialize a pretrained model.

Distillation
Teach a smaller model from a stronger model.

Quantization
Compress numeric representation for efficiency.

RAG
Retrieve documents first, then generate grounded answers.

Agent
A model-driven system that plans and uses tools.

Agent types taxonomy

Five canonical agent types by increasing capability (Russell & Norvig):

Simple reflex agent
Condition-action rules only. No memory, no world model. Works solely in fully observable deterministic environments. Example: thermostat, traffic light.

Model-based reflex agent
Maintains an internal world model updated each turn; handles partial observability by tracking unobserved state. Still uses condition-action rules for decisions.

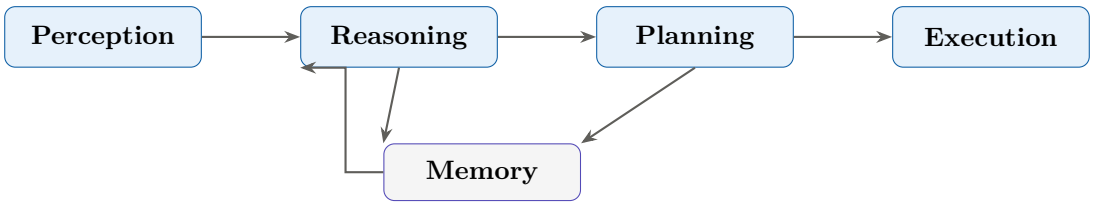
Goal-based agent
Has explicit goals; uses search or planning to find action sequences that achieve them. Requires reasoning about future states. Example: route-finding robot.

Utility-based agent
Extends goal-based agents with a utility function (real-valued desirability per state). Enables rational choice when goals conflict or are achievable to varying degrees.

Learning agent
Improves performance from experience. Components: performance element (acts), critic (evaluates), learning element (updates), problem generator (explores). LLMs trained via RLHF/DPO are learning agents at training time; at inference time they are typically goal- or utility-based.

Agent architecture components

A complete agent system integrates five functional subsystems:



- **Perception:** visual (CV), auditory (ASR), textual (NLP/NER), environmental (sensor fusion), and predictive (anticipating future states).
- **Reasoning:** context understanding, pattern recognition — the LLM forward pass, optionally with chain-of-thought (Chapter 17).
- **Planning:** goal-setting, decision-making. ReAct, ReWOO, MCTS (Chapter 23).
- **Execution:** API calls, tool invocations, code execution (Chapter 20).
- **Memory:** working (context window), episodic (logs), semantic (vector DB), procedural (RL / fine-tuning) (Chapter 22).

One mental model

- Pretraining gives the base capability.
- Fine-tuning aligns it to a job.
- Distillation lowers serving cost.
- Quantization lowers runtime footprint.
- RAG adds external knowledge at inference time.
- Agent tooling turns answers into actions.